

Circuit-Succinct Algebraic Batch Arguments from Projective Functional Commitments

David Balbás^{1,2}, Dario Fiore¹, and Russell W. F. Lai³

¹ IMDEA Software Institute, Madrid, Spain.

david.balbas@imdea.org

dario.fiore@imdea.org

² Universidad Politécnica de Madrid, Madrid, Spain.

³ Aalto University, Espoo, Finland.

russell.lai@aalto.fi

Abstract. Batch arguments for NP (BARGs) are non-interactive proof systems that allow a prover to convince a verifier that k NP statements $\mathbf{x}_1, \dots, \mathbf{x}_k$ are valid relative to some circuit $\mathcal{C}$, i.e. there exist witnesses $\mathbf{w}_i$ such that $(\mathbf{x}_i, \mathbf{w}_i)$ satisfy $\mathcal{C}$ for all i , while the proof size remains sublinear in k . Most existing BARG constructions achieve a proof size of $|\pi| = \text{poly}(\lambda, |\mathcal{C}|, \log k)$ for large or not explicitly specified poly acting on $|\mathcal{C}|$, with two exceptions:

1. Devadas et al. and Paneth and Pass’s “rate-1” constructions [FOCS’22] achieve $|\pi| = |\mathbf{w}| + \mathcal{O}(|\mathbf{w}|/\lambda) + \text{poly}(\lambda, \log k)$ (with matching verification time for Paneth and Pass), but for not explicitly specified poly due to non-black-box use of cryptographic primitives.
2. Waters and Wu’s algebraic (pairing-based) construction [Crypto’22] achieves $|\pi| = \mathcal{O}(\lambda \cdot |\mathcal{C}|)$.

In this work, we give the first algebraic (pairing-based) construction of BARG that achieves proof size and online verifier runtime $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$. We achieve our result by means of a compiler which builds a BARG generically from a *projective chainable functional commitment* (PCFC), which supports somewhere extraction, subvector projection, and functional openings. We then construct a PCFC from the standard MDDH assumption in bilinear groups by building on top of the functional commitment for circuits by Wee and Wu [Eurocrypt’24]. Our black-box transformation may be of independent interest for understanding the connection between functional commitments and BARGs and towards obtaining other algebraic constructions of the latter.

1 Introduction

Succinct non-interactive arguments (SNARGs) [Mic94] allow a prover to convince a verifier of a statement’s validity with a short and quickly verifiable proof. For example, they can prove an NP statement $\mathbf{x}$ checkable by a circuit $\mathcal{C}$, i.e., that $\exists \mathbf{w} : \mathcal{C}(\mathbf{x}, \mathbf{w}) = 1$, with a proof of size $\text{poly}(\lambda, \log |\mathcal{C}|)$. They are a fundamental concept in the theory of proof systems and an essential tool in practical applications like outsourcing of computation, blockchains, and cryptocurrencies.

A central question about succinct arguments is the assumptions required for their construction. Gentry and Wichs provided a significant answer by proving that it is impossible to construct adaptively-sound SNARGs for NP with a black-box reduction to falsifiable assumptions [GW11].

Batch Arguments (BARGs). In search of stable foundations for this primitive, various lines of research have explored alternative methods to circumvent this impossibility result and to construct succinct arguments from standard assumptions. One notable approach is to *relax the full power of SNARGs*, while ensuring that they remain useful in applications. For example, a series of works considered SNARGs for the class P of *deterministic polynomial-time computations*, which have a direct application to verifiable computation, and showed how to construct them from standard falsifiable assumptions [KPY19, GZ21, JKKZ21, CJJ22].

In this work, we focus on another restricted form of SNARGs — batch arguments for NP (BARGs) — which achieve a relaxed notion of succinctness. In a BARG scheme, one can prove a collection of NP statements, $\mathbf{x}_1, \dots, \mathbf{x}_k$, i.e., $\forall i = 1, \dots, k, \exists \mathbf{w}_i : \mathcal{C}(\mathbf{x}_i, \mathbf{w}_i) = 1$, with a proof that scales *sublinearly in the number of statements*, e.g., of size $\text{poly}(\lambda, |\mathcal{C}|, \log k)$, but not necessarily sublinearly in the circuit size. Due to its distinctive properties – succinct proofs and realizations from standard falsifiable assumptions – BARGs are emerging as a useful tool to realise other cryptographic primitives from standard assumptions. Notably, for several primitives that have simple folklore realizations from SNARGs, recent works have shown that the SNARG can be replaced by a BARG. This approach is exemplified by applications of BARGs to non-interactive delegation [BHK17, KPY19, KVZ21, CJJ22], aggregate signatures [WW22, BCJP24], and homomorphic signatures [ABF24, Goy24, ACG24].

Non-Black-Box vs. Algebraic. In the state of the art, the main approach to construct BARGs involves non-black-box combinations of cryptographic primitives, with instantiations that often rely on probabilistically checkable proofs (PCPs) and correlation intractable (CI) hash functions [CGH98, CCH⁺19, PS19, JJ21]. This direction has led to constructions from LWE [CJJ22, DGKV22, PP22] or subexponential DDH [CGJ⁺23]. Notably, the rate-1 constructions of [DGKV22, PP22] achieve a proof size (and verification time for [PP22]) of $|\pi| = |\mathbf{w}| + \mathcal{O}(|\mathbf{w}|/\lambda) + \text{poly}(\lambda, \log k)$, which is sublinear in $|\mathcal{C}|$. However, despite the near-optimal asymptotic efficiency, these constructions often have large concrete overheads, i.e. the poly functions in proof size and verification time expressions are often large or not explicitly specified.

An exception to the non-black-box approach is the work of Waters and Wu [WW22] who gave direct algebraic constructions over bilinear pairings based on the decision linear (DLin) assumption. Making black-box use of the underlying algebraic structure often translates into more concretely efficient and relatively simple constructions. Indeed, the scheme of [WW22] achieves a proof size of $|\pi| = \mathcal{O}(\lambda \cdot |\mathcal{C}|)$. The result is later improved by [GLWW24, CEW25], both of which reduce the public parameter size while keeping essentially the same design. However, in both [WW22, GLWW24] the verifier runs in time $\Omega(\lambda \cdot |\mathcal{C}|)$, even if offline preprocessing is allowed.

1.1 Our Contribution

In this work, we continue the line of work established by [WW22, GLWW24, CEW25] on obtaining algebraic constructions of BARGs. Specifically, we present a new construction of BARG directly over bilinear pairings under the matrix decisional Diffie-Hellman (MDDH) assumption, which achieves proofs of size $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$ with matching online verification time, improving upon [WW22, GLWW24, CEW25].

Additionally, our BARG natively supports proving k different circuits, one for each statement. In prior work, this property is achieved with the overhead of using a universal circuit $\mathcal{U}$ to define the batch language and including the description of each circuit in the statement, i.e., by proving that $\mathcal{U}((\mathcal{C}_i, \mathbf{x}_i), \mathbf{w}_i) = 1$.

To achieve the above result, we rely on a new notion that we call *projective chainable functional commitments (PCFC)*, which is a combination of projective commitments (PC) [GZ21, WW24] and chainable functional commitments (CFC) [BCFL23], together with two technical ingredients:

1. A black-box compiler turning PCFCs with certain properties to BARGs; and
2. An algebraic construction of such a PCFC based on the MDDH assumption.

We believe that our black-box compiler might be of independent interest for understanding the connection between functional commitments (FCs) [LRY16] and BARGs, which relax SNARGs by forgoing soundness and succinctness respectively. It might also pave the way towards obtaining other direct algebraic constructions of BARGs from different assumptions, such as lattices, which is an open problem.

1.2 Other Related Work

The usual BARG succinctness notion requires that the proof size is bounded by $\text{poly}(\lambda, |\mathcal{C}|, \log k)$, where k is the size of the batch. An interesting exception in the state of the art is the BARG for NP of Garg et al. [GSWW22] in which proofs have size $\text{poly}(\lambda, \log |\mathcal{C}|, \log k)$, that is sublinear also in the size of the circuit. The scheme, built from indistinguishability obfuscation and one-way functions, is proven non-adaptively sound. Therefore, it does not bypass the impossibility result of [GW11]. In contrast, our result achieves adaptive soundness, in line with most of the BARG literature.

Another approach to build BARGs from other cryptographic primitives is that of Kalai et al. who show how to build BARGs for NP from *flexible SNARGs for RAM with partial input soundness*, additionally assuming the existence of rate-1 string OT [KLVW23]. On one hand, the spirit of this connection is similar to ours, especially after observing that their notion of SNARG for RAM is similar to that of functional commitments. On the other hand, their BARG construction differs from ours in two main aspects. First, it is non-black-box due to the recursive usage of the SNARG for RAM. Thus, it cannot yield algebraic BARGs, departing from one of the main motivations of our work. Second, they do not provide a direct construction of flexible SNARGs for RAM but show that, in turn, these can be built from BARGs (again, additionally assuming rate-1 string OT). While this result establishes an interesting connection between the two primitives, it ultimately does not yield new BARGs since the flexible SNARG for RAM should be instantiated from existing BARGs and thus from either PCPs and CI-hash, or from [WW22].

2 Technical Overview

The main result of this work is an algebraic construction of BARG with proof size and online verification time $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$. We obtain this result by compiling, in a black-box manner, a new commitment primitive that we call *projective chainable functional commitment* (PCFC), for which we give an algebraic construction over bilinear groups.

To give a high-level overview of our approach, in Section 2.1, we begin by recalling and generalizing the notions of projective commitments (PC), [GZ21, WW24] and chainable functional commitments (CFC) [BCFL23], and introducing the notion of PCFC. We then revisit in Section 2.2 the algebraic BARG construction of Waters and Wu [WW22] by viewing it as a black-box compilation from a PCFC to a BARG. From this abstract perspective, it becomes clear how, given a PCFC with stronger properties, the compiler can be tweaked to improve the proof size and online verification time from $\mathcal{O}(\lambda \cdot |\mathcal{C}|)$ to $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$. We thus present in Section 2.3 how to obtain such a PCFC from pairing groups based on the MDDH assumption. Finally, in Section 2.4 we conclude with a summary of our results and an outlook on future work.

2.1 Projective Chainable Functional Commitments (PCFC)

A (deterministic) commitment scheme allows the generation of a short commitment com_x which is computationally binding to a vector $\mathbf{x} \in \mathcal{M}^\ell$ potentially much longer than com_x . Many variations of

the notion exist that extend the basic functionality and security of a commitment scheme. Our new notion, PCFC, which we formalise in Section 4, is a combination of two such variations: projective commitments (PC) and chainable functional commitments (CFC).

Projective Commitments (PC). In a *Projective commitment (PC)* scheme, introduced by Wee and Wu [WW24]⁴, the commitment key ck of the scheme can be sampled in two modes: the *normal mode* and the *projective mode*. Sampling the key in normal mode leads to a standard commitment key ck that computationally binds a vector $\mathbf{x} \in \mathcal{M}^\ell$ to a commitment com . In projective mode, the commitment key ck is sampled together with a trapdoor td that encodes a hidden index set $I \subseteq [\ell]$. Here, ck hides a *projective key*, which is essentially a secondary commitment key that is sampled only at the positions indexed by I . The trapdoor td allows one to project any commitment com to $\mathbf{x} \in \mathcal{M}$ to a commitment pcom to the subvector $\mathbf{x}_I \in \mathcal{M}^I$.⁵ Without knowing td , both modes are computationally indistinguishable.

In more detail, a PC features the following algorithms (among others):

- $\text{ProjSetup}(1^\lambda, 1^\ell, I)$: Generates an ordinary-looking commitment key ck together with a trapdoor td which encodes a secret index set $I \subseteq [\ell]$.
- $\text{ProjCom}(\text{td}, \mathbf{x}) \rightarrow \text{pcom}$: Generate a projective commitment pcom of the subvector $\mathbf{x}_I$.
- $\text{Proj}(\text{td}, \text{com}) \rightarrow \text{pcom}$: Project a commitment com into a projective commitment pcom .

Projective commitments are closely related to somewhere extractable commitments [HW15, CJJ21], which essentially allow one to generate commitment keys so that a commitment com_x is *statistically binding* at some positions $I \subset [\ell]$. The trapdoor td allows one to actually extract $\mathbf{x}_I \in \bar{\mathcal{M}}$ (where $\bar{\mathcal{M}} \subseteq \mathcal{M}$ is a smaller extraction space) leveraging the statistical binding property. In this work, we enhance the original notion of projective commitments to also provide somewhere extractability. This is, our commitments also feature the following algorithm:

- $\text{ProjExt}(\text{td}, \text{com})$: Given com , which is guaranteed⁶ to be committing to a vector whose positions indexed by I belong to the projective subspace $\bar{\mathcal{M}}^I$, extract $\mathbf{x}_I \in \bar{\mathcal{M}}^I$ such that $\text{ProjCom}(\text{td}, \mathbf{x}_I) = \text{Proj}(\text{td}, \text{com})$.

To achieve the extractability outlined in the above simplified description, a commitment com has to be of size linear in $|I|$, which will be too large for our application of constructing circuit-succinct BARGs. Instead, we further augment the syntax of the commitment algorithm Com (among others) to take as input another index set J chosen from some admissible set $\mathcal{J} \subseteq 2^{[\ell]}$ so that the resulting commitment com is committing to a subsector $\mathbf{x}_J \in \mathcal{M}^J$. Now, the extractability guarantee is only required to hold at the positions $I \cap J$ for any $J \in \mathcal{J}$. By picking $\mathcal{J}$ appropriately, we can ensure that $|I \cap J|$ and hence $|\text{com}|$ is never too large which eventually allows us to achieve circuit-succinctness.

Chainable Functional Commitments (CFC). While ordinary commitments are “all-or-nothing”, functional commitments (FC) [LRY16] enable the generation of a succinct (weakly sound) proof asserting the correctness of $\mathbf{y} = f(\mathbf{x})$ for the vector $\mathbf{x}$ committed in com_x . *Chainable functional commitments (CFC)* [BCFL23] extend this functionality and allow to generate proofs for $\mathbf{y} = f(\mathbf{x}_1, \dots, \mathbf{x}_k)$ where both the inputs $\mathbf{x}_1, \dots, \mathbf{x}_k$ and the output $\mathbf{y}$ are committed. A CFC is “chainable” in the sense that it allows one to prove relations between a sequence of committed inputs and outputs, in a

⁴ Similar properties for commitments appear in [GZ21] under the name of G -extractability.

⁵ The subvector $\mathbf{x}_I$ is obtained by taking the entries of $\mathbf{x}$ indexed by I .

⁶ To be (weakly) asserted by another algorithm called ProjProve and verified by ProjVer .

“commit-and-prove” fashion. A (C)FC usually satisfies a relaxed notion of soundness called evaluation binding – no efficient prover could produce valid proofs for $(f, \text{com}_x, \text{com}_y)$ and $(f, \text{com}_x, \text{com}'_y)$ for $\text{com}_y \neq \text{com}'_y$. In this work, however, we deviate from this usual notion and require a stronger property that we introduce next.

Projective Chainable Functional Commitments (PCFC). Although PC and CFC are useful notions in their own rights, combining them into a PCFC makes the result more powerful than the sum of the two, since it allows us to reason about the functional relations between projective commitments and their extracted messages, whose spaces are hidden from a malicious prover. Such a property already resembles the functionality of a BARG, as both allow to extract part of the inputs to the batch computation while verifying its correctness.

In more detail, recall that a PC commitment key ck hides a secret index set $I \subseteq [\ell]$ while a CFC allows the generation of functional proofs. To glue these two notions together, we consider a class of functions called “ I -separable functions” and define a property called *projective chain binding* with respect to these functions.

I -Separable Functions. We say that a function $f : (\mathcal{M}^\ell)^k \rightarrow \mathcal{M}^\ell$ is I -separable if the coordinates in I of the output vector only depend on the coordinates in I of each input vector.⁷ A simple example is when f is the parallel application of another function $f_I : \mathcal{M}^{\ell/k} \rightarrow \mathcal{M}^{\ell/k}$ such that $f(\mathbf{x}_1, \dots, \mathbf{x}_k) = (f_I(\mathbf{x}_1), \dots, f_I(\mathbf{x}_k))$.

Projective Chain Binding. We say that a PCFC is projective chain binding for an I -separable function f if no PPT adversary, given both a commitment key ck , which is trapdoored to be projective at I , and the trapdoor td , can produce tuples $(\text{com}, \text{com}_y, \pi)$ and $(\text{com}', \text{com}'_y, \pi')$ where

- π and π' are valid CFC proofs for $f(\text{com}) = \text{com}_y$ and $f(\text{com}') = \text{com}'_y$, respectively, and
- $\text{Proj}(\text{td}, \text{com}) = \text{Proj}(\text{td}, \text{com}')$, but
- $\text{Proj}(\text{td}, \text{com}_y) \neq \text{Proj}(\text{td}, \text{com}'_y)$.

Essentially, the projective chain binding property allows a security reduction to transfer the evaluation binding property of the CFC to hold also in the projective subspace. This is especially powerful if we leverage the somewhere extractability property of PC on the projective subspace. It is worth noting that some of the proof systems that are internally used to build the FC in [WW24] achieve a similar notion to projective chain binding (although the notion is not attained by the FC itself). In their case, however, the projection trapdoor is not available to the adversary. As we discuss in the next section, our strengthening is crucial when proving the security of our circuit-succinct BARG construction.

2.2 Black-Box Compiler from PCFC to BARG

Recall that in a BARG the prover aims to convince the verifier that k NP statements $\mathbf{x}_1, \dots, \mathbf{x}_k$ are valid. That is, the prover knows witnesses $\mathbf{w}_1, \dots, \mathbf{w}_k$ such that $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i) = 1$ for all $i \in [k]$. The main security property of BARGs, known as somewhere extractability, requires the existence of an efficient extractor which, given a valid proof, can extract a valid witness for the i^* -th instance $\mathbf{w}_{i^*}$, where i^* is a hidden index programmed into the common reference string.

⁷ Our formal definition generalizes to allow different separability sets for inputs and outputs.

Suppose the circuit $\mathcal{C}$ is such that the input-output relation of each gate $g \in \mathcal{C}$ is captured by a gate function χ_g represented as a quadratic polynomial over a field $\mathbb{F}$, and write $x_{i,g}$ for the output value from the g -th gate (this is, the g -th circuit wire) in the evaluation of $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i)$. In the following, let $\mathcal{G}_{\text{stmt}}, \mathcal{G}_{\text{wit}}, \mathcal{G}_{\text{out}}$ be the sets of wires of $\mathcal{C}$ corresponding to the statement input, the witness input and all gate outputs, respectively.

Basic Compiler. To obtain an improved algebraic BARG, a natural starting point is the algebraic BARG of Waters and Wu [WW22]. Our first step is to view⁸ their construction as black-box compiler which turns a PCFC into a BARG. We formalize this compiler in Appendix A and give a high-level overview here:

Basic Compiler (Abstraction of [WW22]):

1. The prover PCFC-commits to the evaluation of $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i)$ in a wire-by-wire fashion across all k circuits, i.e. for each wire $g \in \mathcal{G}_{\text{wit}} \cup \mathcal{G}_{\text{out}}$, they commit to the values $\mathbf{x}_g = (x_{1,g}, \dots, x_{k,g})$ of the g -th wire as com_g .
2. For each witness wire $g \in \mathcal{G}_{\text{wit}}$, compute a PC proof for the binariness of $\mathbf{x}_g$ committed in com_g , i.e. $\mathbf{x}_g \in \{0, 1\}^k$, where $\{0, 1\} = \bar{\mathcal{M}}$ plays the role of the projective subspace.
3. For each gate output wire $g \in \mathcal{G}_{\text{out}}$, compute a CFC proof for $\mathbf{x}_g$ committed in com_g being computed correctly with χ_g w.r.t. the commitments of the input wires.

Instantiating the PCFC with the implicit PCFC for quadratic functions from [WW22] yields a BARG with proof size $\mathcal{O}(\lambda \cdot |\mathcal{C}|)$, where the linear dependency on $|\mathcal{C}|$ is due to the generation of commitments and proofs in a wire-by-wire fashion.⁹

Circuit-Succinct Compiler. Without going yet into formal details, one aspect of the security proof of the basic compiler stands out as an obstacle for reducing the proof size to be sublinear in $|\mathcal{C}|$: The security reduction has to extract the values of all $|\mathcal{C}|$ wires in one of the k circuits, and verify the functional proofs of all gate functions. The extractability of all $|\mathcal{C}|$ wires of a circuit information-theoretically implies that the BARG proof must have size linear in $|\mathcal{C}|$.

Towards improving the proof size and online verification time from $\mathcal{O}(\lambda \cdot |\mathcal{C}|)$ to $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$, a crucial observation is that the security reduction does not necessarily need to extract all $|\mathcal{C}|$ wires of (the i^* -th copy of) the circuit $\mathcal{C}$ but only its input $\mathbf{w}_{i^*}$. That is, if the PCFC is expressive enough to support generation of CFC proofs for arbitrary circuits, then the burden of checking that the circuit is evaluated correctly can be delegated to the prover. Based on this observation, we obtain the following circuit-succinct compiler, which is detailed in Section 5:

Circuit-Succinct Compiler:

1. The prover PCFC-commits to the *witness input wires* of $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i)$ across all k circuits, i.e. for each witness input wire $g \in \mathcal{G}_{\text{wit}}$, they commit to the values $\mathbf{x}_g = (x_{1,g}, \dots, x_{k,g})$ of the g -th wire as com_g .

⁸ Although Waters and Wu presented the same intuition (obviously using different terminologies than PCFCs) when explaining their construction in [WW22], formally, they constructed the BARG directly in terms of algebraic operations. This results in a construction with many intertwined components, making it difficult to dissect and improve upon.

⁹ Here, we glossed over the detail that the somewhere extractability of the BARG obtained above is proven based on another property of PCFC that we call *functional extractability* instead of projective chain binding. We refer to Appendix A for details.

2. For each witness wire $g \in \mathcal{G}_{\text{wit}}$, compute a PC proof for the binariness of $\mathbf{x}_g$ committed in com_g , i.e. $\mathbf{x}_g \in \{0, 1\}^k$, where $\{0, 1\} = \bar{\mathcal{M}}$ plays the role of the projective subspace.
3. Compute a CFC proof for $f_{\mathcal{C}}(\mathbf{x}_g : g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}) = \mathbf{1}_k$ where $f_{\mathcal{C}}$ consists of k parallel copies of $\mathcal{C}$.

Assuming that the commitments $(\text{com}_g)_{g \in \mathcal{G}_{\text{wit}}}$ and the CFC proof for $f_{\mathcal{C}}$ are sufficiently succinct, we can conclude by inspection that the BARG is circuit-succinct, i.e. a proof is of size $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$.

Security of the Circuit-Succinct Compiler. Assuming that the underlying PCFC is projective chain binding, we can prove that the above black-box compiler yields a somewhere-extractable BARG. In the security experiment, we consider a trapdoored setup which programs the PCFC commitment key so that it is projective on the subspace I_{i^*} defined by the hidden index i^* of the BARG extractor. The set I_{i^*} contains the location of the statement and witness of the i^* -th instance, i.e. $(\mathbf{x}_{i^*}, \mathbf{w}_{i^*})$. Recall that a BARG proof contains commitments com_g for each witness input wire $g \in \mathcal{G}_{\text{wit}}$ across all k circuits, along with their projection proofs. The security proof proceeds in two broad steps.¹⁰

1. Appeal to PC somewhere extractability for each com_g , which ensures the extraction of a binary witness $\mathbf{w}_{i^*} \in \{0, 1\}^{\ell_w}$ and that $\text{ProjCom}(\text{td}, \mathbf{x}_g) = \text{Proj}(\text{td}, \text{com}_g)$ for each $g \in \mathcal{G}_{\text{wit}}$.
2. Use projective chain binding to ensure that $\mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}_{i^*}) = 1$.

To explain the second step in more detail, in the case $\mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}_{i^*}) = 0$, we can construct a reduction $\mathcal{B}$ from projective chain binding. For simplicity, let us ignore the statement wires $g \in \mathcal{G}_{\text{stmt}}$ (we can think of them being hardwired in $f_{\mathcal{C}}$) as they are not important for the reduction. The reduction $\mathcal{B}$ runs the malicious prover $\mathcal{A}$ to obtain the commitments com_g for all witness input wires $g \in \mathcal{G}_{\text{wit}}$ and a valid CFC proof π for $f_{\mathcal{C}}(\mathbf{x}_g : g \in \mathcal{G}_{\text{wit}}) = \mathbf{1}_k$. Using the trapdoor td , $\mathcal{B}$ can extract the i^* -th witness $\mathbf{w}_{i^*}$ from the commitments com_g . It then uses the extracted witness $\mathbf{w}_{i^*}$ together with dummy witnesses $\mathbf{w}_i = \mathbf{0}_{\ell_w}$ for the other slots $i \neq i^*$ to compute the commitments $(\text{com}'_g)_{g \in \mathcal{G}_{\text{wit}}}$ together with another CFC proof π' for $f_{\mathcal{C}}(\mathbf{x}'_g : g \in \mathcal{G}_{\text{wit}}) = \mathbf{b}$ for some $\mathbf{b} \in \{0, 1\}^k$ with $b_{i^*} = 0$. By the correctness of the PCFC, it must hold that $\text{Proj}(\text{td}, \text{com}_g) = \text{Proj}(\text{td}, \text{com}'_g)$ for all $g \in \mathcal{G}_{\text{wit}}$ and that π' is a valid CFC proof. The reduction $\mathcal{B}$ can thus output $(\text{com}_g, \text{com}'_g)_{g \in \mathcal{G}_{\text{wit}}}, \text{com}_y, \text{com}'_y, \pi, \pi'$ where $\text{com}_y = \text{Com}(\text{ck}, \mathbf{1}_k)$ and $\text{com}'_y = \text{Com}(\text{ck}, \mathbf{b})$. Naturally, this violates projective chain binding as $b_{i^*} = 0$ and hence $\text{Proj}(\text{td}, \text{com}_y) \neq \text{Proj}(\text{td}, \text{com}'_y)$. We note that it is crucial in the projective chain binding definition that the adversary is given the trapdoor td , since the reduction $\mathcal{B}$ above needs it to extract the witness $\mathbf{w}_{i^*}$.

2.3 Algebraic PCFC from Pairings

To complete our circuit-succinct algebraic BARG construction, it remains to construct an algebraic PCFC for circuits which is compatible with our compiler. Since explaining details of the construction is tedious and does not help much in gaining intuition, we focus on structural-level ideas and refer to Section 6 for the formal details. Our construction is loosely based on two existing schemes that we introduced earlier in the overview:

1. the implicit PCFC for separable quadratic functions extracted from the Waters-Wu BARG [WW22], and

¹⁰ In our formal construction there is one more step handling the aggregation of commitments. We gloss over this detail for now.

2. the projective functional commitment (PFC) scheme for circuits by Wee and Wu [WW24], which is the only existing algebraic FC for circuits with constant size proofs.

Unfortunately, these schemes are not readily available to fulfill the requirements of our compiler. A major technical challenge is to achieve our projective chain binding property in which the adversary gets access to the projection trapdoor. Notably, this property is stronger than the chain binding of [WW24] where the adversary only sees the commitment key. Indeed, the Wee and Wu’s PFC construction cannot be proven secure if the adversary has the trapdoor.

Melding Waters-Wu’s PCFC with Wee-Wu’s PFC. The PCFC implicit in Waters-Wu’s BARG [WW22] and Wee-Wu’s PFC [WW24] each already satisfy some of our requirements. In particular:

- Waters-Wu’s PCFC supports chainable functional proofs for separable quadratic functions and allows to program the commitment key with a singleton set $I = \{i^*\}$ at which the committed message is extractable (given that it belongs to the projective subspace, which in their case is the boolean set $\mathcal{M} = \{0, 1\}$).
- Wee-Wu’s PFC supports non-chainable¹¹ functional proofs for arbitrary circuits and allows for projecting commitments on prefix sets $I = \{1, 2, \dots, i^*\}$. The functional proofs are built from several specialized proof sub-systems that prove linear and quadratic relations between different types of commitments, ensuring consistency over the projective space. However, the scheme is not extractable and cannot achieve projective chain binding.

Conveniently, both schemes are pairing-based and are similarly structured. Our first step is therefore to meld these two schemes in a compatible way. At a high level, we proceed as follows.

- We redesign the Waters-Wu’s PCFC to embed it with a projective key that allow us to reason about functional binding in the projective subspace. In the original scheme, the size of the commitments is $\kappa + 1$ group elements for a small parameter κ (which parameterizes the κ -Lin assumption). This is sufficient for somewhere extractability, but seems insufficient to reason about functional relations in the projective subspace. We extend it to size 2κ , where, intuitively, we need κ group elements acting on the entire set $[\ell]$ for the normal-mode key and κ group elements acting only on I for the projective key.
- We structure the 2κ -sized key to enable the generation of functional proofs for arbitrary circuits, making it fully compatible with the proof sub-systems that compose the Wee-Wu’s PFC.¹²

Overall, we obtain a PCFC which supports functional proofs for arbitrary circuits, projecting commitments onto arbitrary (bounded cardinality) index sets $I \subseteq [\ell]$, and allows extraction at $I \cap J$ for commitments to subvectors given by J as long as $|I \cap J| = 1$. Note that although this PCFC is correct, it does not yet satisfy the projective chain binding property. To satisfy this property, we design a novel functional proof system that partially relies on two of the specialized proof sub-systems in the Wee-Wu’s PFC.

Achieving Projective Chain Binding. As mentioned above, the main technical challenge that we tackle is to achieve projective chain binding against adversaries with access to the projection trapdoor td . In more detail, in the reduction from the MDDH assumption to projective chain binding, we

¹¹ The authors discussed chainability but did not formalise the idea.

¹² In turn, this would enable us to support the functional proofs of the Wee-Wu’s PFC and obtain standard CFC evaluation binding, although this is not our end goal.

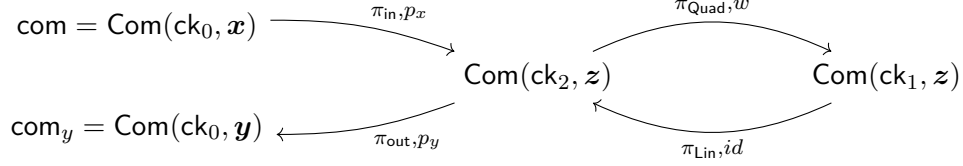


Fig. 1: Overview of the sub-proofs and commitments involved in the PCFC functional proofs.

need to employ a “sliding window” argument, which is quite common in the BARG literature (e.g. [CJJ21, KLVW23]), where we gradually shift the hidden index set I programmed in the trapdoor td from the input wires to the output wires. However, without modifying the construction, this technique contradicts with the requirement that the adversary is given access to td , since then the adversary can notice that the hidden index set I changes.

To circumvent this issue, our construction introduces three commitment sub-keys $\text{ck} = (\text{ck}_0, \text{ck}_1, \text{ck}_2)$ with their corresponding sub-trapdoors $\text{td}_0, \text{td}_1, \text{td}_2$. The first key ck_0 is the primary commitment key that is trapdoored to be projective at I (and which also enables somewhere extractability). ck_1 and ck_2 are functional keys that correspond to the so-called type-I and type-II commitments, respectively, which are required by the functional proof sub-systems of the Wee-Wu’s PFC. The idea is to hand td_0 to the adversary, while td_1 and td_2 are only used in the security proof and never given to the adversary. Then, we can enable the “sliding window” argument only over ck_1 and ck_2 , without ever modifying ck_0 . To execute this idea, we have to augment the PCFC construction to include additional consistency proofs between commitments generated against the primary key ck_0 and the functional keys ck_1, ck_2 .

Constructing our Functional Proofs. In Figure 1 we give an overview of the functional proofs and commitments involved in our PCFC construction. Recall that the end goal is to prove the evaluation of a function f (interpreted as an arithmetic circuit) given an input commitment com and an output commitment com_y , both of which are committed under ck_0 . For the description below, we let $\mathbf{z} \in \mathcal{M}^{|f|}$ be the computation trace of $f(\mathbf{x})$. This is, $\mathbf{z}$ contains the values of the $|f|$ circuit wires when f is evaluated on $\mathbf{x}$. For simplicity, we omit details related to f being a separable function that evaluates multiple copies of a circuit, and refer only to the single-circuit case. The proof contains four main components:

- A linear map sub-proof π_{in} that proves that a linear function s holds between $\text{com} = \text{Com}(\text{ck}_0, \mathbf{x})$ and a type-II commitment $\text{Com}(\text{ck}_2, \mathbf{z})$. The map s is a projection that maps the first ℓ coordinates of the input (i.e., $\mathbf{x}$) to the first ℓ coordinates of the output (i.e., the prefix of $\mathbf{z}$ corresponding to the input).
- A linear map sub-proof π_{Lin} for the identity function id between the type-I commitment $\text{Com}(\text{ck}_1, \mathbf{z})$ and the type-II commitment $\text{Com}(\text{ck}_2, \mathbf{z})$.
- A quadratic map sub-proof π_{Quad} for the *next wire* function w , which given the first t coordinates of $\mathbf{z}$ (i.e., the first t wires of $f(\mathbf{x})$), computes the $t + 1$ -th gate output wire. This is done between the type-II commitment $\text{Com}(\text{ck}_2, \mathbf{z})$ and the type-I commitment $\text{Com}(\text{ck}_1, \mathbf{z})$.
- Finally, a sub-proof π_{out} that proves an output-projection map p which maps the last ℓ coordinates of $\mathbf{z}$ to the first ℓ coordinates of $\mathbf{y}$, which is done between $\text{Com}(\text{ck}_2, \mathbf{z})$ and $\text{com}_y = \text{Com}(\text{ck}_0, \mathbf{y})$.

At a high level, the proof of projective chain binding goes as follows. The sub-proof π_{in} allows one to ensure matching projections between the input commitment com and the type-II commitment

$\text{Com}(\text{ck}_2, z)$ on the first ℓ coordinates (i.e., $I = [\ell]$). Then, π_{Lin} and π_{Quad} enable a sliding window argument where I gradually increases from $[\ell]$ to $[|f|]$, ensuring matching projections of their respective commitments. Finally, π_{out} ensures that the projection (onto $[\ell]$) of the output commitment com_y matches the projection over the last ℓ coordinates of the type-II commitment $\text{Com}(\text{ck}_2, z)$.

2.4 Discussion and Future Work

The techniques introduced above lead to a construction of PCFC over pairing groups that we summarize in the following (simplified) theorem:

Theorem 2 (simplified). *Assuming the Ker-DH assumption and the MDDH assumption over bilinear groups, the construction PCFC satisfies:*

- Somewhere extractability for a message space $\bar{\mathcal{M}} = \{0, 1\}$ if the subvector set J and the projective set I satisfy $|I \cap J| = 1$.
- Projective chain binding for the class of functions $\mathcal{F}_{k, \ell_C}$ which evaluate up to k parallel circuits $\mathcal{C}_i$, each of size at most ℓ_C .
- The commitment key size and the worst-case prover running time are $\mathcal{O}(\lambda \cdot k^5 \cdot \ell_C^5)$.
- Commitments and proofs have size $\mathcal{O}(\lambda)$.
- Efficient verification runs in time $\mathcal{O}(\lambda)$.

Overall, by combining the PCFC with the circuit-succinct compiler, we obtain a fully algebraic construction of BARGs from bilinear pairings, which we summarize as follows:

Corollary 1. *Under the assumptions of Theorem 2, there exists an algebraic somewhere extractable BARG for NP such that:*

- The setup size $|\text{crs}|$ and the prover running time are $\mathcal{O}(\lambda \cdot k^5 \cdot \ell_C^5)$.
- The proof size is $|\pi| = \mathcal{O}(\lambda \cdot |\mathbf{w}|)$.
- Efficient verification runs in time $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$.

Our BARG construction is almost concretely efficient, in the sense that all complexity measures are explicitly expressible. In particular, the resulting online verification time and proof size are $\mathcal{O}(\lambda \cdot |\mathbf{w}|)$ with small constants. The main efficiency bottleneck that remains is the size of the commitment key, which also impacts the prover time. We therefore see shrinking the commitment key size as one of the next immediate future directions.

Besides providing a formal connection between FCs and BARGs, we highlight that our BARG construction is the first *black-box* construction from another cryptographic primitive with falsifiable security notions and standard-model realizations. We believe this connection enhances the understanding of BARGs and could result in new, more efficient BARG realizations. For example, currently, there is no direct algebraic construction of BARGs from lattices, although there are direct and algebraic lattice-based FCs [ACL⁺22, WW23b, dCP23, BCFL23, WW23a], in part due to the difficulty of achieving somewhere extractability in the lattice setting. Nevertheless, we believe that our approach provides valuable insights and a candidate construction blueprint towards this goal.

3 Preliminaries

We denote by $\mathbb{N}$ the set of natural numbers > 0 . We denote the security parameter by $\lambda \in \mathbb{N}$. We call a function ϵ *negligible*, denoted $\epsilon(\lambda) = \text{negl}(\lambda)$, if $\epsilon(\lambda) = O(\lambda^{-c})$ for every constant $c > 0$, and

call a function $p(\lambda)$ *polynomial*, denoted poly , if $p(\lambda) = O(\lambda^c)$ for some constant $c > 0$. We say that an algorithm is *probabilistic polynomial time* (PPT) if it consumes randomness and its running time is bounded by some $p(\lambda) = \text{poly}(\lambda)$. For a finite set S , $x \leftarrow \$ S$ denotes sampling x uniformly at random in S . For an algorithm A , we write $y \leftarrow A(x)$ for the output of A on input x . For a positive $n \in \mathbb{N}$, $[n]$ is the set $\{1, \dots, n\}$. We denote vectors $\mathbf{x}$ and matrices $\mathbf{M}$ using bold fonts. We write $\mathbf{0}_k$ and $\mathbf{1}_k$ for the vectors of k zeros and ones respectively. Given two strings x, y , we denote their concatenation by $x|y$. We use $\mathcal{M}$ to denote the message space, and sometimes $\bar{\mathcal{M}} \subseteq \mathcal{M}$ for some message subspace, for all cryptographic primitives considered. We denote the message space of dimension ℓ by $\mathcal{M}^\ell$. To denote a (sub)vector with coordinates in a position subset $J \subseteq [\ell]$, we write $\mathbf{x}_J \in \mathcal{M}^J$.

3.1 Circuit model

Throughout this work, we model a circuit $\mathcal{C} : \mathcal{M}^{\ell_{\mathbf{x}, \mathbf{w}}} \rightarrow \mathcal{M}$, where $\ell_{\mathbf{x}, \mathbf{w}} = \ell_{\mathbf{x}} + \ell_{\mathbf{w}}$, to take as input a statement $\mathbf{x} \in \mathcal{M}^{\ell_{\mathbf{x}}}$ and a witness $\mathbf{w} \in \mathcal{M}^{\ell_{\mathbf{w}}}$, for some message space $\mathcal{M}$ of cardinality $|\mathcal{M}| \leq 2^{\text{poly}(\lambda)}$. A circuit can be arithmetic (e.g. $\mathcal{M} = \mathbb{Z}_q$) or Boolean (i.e. $\mathcal{M} = \{0, 1\}$). A circuit supports arbitrary fan-in and fan-out arithmetic (resp. Boolean) gates over $\mathcal{M}$. We adopt the convention that $1 \in \mathcal{M}$ and we say that an input $(\mathbf{x}, \mathbf{w})$ is accepted by $\mathcal{C}$ if $\mathcal{C}(\mathbf{x}, \mathbf{w}) = 1$.

We measure the size of a circuit by its total number of wires $\ell_{\mathcal{C}}$, which includes the statement and witness input wires, as well as the output wire. We associate each wire to the output of a gate $g \in [\ell_{\mathcal{C}}]$. For notational convenience, and such that the number of gates and that of wires are both $\ell_{\mathcal{C}}$, we consider that all inputs come from an “input gate” $g \in [\ell_{\mathbf{x}, \mathbf{w}}]$ which simply outputs the hardwired g -th input. Given a circuit computation $\mathcal{C}(\mathbf{x}, \mathbf{w})$, the value of the g -th wire (or the output of the g -th gate) is denoted by x_g . Each gate $g \in [\ell_{\mathcal{C}}]$ computes a function $\chi_g : \mathcal{M}^{|T_g|} \rightarrow \mathcal{M}$ on an arbitrary set of input *wires* denoted by T_g . Each gate has a single output wire, even if this can be input to multiple other gates. Finally, note that χ_g is a quadratic map, as it represents a single gate computation on a Boolean or arithmetic circuit.

To be descriptive, we use $\mathcal{G} = [\ell_{\mathcal{C}}] = \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}} \cup \mathcal{G}_{\text{out}}$ to denote the set of all gates/wires partitioned into statement, witness, internal and output wires respectively. More precisely, we enumerate the circuit gates and wires as follows:

- $\mathcal{G}_{\text{stmt}} := \{1, \dots, \ell_{\mathbf{x}}\}$ are the input wires corresponding to the values in $\mathbf{x}$.
- $\mathcal{G}_{\text{wit}} := \{\ell_{\mathbf{x}} + 1, \dots, \ell_{\mathbf{x}, \mathbf{w}}\}$ are the input wires corresponding to the values in $\mathbf{w}$.
- $\mathcal{G}_{\text{out}} := \{\ell_{\mathbf{x}, \mathbf{w}} + 1, \dots, \ell_{\mathcal{C}}\}$ correspond to the output wires of all (non-input) gates. We arrange these gates that the g -th gate only takes input wires corresponding to smaller indices, namely $T_g \subseteq [g - 1]$.

We note that the subscripts in $\ell_{\mathcal{C}}$, $\ell_{\mathbf{x}}$, $\ell_{\mathbf{w}}$, $\mathcal{G}_{\text{stmt}}$, $\mathcal{G}_{\text{wit}}$, $\mathcal{G}_{\text{out}}$, etc. are symbolic, e.g. $\ell_{\mathcal{C}}$ should not be thought of as a function of a circuit $\mathcal{C}$. We sometimes abuse notation and write $\ell_{\mathcal{C}} = |\mathcal{C}|$, $\ell_{\mathbf{x}} = |\mathbf{x}|$, $\ell_{\mathbf{w}} = |\mathbf{w}|$, etc. to denote the quantities.

3.2 Batch Arguments for NP

A Batch Argument for NP (BARG) is a proof system for a particular subclass of NP, which is the conjunction of k NP statements corresponding to the same NP language. More precisely, given an NP language $\mathcal{L}$, the prover attests that k statements $\mathbf{x}_1, \dots, \mathbf{x}_k$ belong to $\mathcal{L}$. In terms of efficiency, BARGs produce proofs π whose size is sublinear in the number of instances k . As opposed to

SNARGs, the size of a BARG proof is only required to be sublinear in k , but could be linear in the size of the circuit $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i)$ that decides the language given an input $\mathbf{x}_i$ and an NP witness $\mathbf{w}_i$.

We adopt the notion of (somewhere extractable) BARGs for circuit satisfiability from [CJJ22, KLVW23], where we generically consider circuits $\mathcal{C} : \mathcal{M}^{\ell_{\mathbf{x}, \mathbf{w}}} \rightarrow \mathcal{M}$. Such BARGs have two security properties. The first is setup indistinguishability: one can sample a trapdoored BARG crs at some index i^* , but this should be indistinguishable from a non-trapdoored crs . The second is somewhere extractability: given a valid proof for $\mathbf{x}_1, \dots, \mathbf{x}_k$ on a crs trapdoored at i^* , the trapdoor allows one to extract a valid witness $\mathbf{w}^*$ such that $\mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}^*) = 1$.

Definition 1 (BARG for NP [CJJ22, KLVW23]). *Let statement length $\ell_{\mathbf{x}} = \text{poly}(\lambda)$ and witness length $\ell_{\mathbf{w}} = \text{poly}(\lambda)$ be fixed. A somewhere extractable batch argument (BARG) for NP is a tuple of algorithms $\text{BARG} = (\text{Setup}, \text{Prove}, \text{Ver}, \text{SetupTd}, \text{Ext})$:*

$\text{Setup}(1^\lambda, 1^k, 1^{\ell_{\mathcal{C}}}) \rightarrow \text{crs}$: on input the security parameter λ , a number of instances k , a circuit size $\ell_{\mathcal{C}}$, a statement length $\ell_{\mathbf{x}}$, and a witness length $\ell_{\mathbf{w}}$, outputs a common reference string crs . Below, we implicitly assume that circuits $\mathcal{C} : \mathcal{M}^{\ell_{\mathbf{x}, \mathbf{w}}} \rightarrow \mathcal{M}$ are of size $\ell_{\mathcal{C}}$, statements $\mathbf{x} \in \mathcal{M}^{\ell_{\mathbf{x}}}$, and witnesses $\mathbf{w} \in \mathcal{M}^{\ell_{\mathbf{w}}}$.

$\text{Prove}(\text{crs}, \mathcal{C}, \{(\mathbf{x}_i, \mathbf{w}_i)\}_{i \in [k]}) \rightarrow \pi$: on input the common reference string crs , a circuit $\mathcal{C}$, and a batch of input-witness pairs $\{(\mathbf{x}_i, \mathbf{w}_i)\}_{i \in [k]}$, outputs a proof π .

$\text{Ver}(\text{crs}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi) \rightarrow b$: on input crs , a circuit $\mathcal{C}$ of size $\ell_{\mathcal{C}}$, a batch of statements $\{\mathbf{x}_i\}_{i \in [k]}$, and a proof π , accepts ($b = 1$) or rejects ($b = 0$).

$\text{SetupTd}(1^\lambda, 1^k, 1^{\ell_{\mathcal{C}}}, i^*) \rightarrow (\text{crs}, \text{td})$ works as Setup , and additionally outputs a trapdoor td associated to the index i^* that is given as input.

$\text{Ext}(\text{td}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi) \rightarrow \mathbf{w}^*$ On input a trapdoor td , a circuit $\mathcal{C}$, a batch of statements $\{\mathbf{x}_i\}_{i \in [k]}$ and a proof π , outputs a witness $\mathbf{w}^*$ corresponding to the position specified by the trapdoor.

A BARG must satisfy the following properties:

Completeness. For any $\lambda, k, \ell_{\mathcal{C}} \in \mathbb{N}$, any circuit $\mathcal{C}$, all statement-witness pairs $(\mathbf{x}_i, \mathbf{w}_i)_{i \in [k]}$ such that $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i) = 1$ for all $i \in [k]$, $\text{crs} \in \text{Setup}(1^\lambda, 1^k, 1^{\ell_{\mathcal{C}}})$ and $\pi \in \text{Prove}(\text{crs}, \mathcal{C}, \{(\mathbf{x}_i, \mathbf{w}_i)\}_{i \in [k]})$, it holds that $\text{Ver}(\text{crs}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi) = 1$.

Succinctness. For any admissible set of parameters as before, $|\pi| \leq \text{poly}(\lambda, \log k, \ell_{\mathcal{C}})$. Furthermore, we say that BARG is circuit-succinct if $|\pi| \leq \text{poly}(\lambda, \log k, \ell_{\mathbf{w}})$.

Setup Indistinguishability. For any PPT adversary $\mathcal{A}$, any $k, \ell_{\mathcal{C}} = \text{poly}(\lambda)$, and index $i^* \in [k]$, the distributions of crs induced by $(\text{crs}, \text{td}) \leftarrow \text{SetupTd}(1^\lambda, 1^k, 1^{\ell_{\mathcal{C}}}, i^*)$ and $\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^k, 1^{\ell_{\mathcal{C}}})$ are computationally indistinguishable.

Somewhere Extractability. For any PPT adversary $\mathcal{A}$, any $k, \ell_{\mathcal{C}} = \text{poly}(\lambda)$, and index $i^* \in [k]$,

$$\Pr \left[\begin{array}{l} \text{Ver}(\text{crs}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi) = 1 \\ \wedge \mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}^*) \neq 1 \end{array} \middle| \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \text{SetupTd}(1^\lambda, 1^k, 1^{\ell_{\mathcal{C}}}, i^*) \\ (\mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ \mathbf{w}^* \leftarrow \text{Ext}(\text{td}, \{\mathbf{x}_i\}_{i \in [k]}, \mathcal{C}, \pi) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition 2 (Efficient Verification for BARG). A BARG for NP BARG has efficient verification with preprocessing (also called split verification in [WW22]) if there exists a pair of algorithms:

$\text{PreVer}(\text{crs}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}) \rightarrow \text{vk}$: On input the common reference string crs , a circuit $\mathcal{C}$, and a batch of k statements $\{\mathbf{x}_i\}_{i \in [k]}$, it creates a succinct verification key vk such that $|\text{vk}| \leq \text{poly}(\lambda, \log k, \ell_{\mathcal{C}})$
 $\text{EffVer}(\text{vk}, \pi) \rightarrow b$ On input the succinct verification key vk and a proof π , accepts ($b = 1$) or rejects ($b = 0$) in time bounded by $\text{poly}(\lambda, \log k, \ell_{\mathcal{C}})$.

Furthermore, if for $\ell_{\mathcal{C}} \leq 2^{\text{poly}(\lambda)}$ the size of the verification key $|\text{vk}|$ and the runtime of $\text{EffVer}(\text{vk}, \pi)$ are bounded by $\text{poly}(\lambda, \log k, \ell_{\mathbf{w}})$, we say that BARG has a circuit-succinct verification with preprocessing.

Remark 1. In Definition 2 we consider a stronger notion of efficient verification where the circuit $\mathcal{C}$ can be preprocessed by PreVer . This allows us to define the circuit-succinct verifier where the verifier runtime is sublinear in $|\mathcal{C}|$. In prior definitions (e.g. [WW22]), the circuit $\mathcal{C}$ is not preprocessed by PreVer but is instead passed as an input to EffVer . In this case, it is impossible for the online verifier to have circuit-succinctness.

3.3 Bilinear Groups and Assumptions

A *bilinear group generator* $\mathcal{BG}(1^\lambda)$ is an algorithm that returns $\text{bgp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2)$, where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order q , $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ are fixed generators, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is an efficiently computable, non-degenerate, bilinear map. In our work we use Type-3 groups in which it is assumed that there is no efficiently computable isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$. We use the bracket notation of [EHK⁺13] for group elements: for $s \in \{1, 2, T\}$ and $x \in \mathbb{Z}_q$, $[x]_s$ denotes $g_s^x \in \mathbb{G}_s$. We use additive notation for $\mathbb{G}_1$ and $\mathbb{G}_2$ and multiplicative notation for $\mathbb{G}_T$. We note that given an element $[x]_s \in \mathbb{G}_s$, for $s = 1, 2$, and a scalar a , one can efficiently compute $a \cdot [x] = [ax] = g_s^{ax} \in \mathbb{G}_s$; given group elements $[a]_1 \in \mathbb{G}_1$ and $[b]_2 \in \mathbb{G}_2$, one can efficiently compute $[ab]_T = [a]_1 \cdot [b]_2$. For a matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$, we represent a matrix of group elements $g_s^{\mathbf{A}}$ as $[\mathbf{A}]_s \in \mathbb{G}_s^{m \times n}$.

We recall the matrix Diffie-Hellman assumption (MDDH) and the kernel Diffie-Hellman assumption (KerDH), which we adapt from [MRV16]. For MDDH we directly introduce the bilateral variant where the challenge is encoded in both $\mathbb{G}_1$ and $\mathbb{G}_2$, following the notation from [GZ21, WW24].

Definition 3 (Kernel Diffie-Hellman Assumption). Let $\text{bgp} = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2)$ be a bilinear group setting and let $\kappa, \ell, d \in \mathbb{N}$. We say that the kernel Diffie-Hellman assumption $\text{KerDH}_{\kappa, \ell}$ holds in $\mathbb{G}_1$ for bgp if for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$\Pr \left[\mathbf{Ax} = \mathbf{0} \wedge \mathbf{x} \neq \mathbf{0} \mid \begin{array}{l} \mathbf{A} \leftarrow \mathbb{F}_q^{\kappa \times \ell} \\ [\mathbf{x}]_2 \leftarrow \mathcal{A}(\text{bgp}, [\mathbf{A}]_1) \end{array} \right] \leq \text{negl}(\lambda).$$

where the probability is taken over the choice of $\mathbf{A} \leftarrow \mathbb{F}_q^{\kappa \times \ell}$ and the adversary $\mathcal{A}$'s random coins. We define $\text{KerDH}_{\kappa, \ell}$ over $\mathbb{G}_2$ analogously.

Definition 4 (Matrix Diffie-Hellman Assumption). Let $\text{bgp} = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2)$ be a bilinear group setting and let $\kappa, \ell, d \in \mathbb{N}$. We say that the (bilateral) Matrix Diffie-Hellman assumption $\text{MDDH}_{\kappa, \ell, d}$ holds over bgp if for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$\left| \begin{array}{l} \Pr [\mathcal{A}(\text{bgp}, [\mathbf{A}]_1, [\mathbf{A}]_2, [\mathbf{S} \cdot \mathbf{A}]_1, [\mathbf{S} \cdot \mathbf{A}]_2) = 1] \\ - \Pr [\mathcal{A}(\text{bgp}, [\mathbf{A}]_1, [\mathbf{A}]_2, [\mathbf{U}]_1, [\mathbf{U}]_2) = 1] \end{array} \right| \leq \text{negl}(\lambda),$$

where the probability is taken over the choice of $\mathbf{A} \leftarrow \mathbb{F}_q^{\kappa \times \ell}$, $\mathbf{S} \leftarrow \mathbb{F}_q^{d \times \kappa}$, $\mathbf{U} \leftarrow \mathbb{F}_q^{d \times \ell}$ and the adversary $\mathcal{A}$'s random coins.

4 Projective Chainable Functional Commitments

We define several variants of commitments, in particular projective chainable functional commitments (PCFC), and their properties. Extending the usual syntax of commitments, we allow the commitment algorithm to additionally input an index set $J \in \mathcal{J} \subseteq 2^{[\ell]}$ and commit to a (sub)vector $\mathbf{x} \in \mathcal{M}^J$, and extend the other algorithms accordingly. The index set J is taken as $J = [\ell]$ when omitted. Although not often considered explicitly, such a functionality is naturally supported by common commitment schemes.

Throughout this work, we assume that all commitments are deterministic and that the opening information is the committed vector itself. We also assume that all algorithms taking a commitment key ck as input has random access to ck . This is to capture schemes where the commitment key consists of many components and where not all algorithms need access to all components.

Definition 5 (Commitments). *A commitment scheme for an index set family $\mathcal{J} \subseteq 2^{[\ell]}$ is a tuple of PPT algorithms $(\text{Setup}, \text{Com})$ with the following syntax:*

$\text{Setup}(1^\lambda, 1^\ell) \rightarrow \text{ck}$: *On input the security parameter λ and the vector length ℓ , output a commitment key ck .*

$\text{Com}(\text{ck}, J, \mathbf{x}) \rightarrow \text{com}$: *On input the commitment key ck , an index set $J \in \mathcal{J}$ and a vector $\mathbf{x} \in \mathcal{M}^J$, output a commitment com .*

The index set J is taken as $J = [\ell]$ when omitted. A commitment scheme must satisfy the following property:

Binding. *For any PPT adversary $\mathcal{A}$ and vector length $\ell = \text{poly}(\lambda)$,*

$$\Pr \left[\begin{array}{l} \text{Com}(\text{ck}, J, \mathbf{x}) = \text{Com}(\text{ck}, J', \mathbf{x}') \\ \wedge (J, \mathbf{x}) \neq (J', \mathbf{x}') \end{array} \middle| \begin{array}{l} \text{ck} \leftarrow \text{Setup}(1^\lambda, 1^\ell) \\ (\text{com}, J, \mathbf{x}, J', \mathbf{x}') \leftarrow \mathcal{A}(\text{ck}) \\ J, J' \in \mathcal{J} \wedge \mathbf{x}, \mathbf{x}' \in \mathcal{M}^J \end{array} \right] \leq \text{negl}(\lambda).$$

Succinctness. *For any $\lambda, \ell \in \mathbb{N}$, $\mathbf{x} \in \mathcal{M}^\ell$, it holds that $|\text{com}| \leq \text{poly}(\lambda, \log \ell)$.*

We define a notion of aggregatability which allows to aggregate commitments of subvectors with disjoint index sets into a commitment of the union.

Definition 6 (Aggregatability). *Let $\hat{\mathcal{J}} \subset \mathcal{J}$. A commitment scheme is aggregatable if*

- *there exists a deterministic aggregation algorithm $\text{Agg}(\text{ck}, (\text{com}_J)_{J \in \hat{\mathcal{J}}}) \rightarrow \text{com}$, which takes as input ck and tuples of commitments $(\text{com}_J)_{J \in \hat{\mathcal{J}}}$ and outputs a commitment com in time at most $\text{poly}(\lambda, \log \ell) \cdot |\hat{\mathcal{J}}|$, and*
- *if $J \cap J' = \emptyset$ for all distinct $J, J' \in \hat{\mathcal{J}}$, then for any $\mathbf{x}_J \in \mathcal{M}^J$ for $J \in \hat{\mathcal{J}}$, the following property is satisfied:*

$$\Pr \left[\text{Agg}(\text{ck}, (\text{com}_J)_{J \in \hat{\mathcal{J}}}) = \text{com} \middle| \begin{array}{l} \text{ck} \leftarrow \text{Setup}(1^\lambda, 1^\ell) \\ \text{com}_J \leftarrow \text{Com}(\text{ck}, J, \mathbf{x}_J) \quad \forall J \in \hat{\mathcal{J}} \\ \text{com} \leftarrow \text{Com}(\text{ck}, \bigcup_{J \in \hat{\mathcal{J}}} J, \bigcup_{J \in \hat{\mathcal{J}}} \mathbf{x}_J) \end{array} \right] = 1.$$

Remark 2. Our aggregatability definition (Definition 6) is quite strong as it makes sense only for deterministic commitments. We believe that the definition can be relaxed, but made more complicated, to allow for randomized commitments.

4.1 Chainable Functional Commitments (CFC)

A Functional Commitment (FC) [LRY16] is a powerful primitive that allows an entity to first commit to some input $\mathbf{x} \in \mathcal{M}^\ell$ (more generally $\mathbf{x} \in \mathcal{M}^J$ for some $J \in \mathcal{J}$) and then open the commitment to $f(\mathbf{x})$ for some admissible function $f \in \mathcal{F}$. Both the commitment com and the opening proof π are succinct.

Some FC schemes may offer useful composability properties such as *chainability*. A chainable functional commitment (CFC) [BCFL23] is an extension of FC that allows one to commit to multiple inputs $\mathbf{x}_1, \dots, \mathbf{x}_m \in \mathcal{M}^\ell$ and then open to a commitment of $\mathbf{y} = f(\mathbf{x}_1, \dots, \mathbf{x}_m)$, i.e. the output $\mathbf{y}$ remains committed. A CFC generically implies an FC by simply adding an opening proof that the output commitment indeed opens to $\mathbf{y}$. Below, we follow and extend the syntax from [BCFL23] to allow each of $\mathbf{x}_1, \dots, \mathbf{x}_m, \mathbf{y}$ to be indexed by a different index set.

Definition 7 (Chainable Functional Commitments (CFC)). A chainable functional commitment (CFC) for a class of functions¹³ $\mathcal{F} = (\mathcal{F}_\ell)_\ell$ and index set family $\mathcal{J} = (\mathcal{J}_\ell)_\ell$ consists of a commitment scheme $(\text{Setup}, \text{Com})$ for $\mathcal{J}$ and additionally PPT algorithms $(\text{FuncProve}, \text{FuncVer})$ with the following syntax:

$\text{FuncProve}(\text{ck}, (\mathbf{x}_i)_{i \in [m]}, f) \rightarrow \pi$: given vectors $\mathbf{x}_i \in \mathcal{M}^{J_i}$ for $i \in [m]$ and a function $f \in \mathcal{F}$ where $f : \prod_{i \in [m]} \mathcal{M}^{J_i} \rightarrow \mathcal{M}^{J_y}$, returns an opening proof π .

$\text{FuncVer}(\text{ck}, (\text{com}_i)_{i \in [m]}, \text{com}_y, f, \pi) \rightarrow b \in \{0, 1\}$: on input commitments $(\text{com}_i)_{i \in [m]}$ for $i \in [m]$ to the m inputs and com_y to the output, opening proof π , and function $f \in \mathcal{F}$ where $f : \prod_{i \in [m]} \mathcal{M}^{J_i} \rightarrow \mathcal{M}^{J_y}$, accepts ($b = 1$) or rejects ($b = 0$).

A CFC must satisfy the following properties.

Correctness. For $\lambda, \ell, m, \in \mathbb{N}$, $J_1, \dots, J_m, J_y \in \mathcal{J}$, $f \in \mathcal{F}$ where $f : \prod_{i \in [m]} \mathcal{M}^{J_i} \rightarrow \mathcal{M}^{J_y}$, and $\mathbf{x}_i \in \mathcal{M}^{J_i}$ for $i \in [m]$, it holds that

$$\Pr \left[\text{FuncVer}(\text{ck}, (\text{com}_i)_{i \in [m]}, \text{com}_y, f, \pi) = 1 \mid \begin{array}{l} \text{ck} \leftarrow \text{Setup}(1^\lambda, 1^\ell) \\ \text{com}_i \leftarrow \text{Com}(\text{ck}, J_i, \mathbf{x}_i) \quad \forall i \in [m] \\ \text{com}_y \leftarrow \text{Com}(\text{ck}, J_y, f(\mathbf{x}_1, \dots, \mathbf{x}_m)) \\ \pi \leftarrow \text{FuncProve}(\text{ck}, (\mathbf{x}_i)_{i \in [m]}, f) \end{array} \right] = 1.$$

Succinctness. For any admissible set of parameters as before, $|\pi| \leq \text{poly}(\lambda, \log \ell, \log m, \log |f|)$, where $|f|$ denotes the size of the circuit description of f .

The security notion of CFC considered in [BCFL23] is evaluation binding, recalled for completeness in Definition 8. In this work, however, we will not rely on evaluation binding but other properties to be defined throughout this section.

Definition 8 (CFC Evaluation Binding). A CFC satisfies evaluation binding if for any PPT adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{FuncVer}(\text{ck}, (\text{com}_i)_{i \in [m]}, \text{com}_y, f, \pi) = 1 \\ \wedge \text{FuncVer}(\text{ck}, (\text{com}_i)_{i \in [m]}, \text{com}'_y, f, \pi') = 1 \\ \wedge \text{com}_y \neq \text{com}'_y \end{array} \mid \begin{array}{l} \text{ck} \leftarrow \text{Setup}(1^\lambda, 1^\ell) \\ \left(\begin{array}{cc} (\text{com}_i)_{i \in [m]}, & f, \\ \text{com}_y, & \pi, \\ \text{com}'_y, & \pi' \end{array} \right) \leftarrow \mathcal{A}(\text{ck}) \end{array} \right] \leq \text{negl}(\lambda).$$

¹³ We often omit the subscript ℓ of $\mathcal{F}_\ell$ and treat the length parameter implicitly.

Definition 9 (CFC Efficient Verification). A CFC admits efficient verification with preprocessing if there exists a pair of algorithms:

$\text{PreFuncVer}(\text{ck}, f) \rightarrow \text{ck}_f$ on input the commitment key ck and a function $f \in \mathcal{F}$ where $f : \prod_{i \in [m]} \mathcal{M}^{J_i} \rightarrow \mathcal{M}^{J_y}$, outputs a function key ck_f of size $|\text{ck}_f| \leq \text{poly}(\lambda, \log \ell, \log m, \log |f|)$.
 $\text{EffFuncVer}(\text{ck}_f, (\text{com}_i)_{i \in [m]}, \text{com}_y, \pi) \rightarrow b \in \{0, 1\}$ on input a function key ck_f , commitments $(\text{com}_i)_{i \in [m]}$ to the m inputs and com_y to the output, and an opening proof π , accepts ($b = 1$) or rejects ($b = 0$) in time bounded by $\text{poly}(\lambda, \log \ell, \log m, \log |f|)$.

Furthermore, the following function equivalence holds:

$$\text{FuncVer}(\text{ck}, (\text{com}_i)_{i \in [m]}, \text{com}_y, f, \pi) \equiv \text{EffFuncVer}(\text{PreFuncVer}(\text{ck}, f), (\text{com}_i)_{i \in [m]}, \text{com}_y, \pi).$$

4.2 Projective Commitments (PC)

In [WW24], Wee and Wu introduce the notion of projective commitments. We generalise their notion to allow for arbitrary projective subspaces, but not only prefixes, along with the flexibility of committing to subvectors. We adopt the notation that $\mathcal{I}$ is a family of admissible index sets $I \in \mathcal{I} \in 2^{[\ell]}$. We write $\mathbf{x}_I$ to refer to a subvector of $\mathcal{M}^\ell$ that contains only the coordinates in I , namely $\mathbf{x}_I \in \mathcal{M}^I$.

Definition 10 (Projective Commitments (PC)). Let $\mathcal{I}, \mathcal{J}$ be families of index sets parametrised by $\ell \in \mathbb{N}$. A projective commitment (PC) scheme for $(\mathcal{I}, \mathcal{J})$ consists of a commitment scheme $(\text{Setup}, \text{Com})$ for $\mathcal{J}$ and additional PPT algorithms $(\text{ProjSetup}, \text{ProjCom}, \text{Proj})$ with the following syntax:

$\text{ProjSetup}(1^\lambda, 1^\ell, I) \rightarrow (\text{ck}, \text{td})$: On input the security parameter λ , vector length ℓ , and index set $I \in \mathcal{I}$, output a commitment key ck and a trapdoor td .

$\text{ProjCom}(\text{td}, J, \mathbf{x}) \rightarrow \text{pcom}$: On input the trapdoor td , an index set $J \in \mathcal{J}$ and a vector $\mathbf{x} \in \mathcal{M}^{I \cap J}$, output a projective commitment pcom .

$\text{Proj}(\text{td}, \text{com}) \rightarrow \text{pcom}$: On input the trapdoor td and a commitment com , output a projective commitment pcom .¹⁴

The index set J is taken as $J = [\ell]$ when omitted. A PC must satisfy the following properties:

Projection Correctness. For any index sets $I \in \mathcal{I}$, $J \in \mathcal{J}$, any vector $\mathbf{x} \in \mathcal{M}^{I \cap J}$, and any $(\text{ck}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I)$, then $\Pr[\text{Proj}(\text{td}, \text{Com}(\text{ck}, J, \mathbf{x})) = \text{ProjCom}(\text{td}, J, \mathbf{x})] = 1$.

Setup Indistinguishability. For any PPT adversary $\mathcal{A}$, and subspace $I \in \mathcal{I}$, the distributions of ck induced by $(\text{ck}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I)$ and $\text{ck} \leftarrow \text{Setup}(1^\lambda, 1^\ell)$ are computationally indistinguishable.

Extending the notion of aggregatability, we define the notion of projective aggregatability which states that not only ordinary commitments but also projective commitments are aggregatable.

Definition 11 (PC Projective Aggregatability). A PC $(\text{Setup}, \text{Com}, \text{ProjSetup}, \text{ProjCom}, \text{Proj})$ for $(\mathcal{I}, \mathcal{J})$ is projective aggregatable if, given $\hat{\mathcal{J}} \subset \mathcal{J}$,

¹⁴ Note that we define Proj not to input the (alleged) index set J of the commitment com . This leads to stronger properties related to projection. Clearly, the syntax and properties can be relaxed to allow Proj input J .

- (Setup, Com) is aggregatable with an aggregation algorithm Agg,
- there exists a projective aggregation algorithm $\text{ProjAgg}(\text{ck}, (\text{pcom}_J)_{J \in \hat{\mathcal{J}}}) \rightarrow \text{pcom}$ inputting the commitment key ck and tuples of projective commitments $(\text{pcom}_J)_{J \in \hat{\mathcal{J}}}$ and outputs a projective commitment pcom ,
- if $J \cap J' = \emptyset$ for all distinct $J, J' \in \hat{\mathcal{J}}$, then for any $I \in \mathcal{I}$ and any $\mathbf{x}_J \in \mathcal{M}^{I \cap J}$ for $J \in \hat{\mathcal{J}}$, the following property is satisfied:

$$\Pr \left[\text{ProjAgg}(\text{ck}, (\text{pcom}_J)_{J \in \hat{\mathcal{J}}}) = \text{pcom} \left| \begin{array}{l} (\text{ck}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I) \\ \text{pcom}_J \leftarrow \text{ProjCom}(\text{td}, J, \mathbf{x}_J) \quad \forall J \in \hat{\mathcal{J}} \\ \text{pcom} \leftarrow \text{ProjCom}(\text{ck}, \bigcup_{J \in \hat{\mathcal{J}}} J, \bigcup_{J \in \hat{\mathcal{J}}} \mathbf{x}_J) \end{array} \right. \right] = 1.$$

Remark 3 (Projection commutativity). Projection correctness (Definition 10) and projective aggregatability (Definition 11), which implies aggregatability (Definition 6), together imply “projection commutativity”. In more detail, suppose

$$\begin{aligned} \text{com}_J &= \text{Com}(\text{ck}, J, \mathbf{x}_J), & \text{pcom}_J &= \text{ProjCom}(\text{td}, J, \mathbf{x}_J), \\ \text{com} &= \text{Com}(\text{ck}, \bigcup_{J \in \hat{\mathcal{J}}} J, \bigcup_{J \in \hat{\mathcal{J}}} \mathbf{x}_J), & \text{and} & \quad \text{pcom} = \text{ProjCom}(\text{td}, \bigcup_{J \in \hat{\mathcal{J}}} J, \bigcup_{J \in \hat{\mathcal{J}}} \mathbf{x}_J). \end{aligned}$$

The above three properties imply the following:

- projection correctness: $\text{Proj}(\text{td}, \text{com}_J) = \text{pcom}_J$ and $\text{Proj}(\text{td}, \text{com}) = \text{pcom}$.
- aggregatability: $\text{Agg}(\text{ck}, (\text{com}_J)_{J \in \hat{\mathcal{J}}}) = \text{com}$.
- projective aggregatability: $\text{ProjAgg}(\text{ck}, (\text{pcom}_J)_{J \in \hat{\mathcal{J}}}) = \text{pcom}$.

Putting the above together, we obtain

$$\text{ProjAgg}(\text{ck}, (\text{Proj}(\text{td}, \text{com}_J))_{J \in \hat{\mathcal{J}}}) = \text{Proj}(\text{td}, \text{Agg}(\text{ck}, (\text{com}_J)_{J \in \hat{\mathcal{J}}})).$$

For certain message subspaces $\bar{\mathcal{M}} \subset \mathcal{M}$ and vectors $\mathbf{x} \in \bar{\mathcal{M}}^\ell$, it might be possible to augment a projective commitment scheme with an embedded proof system that provides somewhere extractability. It means that, if the commitment key is extractable at set I and an adversary provides a commitment com and a proof π that verifies, then one can extract a subvector $\mathbf{x}_I \in \bar{\mathcal{M}}^I$ such that $\text{Proj}(\text{td}, \text{com}) = \text{ProjCom}(\text{td}, \mathbf{x}_I)$ except with negligible probability.

To illustrate this property, consider the example of a commitment scheme such that the message space $\mathcal{M} = \mathbb{F}_q$ for a prime $q > 2$. Moreover, the commitment supports an embedded “booleanity” proof that ensures that a committed value $\mathbf{x}$ satisfies $x_i(x_i - 1) = 0 \quad \forall i \in [\ell]$, namely that $x_i \in \{0, 1\} = \bar{\mathcal{M}}$. Then, our somewhere extractability notion ensures that we can extract a vector $\mathbf{x}_I \in \{0, 1\}^I$.

Definition 12 (PC Somewhere Extractability). Let $\mathcal{I}, \mathcal{J}$ be index sets families parametrised by $\ell \in \mathbb{N}$ and $\bar{\mathcal{M}} \subset \mathcal{M}$ be a message subspace. A somewhere extractable PC for $(\mathcal{I}, \mathcal{J}, \bar{\mathcal{M}})$ consists of a PC for $(\mathcal{I}, \mathcal{J})$ and additional PPT algorithms $(\text{ProjProve}, \text{ProjVer}, \text{ProjExt})$ with the following syntax:

$\text{ProjProve}(\text{ck}, J, \mathbf{x}) \rightarrow \pi$: On input the commitment key ck , an index set $J \in \mathcal{J}$ and a vector $\mathbf{x} \in \mathcal{M}^J$, output a subspace proof π .

$\text{ProjVer}(\text{ck}, J, \text{com}, \pi) \rightarrow 0/1$: On input the commitment key ck , an index set $J \in \mathcal{J}$, a commitment com and a subspace proof π , accepts or rejects.

$\text{ProjExt}(\text{td}, J, \text{com}) \rightarrow \mathbf{x}$: On input the trapdoor td , an index set $J \in \mathcal{J}$ and a commitment com , outputs a value $\mathbf{x} \in \bar{\mathcal{M}}^{I \cap J} \cup \{\perp\}$.

Moreover, these algorithms must satisfy the following properties:

Succinctness. For any honestly generated commitment key ck , index set $J \in \mathcal{J}$ and vector $\mathbf{x} \in \mathcal{M}^J$, a projection proof $\pi \in \text{ProjProve}(\text{ck}, J, \mathbf{x})$ satisfies $|\pi| \leq \text{poly}(\lambda, \log \ell, \log |J|)$.

Somewhere Completeness. For any index sets $I \in \mathcal{I}$ and $J \in \mathcal{J}$ and any vector $\mathbf{x} \in \mathcal{M}^J$:

$$\Pr \left[\text{ProjVer}(\text{ck}, J, \text{com}, \pi) = 1 \mid \begin{array}{l} (\text{ck}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I) \\ \text{com} \leftarrow \text{Com}(\text{ck}, J, \mathbf{x}) \\ \pi \leftarrow \text{ProjProve}(\text{ck}, J, \mathbf{x}) \end{array} \right] = 1.$$

Somewhere Extractability. For any PPT^{15} adversary $\mathcal{A}$ and index set $I \in \mathcal{I}$, the advantage $\text{Adv}_{\text{PCFC}, \mathcal{A}}^{\text{se}}(\lambda)$ defined below is $\text{negl}(\lambda)$:

$$\Pr \left[\begin{array}{l} \text{ProjVer}(\text{ck}, J, \text{com}, \pi) = 1 \\ \wedge \left(\text{ProjCom}(\text{td}, J, \mathbf{x}) \neq \text{Proj}(\text{td}, \text{com}) \vee \mathbf{x} \notin \bar{\mathcal{M}}^{I \cap J} \right) \end{array} \mid \begin{array}{l} (\text{ck}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I) \\ (J, \text{com}, \pi) \leftarrow \mathcal{A}(\text{ck}) \\ \mathbf{x} \leftarrow \text{ProjExt}(\text{td}, J, \text{com}) \\ J \in \mathcal{J} \end{array} \right].$$

Remark 4 (Somewhere extractability for aggregated commitments.). Naturally, the above definition of somewhere extractability (Definition 12) can be extended to aggregated commitments. If $\text{com} = \text{Agg}(\text{ck}, (\text{com}_J)_{J \in \hat{\mathcal{J}}})$ for some non-overlapping family $\hat{\mathcal{J}} \subset \mathcal{J}$, then it follows that $\text{ProjExt}(\text{td}, \bigcup_{J' \in \hat{\mathcal{J}}} J', \text{com}) = \bigcup_{J \in \hat{\mathcal{J}}} \text{ProjExt}(\text{td}, J', \text{com}_J)$. This may enable extraction on aggregated commitments com where running the extractor ProjExt on com directly is not possible, such as when $I \cap J$ is too large.

Definition 13 (PC Efficient Verification). A somewhere extractable PC admits efficient verification with preprocessing if there exists a pair of algorithms:

$\text{PreProjVer}(\text{ck}, J) \rightarrow \text{ck}_J$ on input the commitment key ck and an index set $J \in \mathcal{J}$, outputs a projective key ck_J of size $|\text{ck}_J| \leq \text{poly}(\lambda, \log \ell, \log |J|)$.

$\text{EffProjVer}(\text{ck}_J, \text{com}, \pi) \rightarrow b \in \{0, 1\}$ on input a projective key ck_J , a commitment com , and a projection proof π , accepts ($b = 1$) or rejects ($b = 0$) in time bounded by $\text{poly}(\lambda, \log \ell, \log |J|)$.

Furthermore, the following function equivalence holds:

$$\text{ProjVer}(\text{ck}, J, \text{com}, \pi) \equiv \text{EffProjVer}(\text{PreProjVer}(\text{ck}, J), \text{com}, \pi).$$

4.3 Projective Chainable Functional Commitments (PCFC)

We consider projective commitments that also admit the syntax of chainable functional commitments (Definition 7). We call these commitments *projective chainable functional commitments* (PCFC).

¹⁵ The property may also hold statistically against unbounded adversaries.

Definition 14 (Projective Chainable Functional Commitment (PCFC)). A (somewhere extractable) projective CFC ((se-)PCFC) for a class of functions $\mathcal{F}$, families of admissible index sets $(\mathcal{I}, \mathcal{J})$ (and a message subspace $\bar{\mathcal{M}}$) is a tuple of PPT algorithms consisting of some shared Setup, Com and

$$\overbrace{(\text{ProjSetup}, \text{ProjCom}, \text{Proj}, (\text{ProjProve}, \text{ProjVer}, \text{ProjExt}), \text{FuncProve}, \text{FuncVer})}^{\text{se-PC for } (\mathcal{I}, \mathcal{J}, \bar{\mathcal{M}})}.$$

$\underbrace{\hspace{10em}}_{\text{PC for } (\mathcal{I}, \mathcal{J})} \quad \underbrace{\hspace{10em}}_{\text{CFC for } (\mathcal{F}, \mathcal{J})}$

Definition 15 (PCFC Efficient Verification). A PCFC admits efficient verification with preprocessing if the underlying CFC and PC admit efficient verification with preprocessing, i.e. there exists additional algorithms (PreFuncVer, EffFuncVer, PreProjVer, EffProjVer) with properties defined in Definitions 9 and 13.

We introduce the two main security notions for PCFC: functional extractability and projective chain binding. To define these notions, we first introduce separable functions.

Definition 16 (Separable Function). Let $I_1, \dots, I_\ell, I_y \subseteq [\ell]$. We say that a function $f : \prod_{i \in [m]} \mathcal{M}^{J_i} \rightarrow \mathcal{M}^{J_y}$ is $I = (I_1, \dots, I_\ell, I_y)$ -separable whenever the coordinates in $I_y \cap J_y$ of the output vector only depend on the coordinates in $I_i \cap J_i$ of each of the i -th input vectors. The restriction of f to I is denoted by $f_I : \prod_{i \in [m]} \mathcal{M}^{I_i \cap J_i} \rightarrow \mathcal{M}^{I_y \cap J_y}$. For a class of functions $\mathcal{F}$, we write $\mathcal{F}_I$ for the set of functions $f \in \mathcal{F}$ that are I -separable. If f is I -separable for all $I \in \mathcal{I}^{\ell+1}$ for some set $\mathcal{I}$, we say that f is $\mathcal{I}$ -separable. If $I_1 = \dots = I_\ell = I_y$, we abuse notation and write $I = I_1 = \dots = I_\ell = I_y$.

Example 1. The function $f : \mathbb{Z}^2 \times \mathbb{Z}^2 \rightarrow \mathbb{Z}^2$ given by $f((x_1, x_2), (y_1, y_2)) = (x_1 + y_1, x_1 + x_2 + y_2)$ is $\mathcal{I}$ -separable for $\mathcal{I} = \{\{1\}, \{1, 2\}\}$.

Intuitively, a PCFC is functional extractable if, given a CFC opening π for a separable function f , then the extracted values from the commitments are guaranteed to evaluate f_I correctly.

Definition 17 (PCFC Functional Extractability). A somewhere extractable PCFC satisfies functional somewhere extractability for a class of $\mathcal{I}$ -separable functions $\mathcal{F}$ and index set families $(\mathcal{I}, \mathcal{J})$ if for any PPT adversary $\mathcal{A}$ and set $I \in \mathcal{I}$, the advantage $\text{Adv}_{\text{PCFC}, \mathcal{A}}^{\text{fe}}(\lambda)$ defined below is $\text{negl}(\lambda)$:

$$\Pr \left[\begin{array}{l} \text{FuncVer}(\text{ck}, (\text{com}_i)_{i \in [m]}, \text{com}_y, f, \pi) = 1 \\ \wedge f_I(\mathbf{x}_1, \dots, \mathbf{x}_m) \neq \mathbf{y} \\ \wedge f \in \mathcal{F}_I \end{array} \right] \left[\begin{array}{l} (\text{ck}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I) \\ (f, (\text{com}_i)_{i \in [m]}, \text{com}_y, \pi) \leftarrow \mathcal{A}(\text{ck}) \\ f : \prod_{i \in [m]} \mathcal{M}^{J_i} \rightarrow \mathcal{M}^{J_y} \\ \mathbf{x}_i \leftarrow \text{ProjExt}(\text{td}, J_i, \text{com}_i) \quad \forall i \in [m] \\ \mathbf{y} \leftarrow \text{ProjExt}(\text{td}, J_y, \text{com}_y) \\ \mathbf{x}_i \in \mathcal{M}^{I \cap J_i} \quad \forall i \in [m] \\ \mathbf{y} \in \mathcal{M}^{I \cap J_y} \end{array} \right]$$

Remark 5. Due to the somewhere extractability of PC (Definition 12), an alternative way of defining Functional Extractability for PCFC is to let the adversary output $\mathbf{x}_i$ and $\mathbf{y}$, and to let the experiment run Proj instead of ProjExt.

Finally, we introduce projective chain binding for PCFC which generalises evaluation binding (Definition 8). In particular, if a PCFC for a class of $\mathcal{I}$ -separable functions $\mathcal{F}$ and index set families $(\mathcal{I}, \mathcal{J})$ is projective chain binding, where $\mathcal{I}$ contains $[\ell]$, then the PCFC is also evaluation binding for $\mathcal{F}$.

Definition 18 (PCFC Projective Chain Binding). *A PCFC for a class of $\mathcal{I}$ -separable functions $\mathcal{F}$ and index set families $(\mathcal{I}, \mathcal{J})$ satisfies projective chain binding if for any PPT adversary $\mathcal{A}$ and set $I \in \mathcal{I}$, the advantage $\text{Adv}_{\text{PCFC}, \mathcal{A}}^{\text{pcb}}(\lambda)$ defined below is $\text{negl}(\lambda)$.*

$$\Pr \left[\begin{array}{l} \text{Proj}(\text{td}, \text{com}_y) \neq \text{Proj}(\text{td}, \text{com}'_y) \\ \wedge f \in \mathcal{F}_I \end{array} \middle| \begin{array}{l} (\text{ck}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I) \\ (f, (\text{com}_i, \text{com}'_i)_{i \in [m]}, \text{com}_y, \text{com}'_y, \pi, \pi') \leftarrow \mathcal{A}(\text{ck}, \text{td}) \\ \text{FuncVer}(\text{ck}, (\text{com}_i)_{i \in [m]}, \text{com}_y, f, \pi) = 1 \\ \text{FuncVer}(\text{ck}, (\text{com}'_i)_{i \in [m]}, \text{com}'_y, f, \pi') = 1 \\ \text{Proj}(\text{td}, \text{com}_i) = \text{Proj}(\text{td}, \text{com}'_i) \quad \forall i \in [m] \end{array} \right].$$

Note that the adversary not only gets access to the commitment key ck , but also to the projection trapdoor td . One may consider a weaker variant where td is not given to the adversary.

On functional extractability, projective chain binding, and partial input soundness. Partial input soundness [KLVW23] is a security notion that arises in the context of RAM SNARGs, more specifically in *flexible RAM SNARGs*. A flexible RAM SNARG with partial input soundness intuitively guarantees soundness if some part of the RAM memory can be extracted from the hash of the memory tape, as long as the RAM computation depends only on the part that was extracted. The notion is defined with respect to a somewhere extractable hash function that is programmed on a memory part I . Then, partial input soundness is stated roughly as follows: given a somewhere extractable hash key programmed on I , it is hard for any adversary to output a valid SNARG proof π and a hash digest rt such that the RAM machine $\mathcal{R}$ reads only I and the extracted $\mathbf{x}_I$ satisfies that $\mathcal{R}(\mathbf{x}_I) = 0$.

Functional extractability (Definition 17) and partial input soundness are essentially the same property in different environments, as both notions refer to local computations where a part of the input can be extracted. Their main differences are the following two. First, functional extractability is not defined with respect to any external building block such as a somewhere extractable hash function, as it is an intrinsic property of PCFC schemes. Second, the property is defined for separable functions over circuits, instead of RAM machines that read only a part of their memory tape.

On the other hand, projective chain binding (Definition 18) is a stronger notion than both functional extractability and partial input soundness, as it must hold even if the adversary knows the trapdoor of the scheme. Moreover, projective chain binding holds for projections, which may or may not be (somewhere) extractable. In case the projected input commitments are indeed extractable, it is not hard to see that projective chain binding implies functional extractability.

5 Circuit-Succinct Compiler from PCFC to BARG

In this section we construct our black-box compiler that turns a PCFC (Definition 14) into a circuit-succinct BARG for NP, this is, a BARG such that the proof size grows linearly on the size of the witness ℓ_w . Our BARG can prove batch relations $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i)$ for $i \in [k]$, where $\mathcal{C} : \mathcal{M}^{\ell_{x,w}} \rightarrow \mathcal{M}$ are (possibly distinct, see Remark 7) arithmetic circuits of bounded size $|\mathcal{C}| \leq \ell_{\mathcal{C}}$.

We briefly describe our circuit-succinct compiler in Figure 2. The prover starts by committing to each of the witness wires across the k batch instances, obtaining commitments $\text{com}_{\ell_x+1}, \dots, \text{com}_{\ell_{x,w}}$. The prover also provides subspace proofs on each of these commitments. Until this point, the compiler is analogous to the first step of the BARG construction in [WW22] (which we abstract in our wire-by-wire compiler in Appendix A). Then, instead of carrying out functional proofs for each circuit wire, the prover generates a PCFC functional opening as follows. First, it defines a function f_C to be the parallel evaluation of C k times. Then, it sets an input $\mathbf{x}$ which is the concatenation of $(\mathbf{x}_i, \mathbf{w}_i)$ for every $i \in [k]$. Finally, it provides a functional proof of $f_C(\mathbf{x}) = (C(\mathbf{x}_1, \mathbf{w}_1), \dots, C(\mathbf{x}_k, \mathbf{w}_k)) = \mathbf{1}_k$. Along with this proof, the prover provides an input commitment com and an output commitment com_y which commits to $\mathbf{1}_k$. The validity of com can be verified as it must match the aggregation of the commitments to the statement wires, which are computed by the verifier, along with the aggregation of $\text{com}_{\ell_x+1}, \dots, \text{com}_{\ell_{x,w}}$.

The proof of BARG somewhere extractability for instance i^* intuitively works as follows. First, PC somewhere extractability enables the extraction of a binary witness $\mathbf{w}_{i^*}$ from each of the ℓ_w commitment wires, where $\mathbf{w}_{i^*}$ is guaranteed to match the projections of these commitments. Second, we apply PC projective aggregatability to ensure that the projection of com indeed commits to $(\mathbf{x}_{i^*}, \mathbf{w}_{i^*})$. Finally, we crucially rely on PCFC projective chain binding to ensure that $C(\mathbf{x}_{i^*}, \mathbf{w}_{i^*}) = 1$.

Setup $(1^\lambda, k, 1^{\ell_C})$ $\ell := k \cdot \ell_{x,w}$ $\text{ck} \leftarrow \text{PCFC.Setup}(1^\lambda, 1^\ell)$ $\text{com}_{\text{out}} \leftarrow \text{Com}(\text{ck}, 1^\ell)$ return $\text{crs} := (\text{ck}, \text{com}_{\text{out}})$	Prove $(\text{crs}, C, \{(\mathbf{x}_i, \mathbf{w}_i)\}_{i \in [k]})$ $\mathbf{x}_g := (x_{1,g}, \dots, x_{k,g}) \quad \forall g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}$ $\mathbf{x} := (\mathbf{x}_g)_{g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}}$ $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, J_g, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}$ $\pi_g^b \leftarrow \text{PCFC.ProjProve}(\text{ck}, J_g, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{wit}}$ $\pi^x \leftarrow \text{PCFC.FuncProve}(\text{ck}, \mathbf{x}, f_C)$ return $\pi := ((\text{com}_g, \pi_g^b)_{g \in \mathcal{G}_{\text{wit}}}, \pi^x)$
SetupTd $(1^\lambda, k, 1^{\ell_C}, i^*)$ $\ell := k \cdot \ell_{x,w}$ $(\text{ck}, \text{td}) \leftarrow \text{PCFC.ProjSetup}(1^\lambda, 1^\ell, I_{i^*})$ $\text{com}_{\text{out}} \leftarrow \text{Com}(\text{ck}, 1^\ell)$ return $(\text{crs}, \text{td}) := ((\text{ck}, \text{com}_{\text{out}}), \text{td})$	Ver $(\text{crs}, C, \{\mathbf{x}_i\}_{i \in [k]}, \pi)$ $\mathbf{x}_g := (x_{1,g}, \dots, x_{k,g}) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, J_g, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{com} \leftarrow \text{PCFC.Agg}(\text{ck}, (\text{com}_g)_{g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}})$ assert $\text{PCFC.ProjVer}(\text{ck}, J_g, \text{com}_g, \pi_g^b) \quad \forall g \in \mathcal{G}_{\text{wit}}$ assert $\text{PCFC.FuncVer}(\text{ck}, \text{com}, \text{com}_{\text{out}}, f_C, \pi^x)$ return 1
Ext $(\text{td}, C, \{\mathbf{x}_i\}_{i \in [k]}, \pi)$ $\mathbf{w}_{i^*,g} \leftarrow \text{PCFC.ProjExt}(\text{td}, J_g, \text{com}_g) \quad \forall g \in \mathcal{G}_{\text{wit}}$ return $\mathbf{w}_{i^*} := (\mathbf{w}_{i^*,g})_{g \in \mathcal{G}_{\text{wit}}}$	EffVer (vk, π) $\text{com}_{\mathcal{G}_{\text{wit}}} \leftarrow \text{PCFC.Agg}(\text{ck}, (\text{com}_g)_{g \in \mathcal{G}_{\text{wit}}})$ $\text{com} \leftarrow \text{PCFC.Agg}(\text{ck}, (\text{com}_{\mathcal{G}_{\text{stmt}}}, \text{com}_{\mathcal{G}_{\text{wit}}}))$ assert $\text{PCFC.EffProjVer}(\text{ck}_{J_g}, \text{com}_g, \pi_g^b) \quad \forall g \in \mathcal{G}_{\text{wit}}$ assert $\text{PCFC.EffFuncVer}(\text{ck}_{f_C}, \text{com}, \text{com}_{\text{out}}, \pi^x)$ return 1
PreVer $(\text{crs}, C, \{\mathbf{x}_i\}_{i \in [k]})$ $\mathbf{x}_g := (x_{1,g}, \dots, x_{k,g}) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, J_g, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{com}_{\mathcal{G}_{\text{stmt}}} \leftarrow \text{PCFC.Agg}(\text{ck}, (\text{com}_g)_{g \in \mathcal{G}_{\text{stmt}}})$ $\text{ck}_{f_C} \leftarrow \text{PCFC.PreFuncVer}(\text{ck}, f_C)$ $\text{ck}_{J_g} \leftarrow \text{PCFC.PreProjVer}(\text{ck}, J_g) \quad \forall g \in \mathcal{G}_{\text{wit}}$ $\text{vk} := (\text{com}_{\mathcal{G}_{\text{stmt}}}, \text{ck}_{f_C}, (\text{ck}_{J_g})_{g \in \mathcal{G}_{\text{wit}}})$ return vk	

Fig. 2: Compiler from PCFC to fully succinct BARG.

Notation and parameters. For an (arithmetic or Boolean) circuit $\mathcal{C} : \bar{\mathcal{M}}^{\ell_{x,w}} \rightarrow \bar{\mathcal{M}}$, we define $f_{\mathcal{C}} : \bar{\mathcal{M}}^{k \cdot \ell_{x,w}} \rightarrow \bar{\mathcal{M}}^{k \cdot \ell_{x,w}}$ as $f_{\mathcal{C}}(\mathbf{x}_1, \dots, \mathbf{x}_k) = (\mathcal{C}(\mathbf{x}_1), \dots, \mathcal{C}(\mathbf{x}_k))$. Let $\mathcal{F}$ be the family of such functions $\mathcal{F} = \{f_{\mathcal{C}} : \mathcal{C} : \bar{\mathcal{M}}^{\ell_{x,w}} \rightarrow \bar{\mathcal{M}}\}$, and we let the input length $\ell = k \cdot \ell_{x,w}$.

To express $(i-1)\ell_{x,w} + g \in [\ell]$ for $i \in [k], g \in \ell_{x,w}$, we abuse notation and denote it by $(i, g) \in [\ell]$. That is, we use a bijective mapping between $[k] \times [\ell_{x,w}]$ and $[\ell]$ given by $(i, g) \mapsto (i-1)\ell_{x,w} + g$. This allows us to directly map values to their corresponding instance and wire. We define the index subsets $I_i := \{(i, g) : g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}\}$ for $i \in [k]$ and $J_g := \{(i, g) : i \in [k]\}$ for $g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}$. Note that $I_i \cap J_g = \{(i, g)\}$ isolates a single index $(i, g) \in [\ell]$. By construction, all functions in $\mathcal{F}$ are $\mathcal{I}$ -separable for the family of index sets $\mathcal{I} = \{I_i\}_{i \in [k]}$.

	statements				witnesses				wires				
	J_1	J_2	$\cdots$	J_{ℓ_x}	J_{ℓ_x+1}	$\cdots$	J_{ℓ_x+j}	$\cdots$	$J_{\ell_x,w}$	$J_{\ell_x,w+1}$	$J_{\ell_x,w+2}$	$\cdots$	J_{ℓ_C}
I_1	$\mathbf{x}_{1,1}$	$\mathbf{x}_{1,2}$	$\cdots$	$\mathbf{x}_{1,\ell_x}$	$\mathbf{w}_{1,1}$	$\cdots$	$\mathbf{w}_{1,j}$	$\cdots$	$\mathbf{w}_{1,\ell_w}$	$x_{1,\ell_x,w+1}$	$x_{1,\ell_x,w+2}$	$\cdots$	x_{1,ℓ_C}
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$		$\vdots$		$\vdots$	$\vdots$	$\vdots$		$\vdots$
I_i	$\mathbf{x}_{i,1}$	$\mathbf{x}_{i,2}$	$\cdots$	$\mathbf{x}_{i,\ell_x}$	$\mathbf{w}_{i,1}$	$\cdots$	$\mathbf{w}_{i,j}$	$\cdots$	$\mathbf{w}_{i,\ell_w}$	$x_{i,\ell_x,w+1}$	$x_{i,\ell_x,w+2}$	$\cdots$	x_{i,ℓ_C}
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$		$\vdots$		$\vdots$	$\vdots$	$\vdots$		$\vdots$
I_k	$\mathbf{x}_{k,1}$	$\mathbf{x}_{k,2}$	$\cdots$	$\mathbf{x}_{k,\ell_x}$	$\mathbf{w}_{k,1}$	$\cdots$	$\mathbf{w}_{k,j}$	$\cdots$	$\mathbf{w}_{k,\ell_w}$	$x_{k,\ell_x,w+1}$	$x_{k,\ell_x,w+2}$	$\cdots$	x_{k,ℓ_C}
	$\downarrow$	$\downarrow$		$\downarrow$	$\downarrow$		$\downarrow$		$\downarrow$				
	com_1	com_2		com_{ℓ_x}	com_{ℓ_x+1}		com_{ℓ_x+j}		$\text{com}_{\ell_x,w}$				

Table 1: Structure of the committed values in Figure 2. The rows are associated to projection sets $I_i \in \mathcal{I}$. The columns are associated to the subvector sets $J_j \in \mathcal{J}$.

In Table 1, we show how the BARG inputs are arranged with respect to instances and subvectors. Recall that, for a given batch $i \in [k]$, we denote $x_{i,j} \leftarrow \mathbf{x}_{i,j}$ for $j = 1, \dots, \ell_x$, and $x_{i,j} \leftarrow \mathbf{w}_{i,(j-\ell_x)}$ for $j = \ell_x + 1, \dots, \ell_{x,w}$.

Theorem 1 (Circuit-Succinct Compiler). Let $\mathcal{I} := \{I_i : i \in [k]\}$ where $I_i := \{(i, g) : g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}\}$, $\mathcal{J} := \{J_g : g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}\}$ where $J_g := \{(i, g) : i \in [k]\}$, $\mathcal{F} = \{f_{\mathcal{C}} : \mathcal{C} : \bar{\mathcal{M}}^{\ell_{x,w}} \rightarrow \bar{\mathcal{M}}\}$ where $f_{\mathcal{C}}(\mathbf{x}_1, \dots, \mathbf{x}_k) = (\mathcal{C}(\mathbf{x}_1), \dots, \mathcal{C}(\mathbf{x}_k))$, $\bar{\mathcal{M}} \supseteq \{0, 1\}$ be some message space, PCFC be a projective chainable functional commitment scheme (Definition 14) for the $\mathcal{I}$ -separable functions $\mathcal{F}$ and index set families $(\mathcal{I}, \mathcal{J})$ which satisfies

- (Definition 11) PC projective aggregatability,
- (Definition 12) PC somewhere extractability for $(\mathcal{I}, \mathcal{J}, \bar{\mathcal{M}})$ and
- (Definition 18) PCFC projective chain binding for $(\mathcal{F}, \mathcal{I}, \mathcal{J})$.

Then, the construction BARG in Figure 2 is a somewhere extractable batch argument (Definition 1) for NP whose proof size is $|\pi| = \ell_w \cdot (|\text{com}| + |\pi^b|) + |\pi^x|$ where π^b is a PC subspace proof, and π^x is a PCFC functional proof for $f \in \mathcal{F}$. Moreover, BARG admits efficient verification with preprocessing.

Proof. The completeness, full succinctness and setup indistinguishability of BARG follow by inspection, given the correctness, succinctness and setup indistinguishability of PCFC. The somewhere extractability of BARG follows from Lemma 1. Its efficient verification property follows from Lemma 5. $\square$

Remark 6. The projective aggregatability property required by Theorem 1 is optional – its only purpose is so that a BARG proof does not need to include $|\mathcal{G}_{\text{wit}}|$ commitments. If we were to drop the projective aggregatability property, the compiler needs to be slightly modified where **FuncProve** and **FuncVer** instead take as input multiple vectors and input commitments respectively.

Remark 7. Although we instantiate our compiler for the standard batch relation defined by $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i)$ for $i \in [k]$, where the circuit is the same for all batch instances, it actually supports batch proofs for distinct circuits $\mathcal{C}_i(\mathbf{x}_i, \mathbf{w}_i)$ for $i \in [k]$ *without any additional overhead*. The only requirements are that each $|\mathcal{C}_i| \leq \ell_{\mathcal{C}}$ and that the PCFC supports the consequently broader family of functions $\mathcal{F}_{k, \ell_f}$ (defined in Section 6), which is the case for our algebraic PCFC introduced there. This property can be useful in applications such as the aggregation of signature schemes with different verification equations.

5.1 Proof of Somewhere Extractability

Lemma 1. *Let PCFC and BARG be as defined in Theorem 1. It holds that BARG has somewhere extractability.*

Hyb_A⁰(λ): $(\text{ck}, \text{td}) \leftarrow \text{PCFC.ProjSetup}(1^\lambda, 1^k, I_{i^*})$ $\text{com}_{\text{out}} \leftarrow \text{Com}(\text{ck}, 1^\ell)$ $(\mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, ((\text{com}_g, \pi_g^b)_{g \in \mathcal{G}_{\text{wit}}}, \pi^\chi)) \leftarrow \mathcal{A}(\text{ck}, \text{com}_{\text{out}})$ $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, J_g, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{com} \leftarrow \text{PCFC.Agg}(\text{ck}, (\text{com}_g)_{g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}})$ $x_g^* \leftarrow \text{PCFC.ProjExt}(\text{td}, J_g, \text{com}_g) \quad \forall g \in \mathcal{G}_{\text{wit}}$ $\mathbf{w}^* := (x_g^*)_{g \in \mathcal{G}_{\text{wit}}}$ assert $\text{PCFC.ProjVer}(\text{ck}, J_g, \text{com}_g, \pi_g^b) = 1 \quad \forall g \in \mathcal{G}_{\text{wit}}$ assert $\text{PCFC.FuncVer}(\text{ck}, \text{com}, \text{com}_{\text{out}}, f_{\mathcal{C}}, \pi^\chi) = 1$ assert $\mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}^*) \neq 1$ return 1 Hyb_A^{1,j}(λ), $0 \leq j \leq \ell_{\mathbf{w}}$: <i>// identical to Hyb_A⁰(λ) until before “return 1”</i> assert $x_g^* \in \bar{\mathcal{M}} \quad \forall g \in \mathcal{G}_{\text{wit}} \cap [\ell_{\mathbf{x}} + j]$ assert $\text{PCFC.ProjCom}(\text{td}, J_g, x_g^*) = \text{PCFC.Proj}(\text{td}, \text{com}_g) \quad \forall g \in \mathcal{G}_{\text{wit}} \cap [\ell_{\mathbf{x}} + j]$ Hyb_A²(λ), $0 \leq j \leq \ell_{\text{out}}$: <i>// identical to Hyb_A^{1,ℓ_w}(λ) until before “return 1”</i> assert $\text{PCFC.ProjCom}(\text{td}, I_{i^*}, (\mathbf{x}_{i^*}, \mathbf{w}^*)) = \text{PCFC.Proj}(\text{td}, \text{com})$
--

Fig. 3: Games $\text{Hyb}_A^0, \text{Hyb}_A^{1,j}, \text{Hyb}_A^2$ for the proof of BARG somewhere extractability. All games output 1 by default. We **highlight** changes between games.

Proof. We proceed by a series of hybrid games $\text{Hyb}^0, \text{Hyb}^{1,0}, \dots, \text{Hyb}^{1,\ell_w}, \text{Hyb}^2$ which progressively include additional winning conditions. The formal descriptions of the games are given in Figure 3 with explanations given below.

Hyb⁰: This is the standard somewhere extractability game (Definition 1) with the algorithms in BARG inlined. To recall, in this game the adversary outputs a proof π and the experiment runs $\mathbf{w}_{i^*} \leftarrow \text{BARG.Ext}(\text{td}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi)$. The adversary wins if the proof verifies yet $\mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}_{i^*}) \neq 1$.

Hyb^{1,j}: $\text{Hyb}^{1,0}$ is identical to Hyb^0 . In $\text{Hyb}^{1,j}$, the game additionally checks that the first j entries of the extracted witness, i.e. those indexed by g belonging to the first j values in $\mathcal{G}_{\text{wit}}$, resides in the message subspace $\mathcal{M}$, i.e. $x_g^* \in \mathcal{M}^{\ell_w}$, and that they are consistent with the projected commitments, i.e. $\text{PCFC.ProjCom}(\text{td}, J_g, x_g^*) = \text{PCFC.Proj}(\text{td}, \text{com}_g)$.

Hyb²: This game simply adds the winning condition that

$$\text{PCFC.Proj}(\text{td}, \text{com}) = \text{PCFC.ProjCom}(\text{td}, I_{i^*}, (\mathbf{x}_{i^*}, \mathbf{w}^*)).$$

We now proceed to bound the advantage of any PPT adversary $\mathcal{A}$ against Hyb^0 by a series of lemmas.

Lemma 2. *For any $j \in [\ell_w]$, there exists a PPT adversary $\mathcal{B}$ against the somewhere extractability for $(\mathcal{I}, \mathcal{J}, \bar{\mathcal{M}})$ of PCFC such that*

$$\left| \Pr[\text{Hyb}_{\mathcal{A}}^{1,j-1}(\lambda) = 1] - \Pr[\text{Hyb}_{\mathcal{A}}^{1,j}(\lambda) = 1] \right| \leq \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{se}}(\lambda).$$

Proof. Fix $j \in [\ell_w]$. We build an adversary $\mathcal{B}$ as follows. $\mathcal{B}$ receives input ck from its challenger and runs $\mathcal{A}(\text{ck})$. Then, it parses the proof π of $\mathcal{A}$ and retrieves the tuple $(\text{com}_{\ell_x+j}, \pi_{\ell_x+j}^b)$. Finally, $\mathcal{B}$ simply forwards these to its challenger.

We argue that if $\mathcal{A}$ wins in $\text{Hyb}^{1,j-1}$ but not in $\text{Hyb}^{1,j}$, then $\mathcal{B}$ also wins the game. First, as $\mathcal{A}$ wins in $\text{Hyb}^{1,j}$, it holds that $\text{PCFC.ProjVer}(\text{ck}, \text{com}_{\ell_x+j}, \pi_{\ell_x+j}^b) = 1$. Also, note that $x_{\ell_x+j}^* \leftarrow \text{PCFC.ProjExt}(\text{td}_{i^*}, \text{com}_{\ell_x+j})$. Therefore, all the preconditions for $\mathcal{B}$ in the PC somewhere extractability game are met. Finally, as $\mathcal{A}$ does not win in $\text{Hyb}^{1,j}$, it must be that either $\text{PCFC.Proj}(\text{td}_{i^*}, \text{com}_{\ell_x+j}) \neq \text{PCFC.ProjCom}(\text{td}_{i^*}, x_{\ell_x+j}^*)$ or $x_{\ell_x+j}^* \notin \bar{\mathcal{M}}$. $\square$

Lemma 3. *It holds that*

$$\Pr[\text{Hyb}_{\mathcal{A}}^{1,\ell_w}(\lambda) = 1] = \Pr[\text{Hyb}_{\mathcal{A}}^2(\lambda) = 1].$$

Proof. Immediate from projective aggregatability (Definition 11). $\square$

Lemma 4. *There exists a PPT adversary $\mathcal{B}$ against projective chain binding for $(\mathcal{F}, \mathcal{I}, \mathcal{J})$ of PCFC such that*

$$\Pr[\text{Hyb}_{\mathcal{A}}^2(\lambda) = 1] \leq \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{pcb}}(\lambda).$$

Proof. We build the adversary $\mathcal{B}$ against projective chain binding from $\mathcal{A}$ against Hyb^2 as follows. Our adversary $\mathcal{B}$ receives (ck, td) from its challenger, computes $\text{com}_{\text{out}} = \text{PCFC.Com}(\text{ck}, 1^\ell)$ and runs $\mathcal{A}$ on $(\text{ck}, \text{com}_{\text{out}})$ (note that $\mathcal{A}$ does not receive td as input). It obtains $(f_{\mathcal{C}}, \text{com}_{\text{out}}, (\text{com}_g)_{g \in \mathcal{G}_{\text{wit}}}, \pi^X)$ from $\mathcal{A}$.

Next, $\mathcal{B}$ extracts the witness values $\mathbf{w}_g^* \leftarrow \text{PCFC.ProjExt}(\text{td}, J_g, \text{com}_g)$ one by one from the commitments $(\text{com}_g)_{g \in \mathcal{G}_{\text{wit}}}$ and generates dummy circuit inputs $\tilde{\mathbf{x}}_i = \mathbf{x}_i | \tilde{\mathbf{w}}_i$ as follows:

$$\tilde{\mathbf{x}}_i \leftarrow \mathbf{x}_i | \mathbf{0}_{\ell_w} \text{ if } i \neq i^*; \quad \mathbf{x}_i | \mathbf{w}^* \text{ if } i = i^*.$$

It commits to the dummy inputs, $\text{c}\tilde{\text{om}} \leftarrow \text{PCFC.Com}(\text{ck}, (\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_k))$. Let also $\tilde{y}_i \leftarrow \mathcal{C}(\tilde{\mathbf{x}}_i)$ for all $i \in [k]$, and let $\text{c}\tilde{\text{om}}_{\text{out}} \leftarrow \text{PCFC.Com}(\text{ck}, (\tilde{y}_1, \dots, \tilde{y}_k))$. Finally, $\mathcal{B}$ generates an honestly computed opening proof that $f_{\mathcal{C}}(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_k) = (y_1, \dots, y_k)$ by running $\pi^{\chi} \leftarrow \text{PCFC.FuncProve}(\text{ck}, \tilde{\mathbf{x}}, f_{\mathcal{C}})$. Then, it returns the tuple

$$(f_{\mathcal{C}}, \text{com}, \text{c}\tilde{\text{om}}, \text{com}_{\text{out}}, \text{c}\tilde{\text{om}}_{\text{out}}, \pi^{\chi}, \tilde{\pi}^{\chi}).$$

Clearly, $\mathcal{B}$ runs in polynomial time. We next analyse its success probability of violating the projective chain binding property. Let $(\text{com}_g)_{g \in \mathcal{G}_{\text{wit}}}, \pi^{\chi}$ be parsed from the outputs of $\mathcal{A}$. Let $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, J_g, \mathbf{x}_g)$ for $g \in \mathcal{G}_{\text{stmt}}$ and $\text{com} \leftarrow \text{PCFC.Agg}(\text{ck}, (\text{com}_g)_{g \in \mathcal{G}_{\text{stmt}} \cup \mathcal{G}_{\text{wit}}})$. By the correctness of PCFC,

- $\text{PCFC.FuncVer}(\text{ck}, \text{c}\tilde{\text{om}}, \text{c}\tilde{\text{om}}_{\text{out}}, f_{\mathcal{C}}, \tilde{\pi}^{\chi}) = 1$,
- $\text{PCFC.Proj}(\text{td}, \text{c}\tilde{\text{om}}) = \text{PCFC.ProjCom}(\text{td}, I_{i^*}, (\mathbf{x}_{i^*}, \mathbf{w}^*))$, and
- $\text{PCFC.Proj}(\text{td}, \text{c}\tilde{\text{om}}_{\text{out}}) = \text{PCFC.ProjCom}(\text{td}, I_{i^*}, 1^{\ell_{\mathbf{x}, \mathbf{w}}} \cdot \tilde{y}_{i^*})$, $\tilde{y}_{i^*} = \mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}^*) \neq 1$.

On the other hand, with probability $\Pr[\text{Hyb}_{\mathcal{A}}^2(\lambda) = 1]$, it holds that

- $\text{PCFC.FuncVer}(\text{ck}, \text{com}, \text{com}_{\text{out}}, f_{\mathcal{C}}, \pi^{\chi}) = 1$,
- $\text{PCFC.Proj}(\text{td}, \text{com}) = \text{PCFC.ProjCom}(\text{td}, I_{i^*}, (\mathbf{x}_{i^*}, \mathbf{w}^*))$, and
- $\text{PCFC.Proj}(\text{td}, \text{com}_{\text{out}}) = \text{PCFC.ProjCom}(\text{td}, I_{i^*}, 1^{\ell_{\mathbf{x}, \mathbf{w}}})$.

In other words, $\mathcal{B}$ successfully wins the PCFC projective chain binding game with probability $\Pr[\text{Hyb}_{\mathcal{A}}^2(\lambda) = 1]$.

We collect the bounds of Lemma 2, Lemma 3, and Lemma 4, conclude that the advantage of $\mathcal{A}$ in the original Hyb^0 is bounded by

$$\Pr[\text{Hyb}_{\mathcal{A}}^0(\lambda) = 1] \leq \ell_{\mathbf{w}} \cdot \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{se}}(\lambda) + \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{pcb}}(\lambda). \quad \square$$

5.2 Efficient Verification

Lemma 5. *Let PCFC and BARG be as defined in Theorem 1. If further PCFC has efficient verification with preprocessing (Definition 15), then BARG has circuit-succinct verification with preprocessing (Definition 2).*

Proof. As per Definition 2, it suffices to show that $|\text{vk}|$ and the runtime of $\text{BARG.EffVer}(\lambda, \text{vk})$ are at most $\text{poly}(\lambda, \log k, \ell_{\mathbf{w}})$. We actually prove below a stronger claim that these quantities are at most $\text{poly}(\lambda, \log k) \cdot \ell_{\mathbf{w}}$. Note that since $\ell = k \cdot \ell_{\mathbf{x}, \mathbf{w}}$ and $\ell_{\mathbf{x}}$ and $\ell_{\mathbf{w}}$ are fixed $\text{poly}(\lambda)$, we have $\text{poly}(\lambda, \log \ell) = \text{poly}(\lambda, \log k)$. Therefore, in the below we express all complexities in terms of ℓ instead of k .

We first examine $|\text{vk}|$. Recall that $\text{vk} = (\text{com}_{\mathcal{G}_{\text{stmt}}}, \text{ck}_{f_{\mathcal{C}}}, (\text{ck}_{J_g})_{g \in \mathcal{G}_{\text{wit}}})$. Note that by Definition 5 fresh commitments are of size at most $\text{poly}(\lambda, \log \ell)$ and by Definition 6 aggregated commitments are identical to fresh ones. Therefore, $|\text{com}_{\mathcal{G}_{\text{stmt}}}| \leq \text{poly}(\lambda, \log \ell)$. Next, by Definition 9 it holds that $|\text{ck}_{f_{\mathcal{C}}}| \leq \text{poly}(\lambda, \log \ell, \log m, \log |f_{\mathcal{C}}|)$ where $m = 1$ in the current context and $|f_{\mathcal{C}}| \leq 2^{\text{poly}(\lambda)}$. Thus, $|\text{ck}_{f_{\mathcal{C}}}| \leq \text{poly}(\lambda, \log \ell)$. Finally, by Definition 13 it holds that $|\text{ck}_{J_g}| \leq \text{poly}(\lambda, \log \ell, \log |J_g|)$ for each $g \in \mathcal{G}_{\text{wit}}$, where $|J_g| = k$ and $|\mathcal{G}_{\text{wit}}| = \ell_{\mathbf{w}}$. Putting everything together, we conclude that $|\text{vk}| \leq \text{poly}(\lambda, \log k) \cdot \ell_{\mathbf{w}}$.

For the runtime of $\text{BARG.EffVer}(\lambda, \text{vk})$, by Definition 6, the two calls to PCFC.Agg take time at most $\text{poly}(\lambda, \log \ell) \cdot |\mathcal{G}_{\text{wit}}| = \text{poly}(\lambda, \log \ell) \cdot \ell_{\mathbf{w}}$ and $\text{poly}(\lambda, \log \ell) \cdot 2$ respectively. Then, each of the $\ell_{\mathbf{w}}$

calls to PCFC.EffProjVer runs in time at most $\text{poly}(\lambda, \log \ell, |J_g|)$ according to Definition 13. Finally, the call to PCFC.EffFuncVer runs in time at most $\text{poly}(\lambda, \log \ell, \log m, \log |f_C|)$ which as analysed above is at most $\text{poly}(\lambda, \log \ell)$. We thus conclude that $\text{BARG.EffVer}(\lambda, \text{vk})$ runs in time at most $\text{poly}(\lambda, \log \ell) \cdot \ell_w$. $\square$

6 Algebraic PCFC from Bilinear Groups

In this section, we give an algebraic construction of a somewhere extractable projective chainable functional commitment scheme PCFC that is sufficient to instantiate Theorem 1, yielding a circuit-succinct BARG for circuit satisfiability.

Let $\mathcal{F}_C = \{\mathcal{C} : \mathbb{F}^\ell \rightarrow \mathbb{F}^\ell : |\mathcal{C}| \leq \ell_C\}$ be the family of arithmetic circuits of size bounded by ℓ_C , where without loss of generality we let the input space be equal to the output space. We define the function family

$$\mathcal{F}_{k, \ell_C} = \{f : \mathbb{F}^{k\ell} \rightarrow \mathbb{F}^{k\ell}; f(\mathbf{x}_1, \dots, \mathbf{x}_k) = (\mathcal{C}_1(\mathbf{x}_1), \dots, \mathcal{C}_k(\mathbf{x}_k)) \wedge \mathcal{C}_i \in \mathcal{F}_C \forall i \in [k]\}.$$

That is, $\mathcal{F}_{k, \ell_C}$ consists of all functions which input k vectors $(\mathbf{x}_i)_{i \in [k]}$ and evaluate some circuit $\mathcal{C}_i \in \mathcal{F}_C$ on $\mathbf{x}_i$ for each $i \in [k]$.

The section is structured in two parts, where we incrementally construct PCFC using only generic operations over pairing groups. We first construct a somewhere extractable PC in Section 6.1 and then equip it with CFC algorithms in Section 6.2 turning it into a PCFC, while ensuring the compatibility of all properties. We remark that we also provide an algebraic interpretation of the compiler in Appendix B when instantiated with the schemes in this section, which offers a different perspective on the scheme.

The results of this section are summarised in the following theorem:

Theorem 2. *Let $\text{bgp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2)$ be a pairing setting description, and let $M = \mathbb{F} = \mathbb{Z}_q$. Assume that the bilateral $\text{MDDH}_{\kappa, \ell, d}$ assumption and the $\text{KerDH}_{\kappa, \ell}$ assumption hold over bgp for some $\kappa \geq 2$ and $\ell, d = \text{poly}(\lambda)$. Then, the construction $\text{PCFC} = (\text{PC}, \text{CFC})$ is a somewhere extractable PCFC with the following properties:*

- (Projective) aggregatable.
- PC is somewhere extractable for $(\mathcal{I}, \mathcal{J}, \bar{\mathcal{M}})$, where $\bar{\mathcal{M}} = \{0, 1\}$ and $\mathcal{I}, \mathcal{J} \subseteq 2^{[\ell]}$ are arbitrary families of sets such that $|I \cap J| \leq 1$ for every $I \in \mathcal{I}, J \in \mathcal{J}$.
- CFC is projective chain binding for $(\mathcal{F}_{k, \ell_C}, \mathcal{I}')$ where $\mathcal{I}' = \{\{i(\ell - 1) + 1, \dots, i\ell\} : i \in [k]\}$.
- The commitment key size $|\text{ck}|$ and the prover running time are $\mathcal{O}(\lambda \cdot \kappa^3 \cdot k^5 \cdot \ell_C^5)$.
- Commitments have size $|\text{com}| = \mathcal{O}(\lambda \cdot \kappa)$.
- The PC subspace proofs have size $|\pi^b| = \mathcal{O}(\lambda \cdot \kappa^2)$.
- The PCFC functional proofs have size $|\pi^f| = \mathcal{O}(\lambda \cdot \kappa^2)$.
- PCFC efficient verification runs in time $\mathcal{O}(\lambda \cdot \kappa^2)$.

Proof. The proof follows from the constructions in Section 6.1 and Section 6.2 and from the series of lemmas proven there. We state the different properties: a) Binding follows from Lemma 6. b) Projection correctness follows from Lemma 7. c) Setup indistinguishability follows from both Lemma 8 and Lemma 14. d) Somewhere completeness follows from Lemma 9. e) Somewhere extractability follows from Lemma 10. f) Aggregatability and projective aggregatability follow from Lemma 12. g) Projective chain binding follows from Theorem 3. h) Efficient verification follows from both Lemma 13 and Lemma 16. $\square$

As κ is a small constant (e.g., $\kappa = 2$), the combination of Theorem 1 and Theorem 2 yields the following corollary:

Corollary 1. *Under the assumptions of Theorem 2, there exists an algebraic somewhere extractable BARG for NP such that:*

- The setup size $|\text{crs}|$ and the prover running time are $\mathcal{O}(\lambda \cdot k^5 \cdot \ell_C^5)$.
- The proof size is $|\pi| = \mathcal{O}(\lambda \cdot \ell_w)$.
- Efficient verification runs in time $\mathcal{O}(\lambda \cdot \ell_w)$.

6.1 Algebraic PC from Bilinear Groups

We introduce a construction of a somewhere extractable PC. Conceptually, our PC is an adaptation of the proof systems in the WW22 BARG [WW22] to work over a larger projective commitment key that is compatible with the WW24 FC proof systems [WW24]. Notably, the projective space in our PC has dimension κ , as opposed to 1 in the commitments in the WW22 BARG. Therefore, our commitments are vectors of length 2κ instead of $\kappa + 1$. Besides, our scheme explicitly supports subvector commitments, as well as their aggregation. We remark that all algorithms are well defined for any $J \in \mathcal{J} = 2^{[\ell]}$. The message space is defined as $\mathcal{M} = \mathbb{Z}_q = \mathbb{F}$ and the small message space as $\bar{\mathcal{M}} = \{0, 1\}$. All vectors $\mathbf{x} \in \mathbb{F}^\ell$ are column vectors. As in Section 6.2 we borrow algorithms from the [WW24] CFC, we follow their notation closely.

Our PC construction.

Setup($1^\lambda, 1^\ell$):

- Generate a bilinear group description $\text{bgp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2) \leftarrow \mathcal{BG}(1^\lambda)$, and let $\mathbb{F} := \mathbb{Z}_q$ and κ be the κ -Lin parameter.
- Sample random invertible $\hat{\mathbf{B}}, \check{\mathbf{B}} \leftarrow_{\$} \mathbb{F}^{2\kappa \times 2\kappa}$ and let $\hat{\mathbf{B}}^* = \hat{\mathbf{B}}^{-1}$ and $\check{\mathbf{B}}^* = \check{\mathbf{B}}^{-1}$.
- Split $\hat{\mathbf{B}} = \begin{bmatrix} \hat{\mathbf{B}}_1 \\ \hat{\mathbf{B}}_2 \end{bmatrix}$ where $\hat{\mathbf{B}}_1, \hat{\mathbf{B}}_2 \in \mathbb{F}^{\kappa \times 2\kappa}$ and split $\hat{\mathbf{B}}^* = \begin{bmatrix} \hat{\mathbf{B}}_1^* & \hat{\mathbf{B}}_2^* \end{bmatrix}$ where $\hat{\mathbf{B}}_1^*, \hat{\mathbf{B}}_2^* \in \mathbb{F}^{2\kappa \times \kappa}$. By construction, $\hat{\mathbf{B}}_1 \cdot \hat{\mathbf{B}}_1^* = \hat{\mathbf{B}}_2 \cdot \hat{\mathbf{B}}_2^* = \mathbf{I}_\kappa$ and $\hat{\mathbf{B}}_1 \cdot \hat{\mathbf{B}}_2^* = \hat{\mathbf{B}}_2 \cdot \hat{\mathbf{B}}_1^* = \mathbf{0}_\kappa$. Define $\check{\mathbf{B}}_1, \check{\mathbf{B}}_2 \in \mathbb{F}^{\kappa \times 2\kappa}$ and $\check{\mathbf{B}}_1^*, \check{\mathbf{B}}_2^* \in \mathbb{F}^{2\kappa \times \kappa}$ analogously.
- Sample $\hat{\mathbf{S}}_1, \check{\mathbf{S}}_1 \leftarrow_{\$} \mathbb{F}^{\kappa \times \ell}$, and define $\hat{\mathbf{T}} = \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{S}}_1 \in \mathbb{F}^{2\kappa \times \ell}$ and $\check{\mathbf{T}} = \check{\mathbf{B}}_1^* \cdot \check{\mathbf{S}}_1 \in \mathbb{F}^{2\kappa \times \ell}$. Let also $\hat{\mathbf{t}}_i \in \mathbb{F}^{2\kappa}$ be each of the column vectors in $\hat{\mathbf{T}}$, and similarly for $\check{\mathbf{t}}_i \in \mathbb{F}^{2\kappa}$ and $\check{\mathbf{T}}$.
- Finally, sample $\mathbf{R}_{i,j} \leftarrow_{\$} \mathbb{F}^{\kappa \times \kappa}$ and set

$$\hat{\mathbf{W}}_{i,j} \leftarrow -\hat{\mathbf{B}}_1^* \mathbf{R}_{i,j}^T, \quad \check{\mathbf{W}}_{i,j} \leftarrow \check{\mathbf{B}}_1^* (\check{\mathbf{S}}_{1,i} \hat{\mathbf{S}}_{1,j}^T + \mathbf{R}_{i,j}).$$

Output the following commitment key:

$$\text{ck} := \left([\check{\mathbf{T}}]_1, [\hat{\mathbf{T}}]_2, [\check{\mathbf{B}}_1^*]_1, [\hat{\mathbf{B}}_1^*]_2, \left\{ [\check{\mathbf{W}}_{i,j}]_1, [\hat{\mathbf{W}}_{i,j}]_2 \right\}_{i,j \in [\ell]} \right).$$

Given any subset $J \subseteq [\ell]$, we let $\hat{\mathbf{t}}_J \leftarrow \sum_{i \in J} \hat{\mathbf{t}}_i$ and $\check{\mathbf{t}}_J \leftarrow \sum_{i \in J} \check{\mathbf{t}}_i$. Note that $[\check{\mathbf{t}}_J]_1, [\hat{\mathbf{t}}_J]_2$ can be computed from ck for any $J \subseteq [\ell]$.

ProjSetup($1^\lambda, 1^\ell, I$): Let $\mathbf{P}_I \in \mathbb{F}^{\ell \times \ell}$ be the (diagonal) projection matrix associated to I , defined by $\mathbf{P}_{I,i} = 1$ if $i \in I$ and $\mathbf{P}_{I,i} = 0$ elsewhere. Sample all matrices as before, and additionally sample $\hat{\mathbf{S}}_2, \check{\mathbf{S}}_2 \leftarrow_{\$} \mathbb{F}^{\kappa \times \ell}$. Then, set

$$\hat{\mathbf{T}} \leftarrow \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{S}}_1 + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{S}}_2 \cdot \mathbf{P}_I, \quad \check{\mathbf{T}} \leftarrow \check{\mathbf{B}}_1^* \cdot \check{\mathbf{S}}_1 + \check{\mathbf{B}}_2^* \cdot \check{\mathbf{S}}_2 \cdot \mathbf{P}_I$$

$$\hat{\mathbf{W}}_{i,j} \leftarrow \begin{cases} -\hat{\mathbf{B}}_1^* \mathbf{R}_{i,j}^T + \hat{\mathbf{t}}_i \check{\mathbf{s}}_{1,j}^T & \text{if } j \in I \\ -\hat{\mathbf{B}}_1^* \mathbf{R}_{i,j}^T & \text{if } j \notin I \end{cases}, \quad \check{\mathbf{W}}_{i,j} \leftarrow \begin{cases} \check{\mathbf{B}}_1^* \mathbf{R}_{i,j} & \text{if } j \in I \\ \check{\mathbf{B}}_1^* \mathbf{R}_{i,j} + \check{\mathbf{t}}_i \hat{\mathbf{s}}_{1,j}^T & \text{if } j \notin I \end{cases}.$$

Output the commitment key as before, and output $\text{td} := (I, \check{\mathbf{B}}_2, \check{\mathbf{S}}_2, \hat{\mathbf{B}}_2, \hat{\mathbf{S}}_2)$.

Com($\text{ck}, J, \mathbf{x}$): Output $[\hat{\mathbf{c}}]_2 \leftarrow \sum_{i \in J} [\hat{\mathbf{t}}_i]_2 \cdot x_i$.

ProjCom($\text{td}, J, \mathbf{x}$): Output $[\hat{\mathbf{d}}]_2 \leftarrow \sum_{i \in I \cap J} [\hat{\mathbf{s}}_{2,i}]_2 \cdot x_i$.

Proj(td, com): Parse $\hat{\mathbf{B}}_2$ from td , and parse $\text{com} = [\hat{\mathbf{c}}]_2$. Output $[\hat{\mathbf{d}}]_2 \leftarrow \hat{\mathbf{B}}_2 \cdot [\hat{\mathbf{c}}]_2$.

ProjProve($\text{ck}, J, \mathbf{x}$):

- Compute the twin commitment $[\check{\mathbf{c}}]_1 \leftarrow \sum_{i \in J} [\check{\mathbf{t}}_i]_1 x_i$.
- Compute the booleanity proofs:

$$[\check{\mathbf{\Pi}}_1]_1 \leftarrow \sum_{i,j \in J, i \neq j} [\check{\mathbf{W}}_{i,j}]_1 \cdot x_i(x_j - 1), \quad [\hat{\mathbf{\Pi}}_1]_2 \leftarrow \sum_{i,j \in J, i \neq j} [\hat{\mathbf{W}}_{i,j}]_2 \cdot x_i(x_j - 1),$$

$$[\check{\mathbf{\Pi}}_2]_1 \leftarrow \sum_{i,j \in J, i \neq j} [\check{\mathbf{W}}_{i,j}]_1 \cdot x_j(x_i - 1), \quad [\hat{\mathbf{\Pi}}_2]_2 \leftarrow \sum_{i,j \in J, i \neq j} [\hat{\mathbf{W}}_{i,j}]_2 \cdot x_j(x_i - 1).$$

- Output $\pi \leftarrow ([\check{\mathbf{c}}]_1, [\check{\mathbf{\Pi}}_1]_1, [\hat{\mathbf{\Pi}}_1]_2, [\check{\mathbf{\Pi}}_2]_1, [\hat{\mathbf{\Pi}}_2]_2)$.

ProjVer($\text{ck}, J, \text{com}, \pi$): Parse $\pi = ([\check{\mathbf{c}}]_1, [\check{\mathbf{\Pi}}_1]_1, [\hat{\mathbf{\Pi}}_1]_2, [\check{\mathbf{\Pi}}_2]_1, [\hat{\mathbf{\Pi}}_2]_2)$ and $\text{com} = [\hat{\mathbf{c}}]_2$. Compute $[\check{\mathbf{t}}_J]_1, [\hat{\mathbf{t}}_J]_2$. Output 1 if and only if both of the following hold:

$$[\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{t}}_J^T]_2 = [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{c}}^T]_2 + [\check{\mathbf{B}}_1^*]_1 \cdot [\hat{\mathbf{\Pi}}_1^T]_2 + [\check{\mathbf{\Pi}}_1]_1 \cdot [\hat{\mathbf{B}}_1^{*T}]_2. \quad (1)$$

$$[\check{\mathbf{t}}_J]_1 \cdot [\hat{\mathbf{c}}^T]_2 = [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{c}}^T]_2 + [\check{\mathbf{B}}_1^*]_1 \cdot [\hat{\mathbf{\Pi}}_2^T]_2 + [\check{\mathbf{\Pi}}_2]_1 \cdot [\hat{\mathbf{B}}_1^{*T}]_2. \quad (2)$$

PreProjVer(ck, J): Output $\text{ck}_J \leftarrow ([\check{\mathbf{t}}_J]_1, [\hat{\mathbf{t}}_J]_2, [\check{\mathbf{B}}_1^*]_1, [\hat{\mathbf{B}}_1^{*T}]_2)$.

EffProjVer($\text{ck}_J, \text{com}, \pi$): Check equations 1, 2 from ProjVer and output 1 if and only if both hold.

ProjExt(td, J, com): Parse $(I, \hat{\mathbf{S}}_2)$ from td and compute $[\hat{\mathbf{d}}]_2 \leftarrow \text{Proj}(\text{td}, \text{com})$. Then, find a vector $\tilde{\mathbf{x}} \in \{0, 1\}^{I \cap J}$ such that $[\hat{\mathbf{d}}]_2 = \sum_{i \in I \cap J} [\hat{\mathbf{s}}_{2,i}]_2 \cdot \tilde{x}_i$. Output $\tilde{\mathbf{x}}$ if found. Otherwise, output $\perp$.

Note that this algorithm runs in time $\mathcal{O}(2^{|I \cap J|} \cdot \text{poly}(\lambda))$.

Agg($\text{ck}, (\text{com}_J)_{J \in \mathcal{J}'}) \rightarrow \text{com}$: Output $[\hat{\mathbf{c}}]_2 \leftarrow \sum_{J \in \mathcal{J}'} [\hat{\mathbf{c}}_J]_2$.

ProjAgg($\text{ck}, (\text{pcom}_J)_{J \in \mathcal{J}'}) \rightarrow \text{pcom}$: Output $[\hat{\mathbf{d}}]_2 \leftarrow \sum_{J \in \mathcal{J}'} [\hat{\mathbf{d}}_J]_2$.

Proofs of the properties of PC. Next, we prove that PC satisfies all the properties required by Theorem 2. We remark that most of the properties and the proofs follow the blueprints of [WW22] with minor modifications. Essentially, we adapt their proofs to work over a projective space of dimension κ , as the commitments in the WW22 BARG implicitly have a projective space of dimension 1.

Lemma 6. *Assume that the $\text{KerDH}_{\kappa,\ell}$ assumption holds over bgp . Then, se-PC is binding (Definition 5).*

Proof. Let $\mathcal{A}(\text{ck})$ be a PPT algorithm against binding that returns $\mathbf{x}, \mathbf{x}' \in \mathbb{F}^J$ such that $\mathbf{x} \neq \mathbf{x}'$ and $\text{Com}(\text{ck}, J, \mathbf{x}) = \text{Com}(\text{ck}, J, \mathbf{x}')$. Note that by construction $[\hat{\mathbf{T}}_J]_2 \cdot (\mathbf{x} - \mathbf{x}') = [\mathbf{0}]_2$.

We show how to use $\mathcal{A}$ to build an adversary $\mathcal{B}$ against $\text{KerDH}_{\kappa,\ell}$. $\mathcal{B}$ receives $[\hat{\mathbf{S}}_1]_2 \in \mathbb{G}_2^{\kappa \times \ell}$ from its challenger, generates all other matrices involved in Setup including $\hat{\mathbf{B}}_1^* \in \mathbb{F}^{2\kappa \times \kappa}$, and generates a commitment key ck which sends to $\mathcal{A}$. $\mathcal{B}$ receives $\mathbf{x}, \mathbf{x}'$ from $\mathcal{A}(\text{ck})$ and outputs the vector $\mathbf{u} = \mathbf{x} - \mathbf{x}' \neq \mathbf{0}$. We argue that $\mathbf{u}$ is in the kernel of $[\hat{\mathbf{S}}_1]_2$. Let $\mathbf{v} = \hat{\mathbf{S}}_1 \cdot \mathbf{u}$. Note that $\hat{\mathbf{B}}_1^* \mathbf{v} = \mathbf{0}$ if and only if $\mathbf{v} = \mathbf{0}$, as $\hat{\mathbf{B}}_1^*$ has maximal rank κ . Therefore, as $[\hat{\mathbf{T}}]_2 \mathbf{u} = [\hat{\mathbf{B}}_1^*]_2 \mathbf{v} = [\mathbf{0}]_2$, we conclude that $\mathbf{v} = \mathbf{0}$, and $\mathcal{B}$ successfully wins the $\text{KerDH}_{\kappa,\ell}$ game.

Lemma 7. *se-PC satisfies projection correctness (Definition 10).*

Proof. Let $\mathbf{x} \in \mathbb{F}^J$, $J \subseteq [\ell]$, $I \subseteq [\ell]$ and $\text{ck} \leftarrow \text{ProjSetup}(1^\lambda, 1^\ell, I)$. Then,

$$\begin{aligned} \text{Proj}(\text{td}, \text{Com}(\text{ck}, J, \mathbf{x})) &= \hat{\mathbf{B}}_2 \cdot \left[\sum_{i \in J} \hat{\mathbf{t}}_i \cdot x_i \right]_1 = \hat{\mathbf{B}}_2 \cdot [\hat{\mathbf{T}} \cdot \mathbf{x}]_1 \\ &= \hat{\mathbf{B}}_2 \cdot [\hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{S}}_1 + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{S}}_2 \cdot \mathbf{P}_I]_1 \cdot \mathbf{x} = [\hat{\mathbf{S}}_2 \cdot \mathbf{P}_I \cdot \mathbf{x}]_1 \\ &= \left[\sum_{i \in I \cap J} \hat{\mathbf{s}}_{2,i} \cdot x_i \right]_1. \end{aligned}$$

Where the simplification follows by construction of the commitment key, as $\hat{\mathbf{B}}_2 \cdot \hat{\mathbf{B}}_2^* = \mathbf{I}_k$ and $\hat{\mathbf{B}}_2 \cdot \hat{\mathbf{B}}_1^* = \mathbf{0}_k$.

Lemma 8. *Assume that the $\text{MDDH}_{\kappa,\ell,2\kappa}$ assumption holds over bgp . Then, PCFC satisfies setup indistinguishability (Definition 10).*

Proof. We define a series of game hops as follows. Note that we display each columnn vectors $\check{\mathbf{t}}_i, \hat{\mathbf{t}}_i$ of the matrices $\check{\mathbf{T}}, \hat{\mathbf{T}}$ separately.

– In Hyb^0 , we have the key in normal mode, where

$$\begin{aligned} \hat{\mathbf{t}}_i &\leftarrow \hat{\mathbf{B}}_1^* \hat{\mathbf{s}}_{1,i}, \quad \check{\mathbf{t}}_i \leftarrow \check{\mathbf{B}}_1^* \check{\mathbf{s}}_{1,i}. \\ \hat{\mathbf{W}}_{i,j} &\leftarrow -\hat{\mathbf{B}}_1^* \mathbf{R}_{i,j}^T, \quad \check{\mathbf{W}}_{i,j} \leftarrow \check{\mathbf{B}}_1^* (\check{\mathbf{s}}_{1,i} \hat{\mathbf{s}}_{1,j}^T + \mathbf{R}_{i,j}) \end{aligned}$$

– In Hyb^1 , we change the distribution of $\hat{\mathbf{W}}_{i,j}, \check{\mathbf{W}}_{i,j}$. Namely, we sample $\hat{\mathbf{S}}_2, \check{\mathbf{S}}_2 \leftarrow_{\$} \mathbb{F}^{\kappa \times \ell}$ and set

$$\hat{\mathbf{W}}_{i,j} \leftarrow \begin{cases} -\hat{\mathbf{B}}_1^* \mathbf{R}_{i,j}^T + \hat{\mathbf{t}}_i \check{\mathbf{s}}_{1,j}^T & \text{if } j \in I \\ -\hat{\mathbf{B}}_1^* \mathbf{R}_{i,j}^T & \text{if } j \notin I \end{cases}, \quad \check{\mathbf{W}}_{i,j} \leftarrow \begin{cases} \check{\mathbf{B}}_1^* \mathbf{R}_{i,j} & \text{if } j \in I \\ \check{\mathbf{B}}_1^* \mathbf{R}_{i,j} + \check{\mathbf{t}}_i \hat{\mathbf{s}}_{1,j}^T & \text{if } j \notin I \end{cases}.$$

- In Hyb^2 , we change the way that $\hat{\mathbf{t}}_i$ is sampled. Namely, we set $\hat{\mathbf{W}}_{i,j}, \check{\mathbf{W}}_{i,j}$ as in Hyb^1 , sample $\hat{\mathbf{u}}_i \leftarrow \$ \mathbb{F}^{2\kappa}$, and set

$$\hat{\mathbf{t}}_i \leftarrow \begin{cases} \hat{\mathbf{u}}_i & \text{if } i \in I \\ \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} & \text{if } i \notin I \end{cases}, \quad \check{\mathbf{t}}_i \leftarrow \check{\mathbf{B}}_1^* \cdot \check{\mathbf{s}}_{1,i}$$

- In Hyb^3 , we sample $\hat{\mathbf{t}}_i$ in trapdoor mode. Namely, we set $\hat{\mathbf{W}}_{i,j}, \check{\mathbf{W}}_{i,j}$ as in Hyb^2 , and then

$$\hat{\mathbf{t}}_i \leftarrow \begin{cases} \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{s}}_{2,i} & \text{if } i \in I \\ \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} & \text{if } i \notin I \end{cases}, \quad \check{\mathbf{t}}_i \leftarrow \check{\mathbf{B}}_1^* \cdot \check{\mathbf{s}}_{1,i}$$

- In Hyb^4 , we change the way that $\check{\mathbf{t}}_i$ is sampled. Namely, we set $\hat{\mathbf{W}}_{i,j}, \check{\mathbf{W}}_{i,j}$ as in Hyb^3 , sample $\check{\mathbf{u}}_i \leftarrow \$ \mathbb{F}^{2\kappa}$, and set

$$\hat{\mathbf{t}}_i \leftarrow \begin{cases} \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{s}}_{2,i} & \text{if } i \in I \\ \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} & \text{if } i \notin I \end{cases}, \quad \check{\mathbf{t}}_i \leftarrow \begin{cases} \check{\mathbf{u}}_i & \text{if } i \in I \\ \check{\mathbf{B}}_1^* \cdot \check{\mathbf{s}}_{1,i} & \text{if } i \notin I \end{cases}$$

- In Hyb^5 , we sample the full key in trapdoor mode. Namely, we set $\hat{\mathbf{W}}_{i,j}, \check{\mathbf{W}}_{i,j}$ as in Hyb^4 , and set

$$\hat{\mathbf{t}}_i \leftarrow \begin{cases} \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{s}}_{2,i} & \text{if } i \in I \\ \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} & \text{if } i \notin I \end{cases}, \quad \check{\mathbf{t}}_i \leftarrow \begin{cases} \check{\mathbf{B}}_1^* \cdot \check{\mathbf{s}}_{1,i} + \check{\mathbf{B}}_2^* \cdot \check{\mathbf{s}}_{2,i} & \text{if } i \in I \\ \check{\mathbf{B}}_1^* \cdot \check{\mathbf{s}}_{1,i} & \text{if } i \notin I \end{cases}$$

We proceed to bound the advantage between the different game hops.

$\text{Hyb}^0 \rightarrow \text{Hyb}^1$: . This is a syntactic relabeling that follows from generating $\mathbf{R}_{i,j}$ as follows: Sample $\bar{\mathbf{R}}_{i,j} \leftarrow \$ \mathbb{F}^{\kappa \times \kappa}$, and let

$$\mathbf{R}_{i,j} \leftarrow \begin{cases} \bar{\mathbf{R}}_{i,j} - \check{\mathbf{s}}_{1,i} \hat{\mathbf{s}}_{1,j}^T & \text{if } j \in I \\ \bar{\mathbf{R}}_{i,j} & \text{if } j \notin I \end{cases}$$

Clearly, $\mathbf{R}_{i,j}$ still follows a uniform distribution over $\mathbb{F}^{\kappa \times \kappa}$, and so the view of the adversary in both games is identical. Therefore,

$$\text{Adv}_{\text{se-PC}, \mathcal{A}}^0(\lambda) \leq \text{Adv}_{\text{se-PC}, \mathcal{A}}^1(\lambda).$$

$\text{Hyb}^1 \rightarrow \text{Hyb}^2$: This follows by a sequence of $|I|$ sub-hybrids where we change each $\hat{\mathbf{t}}_j$ at a time. In each of the sub-hybrids, we carry out a reduction to $\text{MDDH}_{\kappa, 1, 2\kappa}$ where the challenge matrix is $[\hat{\mathbf{B}}_1^*]_2$. We specify the first sub-hybrid, where we change the smallest index $i^* \in I$. Let $\mathcal{A}$ be an adversary against the first sub-hybrid.

- Let $\mathcal{B}$ be an adversary against $\text{MDDH}_{\kappa, 1, 2\kappa}$ which receives a challenge matrix $[\hat{\mathbf{B}}_1^*]_2 \in \mathbb{G}_2^{2\kappa \times \kappa}$ and a challenge vector $[\mathbf{z}]_2 \in \mathbb{G}_2^{2\kappa}$.
- $\mathcal{B}$ generates a ck as follows (we specify only the modified terms):

$$[\hat{\mathbf{t}}_i]_2 \leftarrow \begin{cases} [\mathbf{z}]_2 & \text{if } i = i^* \\ [\hat{\mathbf{B}}_1^*]_2 \cdot \hat{\mathbf{s}}_{1,i} & \text{if } i \neq i^* \end{cases},$$

$$[\hat{\mathbf{W}}_{i,j}]_2 \leftarrow \begin{cases} -[\hat{\mathbf{B}}_1^*]_2 \mathbf{R}_{i,j}^T + [\hat{\mathbf{t}}_i]_2 \check{\mathbf{s}}_{1,j}^T & \text{if } j = i^* \\ -[\hat{\mathbf{B}}_1^*]_2 \mathbf{R}_{i,j}^T & \text{if } j \neq i^* \end{cases}.$$

Note, in particular, that the terms $\hat{\mathbf{s}}_{1,j}$ are always used in $\hat{\mathbf{t}}_i$ for $j \notin I$, and so $\check{\mathbf{W}}_{i,j}$ does never change its distribution.

– The distribution of the key is identical to Hyb^1 whenever $\mathbf{z} = \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i^*}$ as $\hat{\mathbf{s}}_{1,i^*}$ is sampled uniformly at random. On the other hand, it is identical to Hyb^2 whenever $\mathbf{z}$ is uniform. Hence, $\mathcal{B}$ simply gives the key to $\mathcal{A}$ and outputs whatever $\mathcal{A}$ outputs. We conclude that

$$\text{Adv}_{\text{se-PC}, \mathcal{A}}^1(\lambda) \leq \text{Adv}_{\text{se-PC}, \mathcal{A}}^2(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{mddh}}(\lambda).$$

$\text{Hyb}^2 \rightarrow \text{Hyb}^3$: This is just a syntactic relabeling, as the distribution of $\left[\hat{\mathbf{t}}_i\right]_2$ is identical in both games. To see this, note that $\hat{\mathbf{B}}_1^*$ and $\hat{\mathbf{B}}_2^*$ together form a basis of the 2κ -dimensional vector space, and that $\hat{\mathbf{s}}_{1,i}, \hat{\mathbf{s}}_{2,i}$ are sampled at random. Hence,

$$\text{Adv}_{\text{se-PC}, \mathcal{A}}^2(\lambda) \leq \text{Adv}_{\text{se-PC}, \mathcal{A}}^3(\lambda).$$

$\text{Hyb}^3 \rightarrow \text{Hyb}^4$: The argument is the same as in between Hyb^1 and Hyb^2 , so we omit it. It follows that

$$\text{Adv}_{\text{se-PC}, \mathcal{A}}^3(\lambda) \leq \text{Adv}_{\text{se-PC}, \mathcal{A}}^4(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{mddh}}(\lambda).$$

$\text{Hyb}^4 \rightarrow \text{Hyb}^5$: The argument is the same as in between Hyb^2 and Hyb^3 , so we omit it. It follows that

$$\text{Adv}_{\text{se-PC}, \mathcal{A}}^4(\lambda) \leq \text{Adv}_{\text{se-PC}, \mathcal{A}}^5(\lambda).$$

The lemma follows by collecting the different advantages. $\square$

Lemma 9. *se-PC satisfies somewhere completeness (Definition 12).*

Proof. Let $I, J \subseteq [\ell]$ and $\mathbf{x} \in \{0, 1\}^J$. Note that $x_i^2 = x_i$ for every $i \in J$. First, note that for any $i, j \in [\ell]$, the following equality holds

$$\check{\mathbf{B}}_1^* \cdot \hat{\mathbf{W}}_{i,j}^T + \check{\mathbf{W}}_{i,j} \cdot \hat{\mathbf{B}}_1^{*T} = -\check{\mathbf{B}}_1^* \cdot \mathbf{R}_{i,j} \cdot \hat{\mathbf{B}}_1^{*T} + \check{\mathbf{B}}_1^* \cdot (\check{\mathbf{s}}_{1,i} \hat{\mathbf{s}}_{1,j}^T + \mathbf{R}_{i,j}^T) \cdot \hat{\mathbf{B}}_1^{*T} = \check{\mathbf{t}}_i \hat{\mathbf{t}}_j^T$$

We start on the RHS of the first verification equation in the ProjVer algorithm.

$$\begin{aligned} \check{\mathbf{c}} \cdot \hat{\mathbf{c}}^T + \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{\Pi}}_1^T + \check{\mathbf{\Pi}}_1 \cdot \hat{\mathbf{B}}_1^{*T} &= \\ &= \sum_{i,j \in J} \check{\mathbf{t}}_i \hat{\mathbf{t}}_j^T x_i x_j + \sum_{i,j \in J, i \neq j} \left(\check{\mathbf{B}}_1^* \cdot \hat{\mathbf{W}}_{i,j}^T + \check{\mathbf{W}}_{i,j} \cdot \hat{\mathbf{B}}_1^{*T} \right) x_i (x_j - 1) \\ &= \sum_{i,j \in J} \check{\mathbf{t}}_i \hat{\mathbf{t}}_j^T x_i x_j + \sum_{i,j \in J, i \neq j} \check{\mathbf{t}}_i \hat{\mathbf{t}}_j^T x_i (x_j - 1) \\ &= \sum_{i \in J} \check{\mathbf{t}}_i \hat{\mathbf{t}}_i^T x_i^2 + \sum_{i,j \in J, i \neq j} \check{\mathbf{t}}_i \hat{\mathbf{t}}_j^T x_i \\ &= \left(\sum_{i \in J} \check{\mathbf{t}}_i x_i \right) \cdot \left(\sum_{j \in J} \hat{\mathbf{t}}_j^T \right) \\ &= \check{\mathbf{c}} \cdot \hat{\mathbf{t}}_J^T \end{aligned}$$

$\square$

Correctness for the second verification equation follows analogously.

Lemma 10. *se-PC satisfies statistical somewhere extractability (Definition 12) for any sets $I \in \mathcal{I}$, $J \in \mathcal{I}$ such that $|I \cap J| = 1$.*

Proof. Let $I, J \subseteq [\ell]$ such that $I \cap J = \{i^*\}$. Our goal is to prove that the extraction is correct and matches the projection of **com** except with negligible probability against unbounded adversaries. The proof follows the blueprint of the proof of [WW22, Lemma 4.12], although the trapdoor setup in our construction differs. We start by proving the following lemma.

Lemma 11. *With respect to the matrices defined in ProjSetup, any vector $\mathbf{v} \in \mathbb{F}^{2\kappa}$ can be uniquely written as*

$$\mathbf{v} = \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{u}}_1 + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{u}}_2, \quad \text{and} \quad \mathbf{v} = \check{\mathbf{B}}_1^* \cdot \check{\mathbf{u}}_1 + \check{\mathbf{B}}_2^* \cdot \check{\mathbf{u}}_2$$

for some vectors $\hat{\mathbf{u}}_1, \hat{\mathbf{u}}_2, \check{\mathbf{u}}_1, \check{\mathbf{u}}_2 \in \mathbb{F}^\kappa$.

Proof. Note that $\hat{\mathbf{B}}^* = [\hat{\mathbf{B}}_1^* \ \hat{\mathbf{B}}_2^*]$ where $\hat{\mathbf{B}}_1^*, \hat{\mathbf{B}}_2^* \in \mathbb{F}^{2\kappa \times \kappa}$ form a basis of $\mathbb{F}^{2\kappa \times 2\kappa}$. Hence, any vector can be written uniquely as a combination of 2κ basis vectors, i.e.,

$$\mathbf{v} = [\hat{\mathbf{B}}_1^* \ \hat{\mathbf{B}}_2^*] \cdot \begin{bmatrix} \hat{\mathbf{u}}_1 \\ \hat{\mathbf{u}}_2 \end{bmatrix}.$$

And similarly for the basis formed by $\check{\mathbf{B}}_1^*, \check{\mathbf{B}}_2^*$. □

Let $([\check{\mathbf{c}}]_1, [\hat{\mathbf{c}}]_2, [\check{\mathbf{\Pi}}_1]_1, [\hat{\mathbf{\Pi}}_1]_2, [\check{\mathbf{\Pi}}_2]_1, [\hat{\mathbf{\Pi}}_2]_2)$ be commitments and proof terms that pass the two verification equations. Following the lemma, let $\check{\mathbf{u}}_1, \check{\mathbf{u}}_2, \hat{\mathbf{u}}_1, \hat{\mathbf{u}}_2 \in \mathbb{F}^\kappa$ be the unique vectors such that $\check{\mathbf{c}} = \check{\mathbf{B}}_1^* \cdot \check{\mathbf{u}}_1 + \check{\mathbf{B}}_2^* \cdot \check{\mathbf{u}}_2$ and $\hat{\mathbf{c}} = \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{u}}_1 + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{u}}_2$. When we rewrite the following terms that appear in the verification relation in the above form, we obtain

$$\begin{aligned} \check{\mathbf{c}} \cdot \hat{\mathbf{t}}_J^T &= (\check{\mathbf{B}}_1^* \cdot \check{\mathbf{u}}_1 + \check{\mathbf{B}}_2^* \cdot \check{\mathbf{u}}_2) \cdot \left(\sum_{i \in J} \hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{s}}_{1,i} + \sum_{i \in I \cap J} \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{s}}_{2,i} \right)^T, \\ \check{\mathbf{c}} \cdot \hat{\mathbf{c}}^T &= (\check{\mathbf{B}}_1^* \cdot \check{\mathbf{u}}_1 + \check{\mathbf{B}}_2^* \cdot \check{\mathbf{u}}_2) \cdot (\hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{u}}_1 + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{u}}_2)^T, \\ \check{\mathbf{t}}_J \cdot \hat{\mathbf{c}}^T &= \left(\sum_{i \in J} \check{\mathbf{B}}_1^* \cdot \check{\mathbf{s}}_{1,i} + \sum_{i \in I \cap J} \check{\mathbf{B}}_2^* \cdot \check{\mathbf{s}}_{2,i} \right) \cdot (\hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{u}}_1 + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{u}}_2)^T. \end{aligned}$$

Now, consider the two verification equations:

$$\begin{aligned} [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{t}}_J^T]_2 &= [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{c}}^T]_2 + [\check{\mathbf{B}}_1^*]_1 \cdot [\hat{\mathbf{\Pi}}_1^T]_2 + [\check{\mathbf{\Pi}}_1]_1 \cdot [\hat{\mathbf{B}}_1^{*T}]_2, \\ [\check{\mathbf{t}}_J]_1 \cdot [\hat{\mathbf{c}}^T]_2 &= [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{c}}^T]_2 + [\check{\mathbf{B}}_1^*]_1 \cdot [\hat{\mathbf{\Pi}}_2^T]_2 + [\check{\mathbf{\Pi}}_2]_1 \cdot [\hat{\mathbf{B}}_1^{*T}]_2. \end{aligned}$$

If we now project each term to the projective subspace by multiplying each term by $\check{\mathbf{B}}_2$ on the left and $\hat{\mathbf{B}}_2^T$ on the right, we get

$$\begin{aligned} \check{\mathbf{B}}_2 [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{t}}_J^T]_2 \hat{\mathbf{B}}_2^T &= \check{\mathbf{B}}_2 [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{c}}^T]_2 \hat{\mathbf{B}}_2^T, \\ \check{\mathbf{B}}_2 [\check{\mathbf{t}}_J]_1 \cdot [\hat{\mathbf{c}}^T]_2 \hat{\mathbf{B}}_2^T &= \check{\mathbf{B}}_2 [\check{\mathbf{c}}]_1 \cdot [\hat{\mathbf{c}}^T]_2 \hat{\mathbf{B}}_2^T. \end{aligned}$$

Where we have used that $\check{\mathbf{B}}_2 \cdot \check{\mathbf{B}}_1^* = \hat{\mathbf{B}}_2 \cdot \hat{\mathbf{B}}_1^* = \mathbf{0}_k$, and so the terms with a $\mathbf{B}_1^*$ -type matrix are canceled out. Plugging in the expressions for the remaining terms, and using that $I \cap J = \{i^*\}$, we derive

$$\begin{aligned}\check{\mathbf{u}}_2 \cdot \hat{\mathbf{s}}_{2,i^*}^T &= \check{\mathbf{u}}_2 \cdot \hat{\mathbf{u}}_2^T, \\ \check{\mathbf{s}}_{2,i^*} \cdot \hat{\mathbf{u}}_2^T &= \check{\mathbf{u}}_2 \cdot \hat{\mathbf{u}}_2^T,\end{aligned}$$

Where we have used that $\check{\mathbf{B}}_2 \cdot \check{\mathbf{B}}_2^* = \hat{\mathbf{B}}_2 \cdot \hat{\mathbf{B}}_2^* = \mathbf{1}_k$. For the above equations to hold, we must have that:

$$\check{\mathbf{u}}_2 \cdot (\hat{\mathbf{s}}_{2,i^*}^T - \hat{\mathbf{u}}_2^T) = \mathbf{0}_k = (\check{\mathbf{s}}_{2,i^*} - \check{\mathbf{u}}_2) \cdot \hat{\mathbf{u}}_2^T.$$

We analyze the first equality. As these are outer products, we have that, coordinate-wise, $(\check{\mathbf{u}}_2)_i \cdot ((\hat{\mathbf{s}}_{2,i^*})_j - (\hat{\mathbf{u}}_2)_j) = 0$ for every $i, j \in [\kappa]$. Hence, we conclude that either $\check{\mathbf{u}}_2 = \mathbf{0}$, or $\hat{\mathbf{u}}_2 = \hat{\mathbf{s}}_{2,i^*}$. A similar analysis can be carried out for the second equation, which allows us to conclude that $\check{\mathbf{u}}_2 = \check{\rho} \cdot \check{\mathbf{s}}_{2,i^*}$ and that $\hat{\mathbf{u}}_2 = \hat{\rho} \cdot \hat{\mathbf{s}}_{2,i^*}$ for some $\check{\rho}, \hat{\rho} \in \{0, 1\}$.

To conclude the proof, note that the extractor runs $[\hat{\mathbf{d}}]_1 \leftarrow \text{Proj}(\text{td}, \text{com})$, which by definition of Proj and by the previous argument, is

$$\begin{aligned}[\hat{\mathbf{d}}]_2 &= \hat{\mathbf{B}}_2 \cdot [\hat{\mathbf{c}}]_2 \\ &= \hat{\mathbf{B}}_2 \cdot [\hat{\mathbf{B}}_1^* \cdot \hat{\mathbf{u}}_1 + \hat{\mathbf{B}}_2^* \cdot \hat{\mathbf{u}}_2]_2 \\ &= [\hat{\mathbf{u}}_2]_2 = \hat{\rho} \cdot [\hat{\mathbf{s}}_{2,i^*}]_2\end{aligned}$$

Hence, by construction of the extractor, the extracted value $x_{i^*} = \hat{\rho} \in \{0, 1\}$. $\square$

Lemma 12. *se-PC satisfies aggregatability (Definition 6) and projective aggregatability (Definition 11).*

Proof. Both properties follow from the algebraic structure of the commitments, which are linearly homomorphic. The proof is straightforward and is omitted. $\square$

Lemma 13. *se-PC satisfies efficient verification (Definition 13) where EffProjVer runs in time $\mathcal{O}(\lambda \cdot \kappa^2)$.*

Proof. Follows directly from the descriptions of the algorithms $\text{PreProjVer}(\text{ck}, J)$ and $\text{EffProjVer}(\text{ck}_J, \text{com}, \pi)$. $\square$

Remark 8. Given $\mathcal{I} = \{\{i\} : i \in [\ell]\}$, it is straightforward to extend se-PC to build a PCFC for $\mathcal{I}$ -separable quadratic functions that achieves functional extractability (Definition 17). This is necessary to build a BARG via Theorem 4. The extension consists of adding CFC prove and verify algorithms that run as described in [WW22, Remark 4.16], and we refer to their work for the details. Then, the proof of functional extractability follows a similar pattern as the proof of Lemma 10.

6.2 Algebraic PCFC from Bilinear Groups

We construct a (somewhere extractable) projective chainable functional commitment PCFC (Definition 14) by augmenting the PC from the previous section with CFC algorithms. Our starting point, which we argue is the most natural choice given that it is the only fully-succinct algebraic FC in the literature, is the WW24 CFC on its chainable form [WW24, Remark 5.18]. Unfortunately, this scheme does not satisfy projective chain binding (Definition 18), which we require for our circuit-succinct compiler to a BARG.

The WW24 scheme has multiple internal algorithms for committing and proving relations on commitments. Our PCFC will also build on (two of) their algorithms, although we will introduce modifications towards achieving projective chain binding. We start with a description of the internals of WW24, where we already introduce some generalizations. We remark that these algorithms only serve the purpose of modularizing the scheme and the proofs. When the PCFC is used in a black-box way as in our compiler, they are hidden behind the PCFC interface.

Base commitment scheme. There are two families of setup algorithms in WW24: those that generate “base” commitment keys, and those that extend the commitment key with additional terms to enable a proof system. The so-called base setup algorithm $\text{ck}_{\text{base}} \leftarrow \text{SetupBase}(1^\lambda, 1^\ell, 1_{\mathcal{C}}^\ell)$ is parameterized by the maximum input length ℓ and the maximum circuit size $\ell_{\mathcal{C}}$. Such a setup supports arbitrary arithmetic circuits $\mathcal{C} : \mathbb{F}^\ell \rightarrow \mathbb{F}^\ell$ that have at most $\ell_{\mathcal{C}}$ arithmetic gates.¹⁶ The algorithm generates two independent projective commitment keys. The type-I key $[\hat{\mathbf{T}}]_2 \in \mathbb{G}_2^{2\kappa \times \ell_{\mathcal{C}}}$ intentionally coincides with the primary commitment key of PC, and its terms are always represented with a hat accent. The type-II key is defined by two random matrices $\mathbf{T}_A, \mathbf{T}_B \leftarrow_{\$} \mathbb{F}^{2\kappa \times \ell_{\mathcal{C}}}$. To commit to an input $\mathbf{x} \in \mathbb{F}^\ell$, and an output $\mathbf{y} \in \mathbb{F}^\ell$ via PCFC.Com , one uses the type-I key on the set $[\ell]$ or $[\ell]_{\mathcal{C}}$, respectively. The type-II key is used internally to enable interactions between the functional proof algorithms.

We remark that, similarly to our PC construction, both keys are projective. One can sample them in trapdoor mode by specifying two (independent) projection sets $I_1, I_2 \subseteq [\ell_{\mathcal{C}}]$ for the type-I and type-II keys, respectively.¹⁷ We denote this operation by $(\text{ck}_{\text{base}}, \text{td}) \leftarrow \text{ProjSetupBase}(1^\lambda, 1^\ell, 1_{\mathcal{C}}^\ell, (I_1, I_2))$.

Functional proof systems. For functional openings, there are three proof systems that build on top of the type-I and type-II keys: prefix proofs (Pre), linear map proofs (Lin), and quadratic map proofs (Quad). We describe the last two in the generalized form we use in our construction, where the separability sets can be arbitrary and not necessarily prefixes. The prefix proof system is not needed in our construction, so we omit its details.

Lin: is a tuple of algorithms $(\text{SetupLin}, \text{OpenLin}, \text{VerLin})$ that extend the base FC algorithms.

$\text{SetupLin}(\text{ck}_{\text{base}}, \mathcal{S})$ extends the base commitment key, requiring a separability set $\mathcal{S} \subseteq 2^{[\ell_{\mathcal{C}}] \times [\ell_{\mathcal{C}}]}$ and outputting an extended ck. Let σ_1 be a type-I commitment to a vector $\mathbf{x} \in \mathbb{F}_{\mathcal{C}}^\ell$ and let σ_2 be a type-II commitment to $\mathbf{y} \in \mathbb{F}_{\mathcal{C}}^\ell$. Let also $f : \mathbb{F}_{\mathcal{C}}^\ell \rightarrow \mathbb{F}_{\mathcal{C}}^\ell$ be a (I_1, I_2) -separable linear function such that $f(\mathbf{x}) = \mathbf{y}$. Then, $\text{OpenLin}(\text{ck}, \mathbf{x}, f)$ outputs a linear map proof π . The proof can be verified using $\text{VerLin}(\text{ck}, \sigma_1, \sigma_2, f, \pi)$. The proof system also admits algorithms $(\text{PreVerLin}, \text{EffVerLin})$ for efficient verification with pre-processing, which run in time $\mathcal{O}(\lambda \cdot \kappa^2)$.

¹⁶ We remind that, without loss of generality and to simplify the notation, we assume that the input space and the output space are the same.

¹⁷ In the original WW24 scheme, I_1, I_2 are always prefix sets $[j_1], [j_2]$ for $j_1, j_2 \leq \ell_{\mathcal{C}}$.

Its associated security notion is called *linear chain binding* (Definition 19) and can be seen as a restriction of our projective chain binding (Definition 18) for linear functions.¹⁸ Given a trapdoored setup at $(I_1, I_2) \in \mathcal{S}$, it is hard for any PPT adversary to output two input commitments σ_1, σ'_1 , output commitments σ_2, σ'_2 , a (I_1, I_2) -local function f and proofs π, π' such that (1) both proofs verify, (2) $\text{Proj}(\text{td}, \sigma_1) = \text{Proj}(\text{td}, \sigma'_1)$, and (3) $\text{Proj}(\text{td}, \sigma_2) \neq \text{Proj}(\text{td}, \sigma'_2)$.

Quad: is a tuple of algorithms ($\text{SetupQuad}, \text{OpenQuad}, \text{VerQuad}$). Its properties are identical to those of Lin , except that it admits (I_1, I_2) -separable quadratic functions $f : \mathbb{F}_C^\ell \rightarrow \mathbb{F}_C^\ell$, and except that the roles of σ_1 and σ_2 are reversed. Its security notion, *quadratic chain binding* (Definition 20, which generalizes [WW24, Definition 4.35]), is analogous to linear chain binding except that it considers quadratic functions.

In Appendix C, we introduce an explanation of how the different proof systems compose to achieve evaluation binding. We next introduce the formal definitions of our generalizations of linear chain binding and quadratic chain binding, as well as the corresponding definitions of projective matrices that we use in our construction.

Generalization of chain binding properties. For our generalizations of linear chain binding and quadratic chain binding, we refer to dual-type keys generated by ProjSetupBase as described before, where σ_1 is a type-I commitment and σ_2 is a type-II commitment. We also allow the adversary to have the trapdoor of the scheme, which is needed in our security reductions. Note that the definitions of linear and quadratic chain binding differ only on the ordering of the commitments.

Definition 19 (Generalized Linear Chain Binding for Lin). Let $\mathcal{S}_{\text{Lin}} \subseteq 2^{[\ell c] \times [\ell c]}$ and let $\mathcal{F}_{\text{Lin}}^\mathcal{S}$ be the family of linear functions $f : \mathbb{F}_C^\ell \rightarrow \mathbb{F}_C^\ell$ such that f is $\mathcal{S}_{\text{Lin}}$ -separable. A PC augmented with Lin satisfies generalized linear chain binding if for any PPT adversary $\mathcal{A}$, for any $\ell \in \mathbb{N}$, and for any $(I_1, I_2) \in \mathcal{S}_{\text{Lin}}$,

$$\Pr \left[\begin{array}{l} \text{Proj}(\text{td}, \sigma_2) \\ \neq \text{Proj}(\text{td}, \sigma'_2) \\ \wedge f \in \mathcal{F}_{\text{Lin}}^\mathcal{S} \end{array} \middle| \begin{array}{l} (\text{ck}_{\text{base}}, \text{td}) \leftarrow \text{ProjSetupBase}(1^\lambda, 1_C^\ell, (I_1, I_2)) \\ \text{ck} \leftarrow \text{SetupLin}(\text{ck}_{\text{base}}, \mathcal{S}_{\text{Lin}}) \\ (f, \sigma_1, \sigma'_1, \sigma_2, \sigma'_2, \pi, \pi') \leftarrow \mathcal{A}(\text{ck}, \text{td}) \\ \text{VerLin}(\text{ck}, \sigma_1, \sigma_2, f, \pi) = 1 \\ \text{VerLin}(\text{ck}, \sigma'_1, \sigma'_2, f, \pi') = 1 \\ \text{Proj}(\text{td}, \sigma_1) = \text{Proj}(\text{td}, \sigma'_1) \end{array} \right] \leq \text{negl}(\lambda)$$

Definition 20 (Generalized Quadratic Chain Binding for Quad). Let $\mathcal{S}_{\text{Quad}} \subseteq 2^{[\ell c] \times [\ell c]}$ and let $\mathcal{F}_{\text{Quad}}^\mathcal{S}$ be the family of quadratic functions $f : \mathbb{F}_C^\ell \rightarrow \mathbb{F}_C^\ell$ such that f is $\mathcal{S}_{\text{Quad}}$ -separable. A PC augmented with Lin satisfies generalized quadratic chain binding if for any PPT adversary $\mathcal{A}$, for any $\ell \in \mathbb{N}$, and for any $(I_1, I_2) \in \mathcal{S}_{\text{Quad}}$,

$$\Pr \left[\begin{array}{l} \text{Proj}(\text{td}, \sigma_1) \\ \neq \text{Proj}(\text{td}, \sigma'_1) \\ \wedge f \in \mathcal{F}_{\text{Quad}}^\mathcal{S} \end{array} \middle| \begin{array}{l} (\text{ck}_{\text{base}}, \text{td}) \leftarrow \text{ProjSetup}(1^\lambda, 1_C^\ell, I_1, I_2) \\ \text{ck} \leftarrow \text{SetupQuad}(\text{ck}_{\text{base}}, \mathcal{S}_{\text{Quad}}) \\ (f, \sigma_1, \sigma'_1, \sigma_2, \sigma'_2, \pi, \pi') \leftarrow \mathcal{A}(\text{ck}, \text{td}) \\ \text{VerQuad}(\text{ck}, \sigma_2, \sigma_1, f, \pi) = 1 \\ \text{VerQuad}(\text{ck}, \sigma'_2, \sigma'_1, f, \pi') = 1 \\ \text{Proj}(\text{td}, \sigma_2) = \text{Proj}(\text{td}, \sigma'_2) \end{array} \right] \leq \text{negl}(\lambda)$$

¹⁸ This is a generalization of the original chain binding notion [WW24, Definition 4.20] where I_1, I_2 are required to be prefixes.

Projection matrices. We describe the projection matrices $\mathbf{P}^{(\mathcal{S}_{\text{Lin}})}, \mathbf{P}^{(\mathcal{S}_{\text{Quad}})}$ for separable linear and quadratic functions below, which we adapt from [WW24, Definitions 4.21, 4.36]. Recall that P_I are diagonal projection matrices such that $P_{I,i} = 1$ if $i \in I$ and $P_{I,i} = 0$ otherwise.

Definition 21 (Projection matrix for a separable linear function). Let $\ell_C \in \mathbb{N}$ be an input length. For sets $I_1, I_2 \subseteq [\ell_C]$, we define the projection matrix $\mathbf{P}_{\text{Lin}}^{(I_1, I_2)}$ to be

$$\mathbf{P}_{\text{Lin}}^{(I_1, I_2)} := \mathbf{I}_{\ell_C^2} - (\mathbf{I}_{\ell_C} - \mathbf{P}_{I_1}) \otimes \mathbf{P}_{I_2} \in \{0, 1\}^{\ell_C^2 \times \ell_C^2}.$$

For a separability set $\mathcal{S} \subseteq 2^{[\ell_C] \times [\ell_C]}$, we define the linear projection matrix for $\mathcal{S}$ to be

$$\mathbf{P}^{(\mathcal{S})} := \prod_{(I_1, I_2) \in \mathcal{S}} \mathbf{P}_{\text{Lin}}^{(I_1, I_2)} \in \{0, 1\}^{\ell_C^2 \times \ell_C^2}.$$

Definition 22 (Projection matrix for a separable quadratic function). Let $\ell_C \in \mathbb{N}$ be an input length. For sets $I_1, I_2 \subseteq [\ell_C]$, we define the projection matrix $\mathbf{P}_{\text{Quad}}^{(I_1, I_2)}$ to be

$$\mathbf{P}_{\text{Quad}}^{(I_1, I_2)} := \mathbf{I}_{\ell_C^3} - (\mathbf{I}_{\ell_C^2} - (\mathbf{P}_{I_1} \otimes \mathbf{P}_{I_1})) \otimes \mathbf{P}_{I_2} \in \{0, 1\}^{\ell_C^3 \times \ell_C^3}.$$

For a separability set $\mathcal{S} \subseteq 2^{[\ell_C] \times [\ell_C]}$, we define the quadratic projection matrix for $\mathcal{S}$ to be

$$\mathbf{P}^{(\mathcal{S})} := \prod_{(I_1, I_2) \in \mathcal{S}} \mathbf{P}_{\text{Quad}}^{(I_1, I_2)} \in \{0, 1\}^{\ell_C^3 \times \ell_C^3}.$$

Our PCFC construction. Our PCFC is designed for the function family $\mathcal{F}_{k, \ell_C}$ defined in Theorem 2. We additionally define ProjSetup that takes two arguments (I_0, I_1, I_2) – a necessity for granularity in the projective chain binding proof. In our construction, the setup of Lin, Quad is modified with respect to WW24 to express functions in $\mathcal{F}_{k, \ell_C}$ accurately, such that there are no cross-wires between any two parallel “circuit blocks”.

The construction essentially boils down to three commitment keys and four functional proofs (recall Figure 1 from the technical overview). The primary commitment key ck_0 is the same as in PC described in Section 6.1. Then, we rely on auxiliary commitment keys ck_1, ck_2 to realise the following functional proofs:

- A linear proof π_{in} for an input-projection map s that maps (the first ℓ coordinates of) the input commitment com under ck_0 to a commitment σ_2 under ck_2 to the circuit trace.
- A linear proof π_{Lin} between σ_1 (under ck_1) and σ_2 (under ck_2) for the identity function id .
- A quadratic proof π_{Quad} between σ_2 and σ_1 for a next-wire function w .
- A quadratic proof π_{out} for an output-projection map p that maps (the last ℓ coordinates of) σ_2 into (the first ℓ coordinates of) com_y .

Setup($1^\lambda, 1^{k\ell}$):

- Generate $\text{ck}_0 \leftarrow \text{PC.Setup}(1^\lambda, 1^{k\ell_C})$.
- Sample matrices $\mathbf{T}_1, \mathbf{T}_A, \mathbf{T}_B \xleftarrow{\$} \mathbb{F}^{2\kappa \times \ell_C}$ at random. Let $\mathbf{T}_* \leftarrow \mathbf{T}_A \otimes \mathbf{T}_B \in \mathbb{F}^{4\kappa^2 \times \ell_C^2}$ and let $\text{ck}_{\text{base}} \leftarrow ([\mathbf{T}_1]_2, [\mathbf{T}_A]_1, [\mathbf{T}_A]_2, [\mathbf{T}_B]_2, [\mathbf{T}_*]_2)$
For convenience, we name $\text{ck}' \leftarrow ([\hat{\mathbf{T}}]_2, [\mathbf{T}_A]_1, [\mathbf{T}_A]_2, [\mathbf{T}_B]_2, [\mathbf{T}_*]_2) \subseteq \text{ck}$. Note that the first term is (the type-I key) taken from ck_0 and the remaining terms correspond to the type-II key from ck_{base} .

- Define the following sets and families of sets:

$$\begin{aligned}
I_{i,j} &= \{(i-1)\ell_C + 1, \dots, (i-1)\ell_C + j\}, \\
\mathcal{S}_{\text{in}} &= \{(I_{i,j}, I_{i,j}) : i \in [k], j \in [\ell]\}, \\
\mathcal{S}_{\text{Lin}} &= \{(I_{i,j}, I_{i,j}) : i \in [k], j \in [\ell_C]\}, \\
\mathcal{S}_{\text{Quad}} &= \{(I_{i,j}, I_{i,j+1}) : i \in [k], j \in [\ell_C - 1]\}, \\
\mathcal{S}_{\text{out}} &= \{(I_{i,j+\ell_C-\ell}, I_{i,j}) : i \in [k], j \in [\ell]\}.
\end{aligned}$$

- Run $\text{SetupLin}(\text{ck}', \mathcal{S}_{\text{in}})$ as follows. Let $\mathbf{P}_{\text{Lin}} = \mathbf{P}^{(\mathcal{S}_{\text{Lin}})}$ defined as in Definition 21. Then, run SetupLin as defined in [WW24, Construction 4.23] with the above $\mathbf{P}_{\text{Lin}}$. Obtain ck_{in} .
 - Run $\text{SetupLin}(\text{ck}_{\text{base}}, \mathcal{S}_{\text{Lin}})$ as before but over all terms on ck_{base} . Obtain ck_{Lin} .
 - Run $\text{SetupQuad}(\text{ck}_{\text{base}}, \mathcal{S}_{\text{Quad}})$ as follows. Let $\mathbf{P}_{\text{Quad}} = \mathbf{P}^{(\mathcal{S}_{\text{Quad}})}$ defined as in Definition 22. Then, run SetupQuad as defined in [WW24, Construction 4.38] with the above $\mathbf{P}_{\text{Quad}}$. Obtain ck_{Quad} .
 - Finally, run $\text{SetupQuad}(\text{ck}', \mathcal{S}_{\text{out}})$ (again over ck' , the of terms from ck_0 and ck_{base}), Obtain ck_{out} .
- Output

$$\text{ck} := \text{ck}_0 \cup \text{ck}_{\text{base}} \cup \text{ck}_{\text{in}} \cup \text{ck}_{\text{Lin}} \cup \text{ck}_{\text{Quad}} \cup \text{ck}_{\text{out}}.$$

ProjSetup($1^\lambda, 1^{k\ell}, I$): As $I \in \mathcal{I}'$ as defined in Theorem 2, we have that $I = \{i(\ell-1) + 1, \dots, i\ell\}$ for some $i \in [k]$.

Run $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell}, (I_{i,\ell}, I_{i,\ell}, I_{i,\ell}))$ as defined below. Then, output $(\text{ck}, \text{td} = \text{td}_0)$.

ProjSetup($1^\lambda, 1^{k\ell}, (I_0, I_1, I_2)$):

- Run $(\text{ck}_0, \text{td}_0) \leftarrow \text{PC.ProjSetup}(1^\lambda, 1^{k\ell_C}, I_0)$.
- Sample random $\mathbf{B}_1, \mathbf{B}_A, \mathbf{B}_B \leftarrow_{\$} \mathbb{F}^{2\kappa \times 2\kappa}$ and let $\mathbf{B}_A^* = \mathbf{B}_A^{-1}$, $\mathbf{B}_B^* = \mathbf{B}_B^{-1}$ and $\mathbf{B}_1^* = \mathbf{B}_1^{-1}$.
- Split $\mathbf{B}_A = \begin{bmatrix} \mathbf{B}_{A,1} \\ \mathbf{B}_{A,2} \end{bmatrix}$ where $\mathbf{B}_{A,1}, \mathbf{B}_{A,2} \in \mathbb{F}^{\kappa \times 2\kappa}$. Similarly, split $\mathbf{B}_A^* = \begin{bmatrix} \mathbf{B}_{A,1}^* & \mathbf{B}_{A,2}^* \end{bmatrix}$ where $\mathbf{B}_{A,1}^*, \mathbf{B}_{A,2}^* \in \mathbb{F}^{2\kappa \times \kappa}$. Note that, by construction, $\mathbf{B}_{A,1} \cdot \mathbf{B}_{A,1}^* = \mathbf{B}_{A,2} \cdot \mathbf{B}_{A,2}^* = \mathbf{I}_\kappa$ and that $\mathbf{B}_{A,1} \cdot \mathbf{B}_{A,2}^* = \mathbf{B}_{A,2} \cdot \mathbf{B}_{A,1}^* = \mathbf{0}_\kappa$. Define $\mathbf{B}_{1,1}, \mathbf{B}_{1,2}, \mathbf{B}_{B,1}, \mathbf{B}_{B,2} \in \mathbb{F}^{\kappa \times 2\kappa}$ and $\mathbf{B}_{1,1}^*, \mathbf{B}_{1,2}^*, \mathbf{B}_{B,1}^*, \mathbf{B}_{B,2}^* \in \mathbb{F}^{2\kappa \times \kappa}$ analogously.
- Sample $\mathbf{S}_{1,1}, \mathbf{S}_{A,1}, \mathbf{S}_{B,1}, \mathbf{S}_{1,2}, \mathbf{S}_{A,2}, \mathbf{S}_{B,2} \leftarrow_{\$} \mathbb{F}^{\kappa \times k\ell_C}$. Let $\mathbf{P}_{I_1}, \mathbf{P}_{I_2}$ be the projection matrices for the sets I_1, I_2 , respectively.

$$\begin{aligned}
\mathbf{T}_1 &\leftarrow \mathbf{B}_{1,1}^* \cdot \mathbf{S}_{1,1} + \mathbf{B}_{1,2}^* \cdot \mathbf{S}_{1,2} \cdot \mathbf{P}_{I_1}, \\
\mathbf{T}_A &\leftarrow \mathbf{B}_{A,1}^* \cdot \mathbf{S}_{A,1} + \mathbf{B}_{A,2}^* \cdot \mathbf{S}_{A,2} \cdot \mathbf{P}_{I_2}, \\
\mathbf{T}_B &\leftarrow \mathbf{B}_{B,1}^* \cdot \mathbf{S}_{B,1} + \mathbf{B}_{B,2}^* \cdot \mathbf{S}_{B,2} \cdot \mathbf{P}_{I_2}.
\end{aligned}$$

- Finally, let $\mathbf{T}_* \leftarrow \mathbf{T}_A \otimes \mathbf{T}_B \in \mathbb{F}^{4\kappa^2 \times \ell_C^2}$.

The remaining terms are computed from ck_0 and the above terms exactly as in Setup . We also define, as construction-specific trapdoors, $\text{td}_1 \leftarrow (\mathbf{B}_{1,2}, \mathbf{B}_{B,2})$. These trapdoors are linked to I_1, I_2 respectively and we make them explicit as they play a role in the security proof. However, they not used for extractability or projection when considering the scheme as a whole. Output the commitment key ck and trapdoors $(\text{td}_0, \text{td}_1, \text{td}_2)$.

FuncProve($\text{ck}, \mathbf{x}, f \in \mathcal{F}_{k,\ell_C}$):

- Parse $f = (C_1, \dots, C_k)$ and split $\mathbf{x} = (\mathbf{x}_i)_{i \in [k]}$. Evaluate $\mathbf{y}_i \leftarrow C_i(\mathbf{x}_i)$ and let $\mathbf{z}_i \in \mathbb{F}_C^\ell$ be the trace of the i -th computation, i.e., the values of all wires when $C_i(\mathbf{x}_i)$ is evaluated. Note that $\mathbf{z}_i = \mathbf{x}_i | \mathbf{z}_{i,C} | \mathbf{y}_i$ where $\mathbf{z}_{i,C} \in \mathbb{F}^{\ell_C - 2\ell}$.
- Commit to the computation trace of $\mathbf{z} \leftarrow \mathbf{z}_1 | \dots | \mathbf{z}_k$ in the type-I and type-II keys,

$$\sigma_1 \leftarrow [\mathbf{T}_1]_2 \mathbf{z}, \quad \sigma_2 \leftarrow ([\mathbf{T}_A]_1 \mathbf{z}, [\mathbf{T}_B]_2 \mathbf{z}).$$

- Let $s_i : \mathbb{F}_C^\ell \rightarrow \mathbb{F}_C^\ell$ be given by $s(\mathbf{z}) = (z_1, \dots, z_\ell, \mathbf{0}_{\ell_C - \ell})$ be a function that projects the first ℓ coordinates of $\mathbf{z}$ into the first ℓ coordinates. Define $s : \mathbb{F}^{k\ell_C} \rightarrow \mathbb{F}^{k\ell_C}$ to be the parallel evaluation of s_i , which is a $\mathcal{S}_{\text{in}}$ -separable linear function. Let now $\mathbf{z}_{\text{in}} = \mathbf{x}_1 | \mathbf{0}_{\ell_C - \ell_{x,w}} | \dots | \mathbf{x}_k | \mathbf{0}_{\ell_C - \ell_{x,w}}$ be a concatenation of input vectors padded appropriately. Compute an input projection proof, given by $\pi_{\text{in}} \leftarrow \text{OpenLin}((\text{ck}', \text{ck}_{\text{in}}), \mathbf{z}_{\text{in}}, s)$.
- Compute a linear proof for consistency between σ_1 and σ_2 given by $\pi_{\text{Lin}} \leftarrow \text{OpenLin}((\text{ck}_{\text{base}}, \text{ck}_{\text{Lin}}), \mathbf{z}, id)$ where $id : \mathbb{F}^{k\ell_C} \rightarrow \mathbb{F}^{k\ell_C}$ is the identity function. Note that id is $\mathcal{S}_{\text{Lin}}$ -separable.
- Let $w_i : \mathbb{F}_C^\ell \rightarrow \mathbb{F}_C^\ell$ be the “next wire function” of circuit C_i , such that the j -th output of $w_i(\mathbf{z}_i)$ is the result of evaluating the j -th gate of C_i on the $j - 1$ first inputs. Define $w : \mathbb{F}^{k\ell_C} \rightarrow \mathbb{F}^{k\ell_C}$ as $w(\mathbf{z}_1, \dots, \mathbf{z}_k) = (w_1(\mathbf{z}_1), \dots, w_k(\mathbf{z}_k))$. Note that w is a $\mathcal{S}_{\text{Quad}}$ -separable quadratic function. Compute a quadratic gate proof $\pi_{\text{Quad}} \leftarrow \text{OpenQuad}((\text{ck}_{\text{base}}, \text{ck}_{\text{Quad}}), \mathbf{z}, w)$.
- Let $p_i : \mathbb{F}_C^\ell \rightarrow \mathbb{F}_C^\ell$ be given by $p(\mathbf{z}) = (z_{\ell_C - \ell + 1}, \dots, z_{\ell_C}, \mathbf{0}_{\ell_C - \ell})$ be a function that projects the last ℓ coordinates of $\mathbf{z}$ into the first ℓ coordinates. Define $p : \mathbb{F}^{k\ell_C} \rightarrow \mathbb{F}^{k\ell_C}$ to be the parallel evaluation of p_i . Note that p is a $\mathcal{S}_{\text{out}}$ -separable linear (and therefore also quadratic) function. Compute an output projection proof from the type, given by $\pi_{\text{out}} \leftarrow \text{OpenQuad}((\text{ck}', \text{ck}_{\text{out}}), \mathbf{z}, p)$.

Output $\pi \leftarrow (\sigma_1, \sigma_2, \pi_{\text{in}}, \pi_{\text{Lin}}, \pi_{\text{Quad}}, \pi_{\text{out}})$.

FuncVer(ck, com, com_y, f, π):

Parse $\pi = (\sigma_1, \sigma_2, \pi_{\text{in}}, \pi_{\text{Lin}}, \pi_{\text{Quad}}, \pi_{\text{out}})$ and output 1 if all the following checks pass:

- VerLin((ck', ck_{in}), com, σ_2 , s , π_{in}).
- VerLin((ck_{base}, ck_{Lin}), σ_1 , σ_2 , id , π_{Lin}).
- VerQuad((ck_{base}, ck_{Quad}), σ_2 , σ_1 , w , π_{Quad}).
- VerQuad((ck', ck_{out}), σ_2 , com_y, p , π_{out}).

PreFuncVer(ck, f):

Run the preprocessing algorithms for s, id, w and p , respectively:

- ck_{f,in} $\leftarrow$ PreVerLin((ck', ck_{in}), s).
- ck_{f,Lin} $\leftarrow$ PreVerLin((ck_{base}, ck_{Lin}), id).
- ck_{f,Quad} $\leftarrow$ PreVerQuad((ck_{base}, ck_{Quad}), w).
- ck_{f,out} $\leftarrow$ PreVerQuad((ck', ck_{out}), p).

Output ck_f $\leftarrow$ (ck_{f,in}, ck_{f,Lin}, ck_{f,Quad}, ck_{f,out})

EffFuncVer(ck_f, com, com_y, π):

Parse $\pi = (\sigma_1, \sigma_2, \pi_{\text{in}}, \pi_{\text{Lin}}, \pi_{\text{Quad}}, \pi_{\text{out}})$ and output 1 if all the following checks pass:

- EffVerLin(ck_{f,in}, com, σ_2 , π_{in}).
- EffVerLin(ck_{f,Lin}, σ_1 , σ_2 , π_{Lin}).
- EffVerQuad(ck_{f,Quad}, σ_2 , σ_1 , π_{Quad}).
- EffVerQuad(ck_{f,out}, σ_2 , com_y, π_{out}).

Lemma 14. Assume that the $\text{MDDH}_{\kappa, k\ell_C, 2\kappa}$ holds over bgp . Then, PCFC satisfies setup indistinguishability.

Proof. First, note that ck_{base} satisfies setup indistinguishability as proven in Lemma 8 and [WW24, Theorem 4.9]. Second, note that the extensions to the commitment key by the algorithms Lin and Quad can be sampled in a black-box way from ck_{base} and do not vary in projective mode. Therefore, ck satisfies setup indistinguishability.

Lemma 15. *PCFC satisfies CFC correctness (Definition 7).*

Proof. The proof follows from [WW24] as we rely on the correctness of Lin and Quad, and our modifications on the projective matrices (Definitions 21 and 22) do not affect any of their correctness properties. We omit the details.

Lemma 16. *PCFC satisfies CFC efficient verification (Definition 9) where EffFuncVer runs in time $\mathcal{O}(\lambda \cdot \kappa^2)$.*

Proof. The property follows directly from the descriptions of $\text{PreFuncVer}(\text{ck}, J)$ and $\text{EffFuncVer}(\text{ck}_J, \text{com}, \pi)$, as well as from the efficient verification properties of the Lin and Quad proof systems. $\square$

Before proving projective chain binding, we state a short lemma about the individual proof systems.

Lemma 17. *Let Lin and Quad, be as defined in [WW24] and modified in our PCFC construction. Then, Lin satisfies generalized linear chain binding (Definition 19) and Quad satisfies generalized quadratic chain binding (Definition 20).*

Proof. The proof follows from [WW24] and from our definitions of projective matrices. The main difference with respect to the original notions is that we allow the adversary to have the trapdoor of the commitment keys. We argue that this does not invalidate the security of the proof systems, as every computational step in the original proofs of linear chain binding ([WW24, Theorem 4.25]) and quadratic chain binding ([WW24, Theorem 4.39]) relies on a MDDH-type assumption whose challenge is on a matrix that is crafted in a black-box way from the original commitment key ck_0 . More in detail, we have that for the proof of linear chain binding:

- The first computational step (Lemma 4.28) uses $\mathbf{A}$ as a Ker-DH challenge, which is sampled uniformly as $\mathbf{A} \leftarrow \mathbb{F}^{\kappa \times (\kappa+1)}$.
- The second computational step (Lemma 4.31) uses $(\hat{\mathbf{S}}_2, \mathbf{Z}_\alpha)$ as a MDDH challenge. $\hat{\mathbf{S}}_2$ is not included in td , and $\mathbf{Z}_\alpha$ is sampled black-box from ck_0 .

For the proof of quadratic chain binding:

- The first computational step (Lemma 4.43) uses $\mathbf{A}$ as a Ker-DH challenge, exactly as before.
- The second computational step (Lemma 4.46) uses $(\mathbf{S}_{B,2}, \mathbf{Z})$ as a MDDH challenge. As before, $\mathbf{S}_{2,2}$ is not included in td , and $\mathbf{Z}$ is sampled black-box from ck_0 .

Theorem 3. *Assume that the $\text{KerDH}_{\kappa, \kappa \ell_C}$ and the $\text{MDDH}_{\kappa, \kappa \ell_C, 2\kappa}$ assumptions hold over bgrp . Then, our construction PCFC satisfies projective chain binding for the class $f \in \mathcal{F}_{k, \ell_C}$ (c.f. Theorem 2).*

Proof. We start by defining the projection sets that we will use in the proof

$$I_j = \{(i^* - 1)\ell_C + 1, \dots, (i^* - 1)\ell_C + j\}.$$

Note that $|I_j| = j$. We also recall that $\pi = (\sigma_1, \sigma_2, \pi_{\text{in}}, \pi_{\text{Lin}}, \pi_{\text{Quad}}, \pi_{\text{out}})$. We define a sequence of hybrid games in Figure 4, Figure 5. The initial hybrid game, $\text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda)$, is the projective chain binding game from Definition 18. Throughout the hybrid games, the adversary obtains the primary trapdoor td_0 , but not the internal trapdoors td_1, td_2 , which are only available to the challenger. This is crucial in the proof as if the adversary had td_1, td_2 , setup indistinguishability for the corresponding subsets I_1, I_2 would not hold, and several game transitions rely on this property. On the other hand, the projection set I_0 is fixed throughout the entire sequence of games. We progress through the hybrids as follows.

$\text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda)$:	$\text{Hyb}_{\mathcal{A}}^{\text{in}}(\lambda)$:
$(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k_{\ell}c}, (I_\ell, I_\ell, I_\ell))$	$\text{// as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$
$(f, \text{com}, \text{com}', \text{com}_y, \text{com}'_y, \pi, \pi') \leftarrow \mathcal{A}(\text{ck}, \text{td}_0)$	assert $\text{Proj}(\text{td}_2, \sigma_2) = \text{Proj}(\text{td}_2, \sigma'_2)$
$(\sigma_1, \sigma_2, \pi_{\text{in}}, \pi_{\text{Lin}}, \pi_{\text{Quad}}, \pi_{\text{out}}) \leftarrow \pi$	return 1
$(\sigma'_1, \sigma'_2, \pi'_{\text{in}}, \pi'_{\text{Lin}}, \pi'_{\text{Quad}}, \pi'_{\text{out}}) \leftarrow \pi'$	
assert $\text{FuncVer}(\text{ck}, \text{com}, \text{com}_y, f, \pi) = 1$	
assert $\text{FuncVer}(\text{ck}, \text{com}', \text{com}'_y, f, \pi') = 1$	
assert $\text{Proj}(\text{td}_0, \text{com}) = \text{Proj}(\text{td}_0, \text{com}')$	
assert $\text{Proj}(\text{td}_0, \text{com}_y) \neq \text{Proj}(\text{td}_0, \text{com}'_y)$	
$\text{// Checkpoint line}$	
return 1	

Fig. 4: First set of games for the proof of Theorem 3. We highlight changes between games.

$\text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{\text{in}}(\lambda)$: This step follows by the linear binding property of Lin (Lemma 17) for the relation in, i.e., the identity map between the base key (ck_0) and the type-II key commitments to the internal circuit wires. To see this, note that ck_{in} is sampled on $\mathcal{S}_{\text{in}}$ and $(I_\ell, I_\ell) \in \mathcal{S}_{\text{in}}$. Next, we provide the details of the reduction for this game hop — the remaining steps follow an identical pattern and we will only give a proof sketch.

Let $\mathcal{A}$ be a PPT adversary that wins $\text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda)$ but does not win $\text{Hyb}_{\mathcal{A}}^{\text{in}}(\lambda)$. Then, we can construct a PPT adversary $\mathcal{B}_{\text{in}}$ that wins $\text{Hyb}_{\mathcal{A}}^{\text{in}}(\lambda)$ as follows:

- $\mathcal{B}_{\text{Lin}}$ receives the key $(\text{ck}', \text{ck}_{\text{Lin}})$ and the trapdoor td_0 .
- $\mathcal{B}_{\text{Lin}}$ simulates the remaining part of the commitment key given $\text{ck}', \text{ck}_{\text{Lin}}$ and generates a complete ck . Then, it calls $\mathcal{A}(\text{ck}, \text{td}_0)$ and receives the proofs π, π' . It parses the commitments $\text{com}, \text{com}', \sigma_2, \sigma'_2$ and the Lin proofs $\pi_{\text{in}}, \pi'_{\text{in}}$ from their respective proofs.
- $\mathcal{B}_{\text{Lin}}$ simply outputs $(s, \text{com}, \text{com}', \sigma_2, \sigma'_2, \pi_{\text{in}}, \pi'_{\text{in}})$ to its challenger, where s is the identity function for the first ℓ coordinates (clearly $s \in \mathcal{F}_{\text{in}}^{\mathcal{S}}$).

It is evident that $\mathcal{B}_{\text{Lin}}$ runs in polynomial time. If $\mathcal{A}$ wins in $\text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda)$ but not in $\text{Hyb}_{\mathcal{A}}^{\text{in}}(\lambda)$, the commitments com, com' and proofs $\pi_{\text{in}}, \pi'_{\text{in}}$ output by $\mathcal{A}$ must satisfy:

- $\text{Proj}(\text{td}_0, \text{com}) = \text{Proj}(\text{td}_0, \text{com}')$,
- $\text{Proj}(\text{td}_2, \sigma_2) \neq \text{Proj}(\text{td}_2, \sigma'_2)$,
- $\text{VerLin}((\text{ck}', \text{ck}_{\text{in}}), \text{com}, \sigma_2, s, \pi_{\text{in}})$,
- $\text{VerLin}((\text{ck}', \text{ck}_{\text{in}}), \text{com}', \sigma'_2, s, \pi'_{\text{in}})$.

$\text{Hyb}_{\mathcal{A}}^{j,0}(\lambda), \ell \leq j \leq \ell_{\mathcal{C}}:$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_j, I_j))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_2, \sigma_2) = \text{Proj}(\text{td}_2, \sigma'_2)$ $\text{return } 1$	$\text{Hyb}_{\mathcal{A}}^{j,1}(\lambda), \ell \leq j \leq \ell_{\mathcal{C}} - 1:$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_{j+1}, I_j))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_2, \sigma_2) = \text{Proj}(\text{td}_2, \sigma'_2)$ $\text{return } 1$
$\text{Hyb}_{\mathcal{A}}^{j,2}(\lambda), \ell \leq j \leq \ell_{\mathcal{C}} - 1:$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_{j+1}, I_j))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_2, \sigma_2) = \text{Proj}(\text{td}_2, \sigma'_2)$ $\text{assert Proj}(\text{td}_1, \sigma_1) = \text{Proj}(\text{td}_1, \sigma'_1)$ $\text{return } 1$	$\text{Hyb}_{\mathcal{A}}^{j,3}(\lambda), \ell \leq j \leq \ell_{\mathcal{C}} - 1:$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_{j+1}, I_j))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_1, \sigma_1) = \text{Proj}(\text{td}_1, \sigma'_1)$ $\text{return } 1$
$\text{Hyb}_{\mathcal{A}}^{j,4}(\lambda), \ell \leq j \leq \ell_{\mathcal{C}} - 1:$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_{j+1}, I_{j+1}))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_1, \sigma_1) = \text{Proj}(\text{td}_1, \sigma'_1)$ $\text{return } 1$	$\text{Hyb}_{\mathcal{A}}^{j,5}(\lambda), \ell \leq j \leq \ell_{\mathcal{C}} - 1:$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_{j+1}, I_{j+1}))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_2, \sigma_2) = \text{Proj}(\text{td}_2, \sigma'_2)$ $\text{assert Proj}(\text{td}_1, \sigma_1) = \text{Proj}(\text{td}_1, \sigma'_1)$ $\text{return } 1$
$\text{Hyb}_{\mathcal{A}}^{\text{out}0}(\lambda):$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_\ell, I_{\ell_{\mathcal{C}}}))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_2, \sigma_2) = \text{Proj}(\text{td}_2, \sigma'_2)$ $\text{return } 1$	$\text{Hyb}_{\mathcal{A}}^{\text{out}1}(\lambda):$ $(\text{ck}, \text{td}_0, \text{td}_1, \text{td}_2) \leftarrow \text{ProjSetup}(1^\lambda, 1^{k\ell_{\mathcal{C}}}, (I_\ell, I_\ell, I_{\ell_{\mathcal{C}}}))$ $\quad // \text{ then, as } \text{Hyb}_{\mathcal{A}}^{\text{pcb}}(\lambda) \text{ until "checkpoint line"}$ $\text{assert Proj}(\text{td}_2, \sigma_2) = \text{Proj}(\text{td}_2, \sigma'_2)$ $\text{assert Proj}(\text{td}_0, \text{com}_y) = \text{Proj}(\text{td}_0, \text{com}'_y)$ $\text{return } 1$

Fig. 5: Second set of games for the proof of Theorem 3. We highlight changes between games.

Hence, $\mathcal{B}_{\text{Lin}}$ wins the generalized linear binding game (Definition 19) if $\mathcal{A}$ is successful. We conclude:

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^0(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{\text{in}}(\lambda) + \text{Adv}_{\text{Lin},\mathcal{B}_{\text{Lin}}}^{\text{lbind}}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{\text{in}}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{\ell,0}(\lambda)$: This is only a renaming step; the games are identical for the case $j = \ell$ that we consider. Therefore,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{\text{in}}(\lambda) = \text{Adv}_{\text{PCFC},\mathcal{A}}^{\ell,0}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{j,0}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{j,1}(\lambda)$: For any $j \in \{\ell, \dots, \ell_C - 1\}$, this step readily follows by the setup indistinguishability property of PCFC (Lemma 14). We have that, for some PPT adversary $\mathcal{B}_{\text{Sind}}$,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{j,0}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{j,1}(\lambda) + \text{Adv}_{\text{PCFC},\mathcal{B}_{\text{Sind}}}^{\text{sbind}}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{j,1}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{j,2}(\lambda)$: For any $j \in \{\ell, \dots, \ell_C - 1\}$, this step follows by the generalized quadratic chain binding of Quad (Lemma 17), as ck_{Quad} is sampled on $\mathcal{S}_{\text{Quad}}$ and $(I_j, I_{j+1}) \in \mathcal{S}_{\text{Quad}}$. We have that, for some PPT adversary $\mathcal{B}_{\text{Quad}}$,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{j,1}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{j,2}(\lambda) + \text{Adv}_{\text{Quad},\mathcal{B}_{\text{Quad}}}^{\text{qbind}}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{j,2}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{j,3}(\lambda)$: This step is a simplification of $\text{Hyb}_{\mathcal{A}}^{j,2}(\lambda)$ where we drop a verification condition on σ_2 . Therefore,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{j,2}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{j,3}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{j,3}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{j,4}(\lambda)$: For any $j \in \{\ell, \dots, \ell_C - 1\}$, this step again follows by the setup indistinguishability property of PCFC (Lemma 14). We have that, for some PPT adversary $\mathcal{B}_{\text{Sind}}$,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{j,3}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{j,4}(\lambda) + \text{Adv}_{\text{PCFC},\mathcal{B}_{\text{Sind}}}^{\text{sbind}}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{j,4}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{j,5}(\lambda)$: For any $j \in \{\ell, \dots, \ell_C - 1\}$, this step follows by the generalized linear chain binding of Lin (Lemma 17), as ck_{Lin} is sampled on $\mathcal{S}_{\text{Lin}}$ and $(I_j, I_j) \in \mathcal{S}_{\text{Lin}}$. We have that, for some PPT adversary $\mathcal{B}_{\text{Lin}}$,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{j,4}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{j,5}(\lambda) + \text{Adv}_{\text{Lin},\mathcal{B}_{\text{Lin}}}^{\text{lbind}}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{j,5}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{j+1,0}(\lambda)$: This step is a simplification of $\text{Hyb}_{\mathcal{A}}^{j,5}(\lambda)$ where we drop a verification condition on σ_1 . Therefore,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{j,5}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{j+1,0}(\lambda).$$

Next, we bound the final sequence of games, starting from $\text{Hyb}_{\mathcal{A}}^{\ell_C,0}(\lambda)$.

$\text{Hyb}_{\mathcal{A}}^{\ell_C,0}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{\text{out0}}(\lambda)$: In this game, we change the extraction sets from $(I_\ell, I_{\ell_C}, I_{\ell_C})$ to $(I_\ell, I_\ell, I_{\ell_C})$. Again, the step follows by the setup indistinguishability property of PCFC (Lemma 14). We have that, for some PPT adversary $\mathcal{B}_{\text{Sind}}$,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{\ell_C,0}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{\text{out0}}(\lambda) + \text{Adv}_{\text{PCFC},\mathcal{B}_{\text{Sind}}}^{\text{bind}}(\lambda).$$

$\text{Hyb}_{\mathcal{A}}^{\text{out0}}(\lambda) \rightarrow \text{Hyb}_{\mathcal{A}}^{\text{out1}}(\lambda)$: Finally, this step follows by the generalized quadratic chain binding of Quad (Lemma 17) for the output vector, as ck_{out} is sampled on $\mathcal{S}_{\text{out}}$ and $(I_\ell, I_{\ell_C}) \in \mathcal{S}_{\text{out}}$. We have that, for some PPT adversary $\mathcal{B}_{\text{Quad}}$,

$$\text{Adv}_{\text{PCFC},\mathcal{A}}^{\text{out0}}(\lambda) \leq \text{Adv}_{\text{PCFC},\mathcal{A}}^{\text{out1}}(\lambda) + \text{Adv}_{\text{Quad},\mathcal{B}_{\text{Quad}}}^{\text{qbind}}(\lambda).$$

$\text{Hyb}_A^{\text{out}1}(\lambda)$: In this game, we have the following contradicting conditions:

- $\text{Proj}(\text{td}_1, \text{com}_y) \neq \text{Proj}(\text{td}_1, \text{com}'_y)$ (present since $\text{Hyb}_A^{\text{pcb}}(\lambda)$), and
- $\text{Proj}(\text{td}_1, \text{com}_y) = \text{Proj}(\text{td}_1, \text{com}'_y)$.

Hence, the advantage of any adversary in this game is $\text{Adv}_{\text{PCFC}, A}^{\text{out}1}(\lambda) = 0$.

The theorem follows by collecting all the bounds and noting the relevant computational assumptions that are used at each step. $\square$

Acknowledgements

We are thankful to Hoeteck Wee and David J. Wu for useful discussions and insights, and to anonymous reviewers for identifying a flaw in an earlier version of the paper.

This work is supported by the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme (Grant agreement No. 101001283), partially supported by projects PRODIGY (TED2021-132464B-I00), ESPADA (PID2022-142290OB-I00), and CEX2024-001471-M funded by MCIN/AEI/10.13039/501100011033/ and the European Union NextGenerationEU / PRTR, and partially funded by Ministerio de Universidades (FPU21/00600).

References

- ABF24. Gaspard Anthoine, David Balbás, and Dario Fiore. Fully-succinct multi-key homomorphic signatures from standard assumptions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part III*, volume 14922 of *Lecture Notes in Computer Science*, pages 317–351. Springer, Cham, August 2024. doi:10.1007/978-3-031-68382-4_10.
- ACG24. Abtin Afshar, Jiaqi Cheng, and Rishab Goyal. Leveled fully-homomorphic signatures from batch arguments. Cryptology ePrint Archive, Report 2024/931, 2024. URL: <https://eprint.iacr.org/2024/931>.
- ACL⁺22. Martin R. Albrecht, Valerio Cini, Russell W. F. Lai, Giulio Malavolta, and Sri Aravinda Krishnan Thyagarajan. Lattice-based SNARKs: Publicly verifiable, preprocessing, and recursively composable – (extended abstract). In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 102–132. Springer, Cham, August 2022. doi:10.1007/978-3-031-15979-4_4.
- BCFL23. David Balbás, Dario Catalano, Dario Fiore, and Russell W. F. Lai. Chainable functional commitments for unbounded-depth circuits. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023: 21st Theory of Cryptography Conference, Part III*, volume 14371 of *Lecture Notes in Computer Science*, pages 363–393. Springer, Cham, November / December 2023. doi:10.1007/978-3-031-48621-0_13.
- BCJP24. Maya Farber Brodsky, Arka Rai Choudhuri, Abhishek Jain, and Omer Paneth. Monotone-policy aggregate signatures. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part IV*, volume 14654 of *Lecture Notes in Computer Science*, pages 168–195. Springer, Cham, May 2024. doi:10.1007/978-3-031-58737-5_7.
- BHK17. Zvika Brakerski, Justin Holmgren, and Yael Tauman Kalai. Non-interactive delegation and batch NP verification from standard computational assumptions. In Hamed Hatami, Pierre McKenzie, and Valerie King, editors, *49th Annual ACM Symposium on Theory of Computing*, pages 474–482. ACM Press, June 2017. doi:10.1145/3055399.3055497.
- CCH⁺19. Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-Shamir: from practice to theory. In Moses Charikar and Edith Cohen, editors, *51st Annual ACM Symposium on Theory of Computing*, pages 1082–1090. ACM Press, June 2019. doi:10.1145/3313276.3316380.
- CEW25. Binyi Chen, Noel Elias, and David J. Wu. Pairing-based batch arguments for NP with a linear-size CRS. Cryptology ePrint Archive, Paper 2025/1323, 2025. URL: <https://eprint.iacr.org/2025/1323>.

- CGH98. Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *30th Annual ACM Symposium on Theory of Computing*, pages 209–218. ACM Press, May 1998. doi:10.1145/276698.276741.
- CGJ⁺23. Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. Correlation intractability and SNARGs from sub-exponential DDH. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 635–668. Springer, Cham, August 2023. doi:10.1007/978-3-031-38551-3_20.
- CJJ21. Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. Non-interactive batch arguments for NP from standard assumptions. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part IV*, volume 12828 of *Lecture Notes in Computer Science*, pages 394–423, Virtual Event, August 2021. Springer, Cham. doi:10.1007/978-3-030-84259-8_14.
- CJJ22. Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. SNARGs for $\mathcal{P}$ from LWE. In *62nd Annual Symposium on Foundations of Computer Science*, pages 68–79. IEEE Computer Society Press, February 2022. doi:10.1109/FOCS52979.2021.00016.
- dCP23. Leo de Castro and Chris Peikert. Functional commitments for all functions, with transparent setup and from SIS. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 287–320. Springer, Cham, April 2023. doi:10.1007/978-3-031-30620-4_10.
- DGKV22. Lalita Devadas, Rishab Goyal, Yael Kalai, and Vinod Vaikuntanathan. Rate-1 non-interactive arguments for batch-NP and applications. In *63rd Annual Symposium on Foundations of Computer Science*, pages 1057–1068. IEEE Computer Society Press, October / November 2022. doi:10.1109/FOCS54457.2022.00103.
- EHK⁺13. Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part II*, volume 8043 of *Lecture Notes in Computer Science*, pages 129–147. Springer, Berlin, Heidelberg, August 2013. doi:10.1007/978-3-642-40084-1_8.
- GLWW24. Rachit Garg, George Lu, Brent Waters, and David J. Wu. Reducing the CRS size in registered ABE systems. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part III*, volume 14922 of *Lecture Notes in Computer Science*, pages 143–177. Springer, Cham, August 2024. doi:10.1007/978-3-031-68382-4_5.
- Goy24. Rishab Goyal. Mutable batch arguments and applications. Cryptology ePrint Archive, Report 2024/737, 2024. URL: <https://eprint.iacr.org/2024/737>.
- GSWW22. Rachit Garg, Kristin Sheridan, Brent Waters, and David J. Wu. Fully succinct batch arguments for NP from indistinguishability obfuscation. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022: 20th Theory of Cryptography Conference, Part I*, volume 13747 of *Lecture Notes in Computer Science*, pages 526–555. Springer, Cham, November 2022. doi:10.1007/978-3-031-22318-1_19.
- GW11. Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd Annual ACM Symposium on Theory of Computing*, pages 99–108. ACM Press, June 2011. doi:10.1145/1993636.1993651.
- GZ21. Alonso González and Alexandros Zacharakis. Fully-succinct publicly verifiable delegation from constant-size assumptions. In Kobbi Nissim and Brent Waters, editors, *TCC 2021: 19th Theory of Cryptography Conference, Part I*, volume 13042 of *Lecture Notes in Computer Science*, pages 529–557. Springer, Cham, November 2021. doi:10.1007/978-3-030-90459-3_18.
- HW15. Pavel Hubacek and Daniel Wichs. On the communication complexity of secure function evaluation with long output. In Tim Roughgarden, editor, *ITCS 2015: 6th Conference on Innovations in Theoretical Computer Science*, pages 163–172. Association for Computing Machinery, January 2015. doi:10.1145/2688073.2688105.
- JJ21. Abhishek Jain and Zhengzhong Jin. Non-interactive zero knowledge from sub-exponential DDH. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 3–32. Springer, Cham, October 2021. doi:10.1007/978-3-030-77870-5_1.
- JKKZ21. Ruta Jawale, Yael Tauman Kalai, Dakshita Khurana, and Rachel Yun Zhang. SNARGs for bounded depth computations and PPAD hardness from sub-exponential LWE. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd Annual ACM Symposium on Theory of Computing*, pages 708–721. ACM Press, June 2021. doi:10.1145/3406325.3451055.

- KLWV23. Yael Kalai, Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. Boosting batch arguments and RAM delegation. In Barna Saha and Rocco A. Servedio, editors, *55th Annual ACM Symposium on Theory of Computing*, pages 1545–1552. ACM Press, June 2023. [doi:10.1145/3564246.3585200](https://doi.org/10.1145/3564246.3585200).
- KPY19. Yael Tauman Kalai, Omer Paneth, and Lisa Yang. How to delegate computations publicly. In Moses Charikar and Edith Cohen, editors, *51st Annual ACM Symposium on Theory of Computing*, pages 1115–1124. ACM Press, June 2019. [doi:10.1145/3313276.3316411](https://doi.org/10.1145/3313276.3316411).
- KVZ21. Yael Tauman Kalai, Vinod Vaikuntanathan, and Rachel Yun Zhang. Somewhere statistical soundness, post-quantum security, and SNARGs. In Kobbi Nissim and Brent Waters, editors, *TCC 2021: 19th Theory of Cryptography Conference, Part I*, volume 13042 of *Lecture Notes in Computer Science*, pages 330–368. Springer, Cham, November 2021. [doi:10.1007/978-3-030-90459-3_12](https://doi.org/10.1007/978-3-030-90459-3_12).
- LRY16. Benoît Libert, Somindu C. Ramanna, and Moti Yung. Functional commitment schemes: From polynomial commitments to pairing-based accumulators from simple assumptions. In Ioannis Chatzigiannakis, Michael Mitzenmacher, Yuval Rabani, and Davide Sangiorgi, editors, *ICALP 2016: 43rd International Colloquium on Automata, Languages and Programming*, volume 55 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 30:1–30:14. Schloss Dagstuhl, July 2016. [doi:10.4230/LIPIcs.ICALP.2016.30](https://doi.org/10.4230/LIPIcs.ICALP.2016.30).
- Mic94. Silvio Micali. CS proofs (extended abstracts). In *35th Annual Symposium on Foundations of Computer Science*, pages 436–453. IEEE Computer Society Press, November 1994. [doi:10.1109/SFCS.1994.365746](https://doi.org/10.1109/SFCS.1994.365746).
- MRV16. Paz Morillo, Carla Ràfols, and Jorge Luis Villar. The kernel matrix Diffie-Hellman assumption. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 729–758. Springer, Berlin, Heidelberg, December 2016. [doi:10.1007/978-3-662-53887-6_27](https://doi.org/10.1007/978-3-662-53887-6_27).
- PP22. Omer Paneth and Rafael Pass. Incrementally verifiable computation via rate-1 batch arguments. In *63rd Annual Symposium on Foundations of Computer Science*, pages 1045–1056. IEEE Computer Society Press, October / November 2022. [doi:10.1109/FOCS54457.2022.00102](https://doi.org/10.1109/FOCS54457.2022.00102).
- PS19. Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part I*, volume 11692 of *Lecture Notes in Computer Science*, pages 89–114. Springer, Cham, August 2019. [doi:10.1007/978-3-030-26948-7_4](https://doi.org/10.1007/978-3-030-26948-7_4).
- WW22. Brent Waters and David J. Wu. Batch arguments for NP and more from standard bilinear group assumptions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 433–463. Springer, Cham, August 2022. [doi:10.1007/978-3-031-15979-4_15](https://doi.org/10.1007/978-3-031-15979-4_15).
- WW23a. Hoeteck Wee and David J. Wu. Lattice-based functional commitments: Fast verification and cryptanalysis. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part V*, volume 14442 of *Lecture Notes in Computer Science*, pages 201–235. Springer, Singapore, December 2023. [doi:10.1007/978-981-99-8733-7_7](https://doi.org/10.1007/978-981-99-8733-7_7).
- WW23b. Hoeteck Wee and David J. Wu. Succinct vector, polynomial, and functional commitments from lattices. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 385–416. Springer, Cham, April 2023. [doi:10.1007/978-3-031-30620-4_13](https://doi.org/10.1007/978-3-031-30620-4_13).
- WW24. Hoeteck Wee and David J. Wu. Succinct functional commitments for circuits from k-Lin. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 280–310. Springer, Cham, May 2024. [doi:10.1007/978-3-031-58723-8_10](https://doi.org/10.1007/978-3-031-58723-8_10).

A Wire-by-Wire Compiler: From Functional Extractability to BARGs

We provide a generic compiler to build a BARG for NP from somewhere extractable projective chainable functional commitments (se-PCFC), as defined in Section 4. Our compiler can be seen as an abstraction of the construction by Waters and Wu [WW22]. The resulting BARG can evaluate circuits $\mathcal{C} : \bar{\mathcal{M}}^\ell \rightarrow \bar{\mathcal{M}}$ with arbitrary gates, proving that $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i) = 1$ for every $i \in [k]$. Notably, the compiler only makes black-box use of the underlying PCFC. Therefore, if the PCFC is algebraic, so does the resulting BARG.

A.1 Compiler construction

Our compiler is described in Figure 6. Given a quadratic gate function $\chi : \bar{\mathcal{M}}^m \rightarrow \bar{\mathcal{M}}$ for (arithmetic or Boolean) circuits $\mathcal{C} : \bar{\mathcal{M}}^{\ell_{\mathbf{x}, \mathbf{w}}} \rightarrow \bar{\mathcal{M}}$, we define $f_\chi : \bar{\mathcal{M}}^{k \cdot m} \rightarrow \bar{\mathcal{M}}^k$ as $f_\chi(\mathbf{x}_1, \dots, \mathbf{x}_k) = (\chi(\mathbf{x}_1), \dots, \chi(\mathbf{x}_k))$. Namely, f_χ is a k -separable function that runs χ in parallel k times, and such that $m < \ell_{\mathcal{C}}$. The compiler turns a PCFC which 1) supports the functions f_χ , 2) is somewhere extractable and 3) is functional extractable into a BARG for NP. The formal requirements and implication are stated in the following Theorem 4. Note that in our construction the crs is independent of the circuit size $\ell_{\mathcal{C}}$.

Theorem 4 (Basic Compiler). *Let $\mathcal{I} := \{\{i\} : i \in [k]\}$ (i.e. singletons of indices in $[k]$), $\bar{\mathcal{M}} \supseteq \{0, 1\}$ be some message space, PCFC be a projective chainable functional commitment scheme (Definition 14) for the $\mathcal{I}$ -separable functions $\mathcal{F} = \{f_\chi : \bar{\mathcal{M}}^{k \cdot m} \rightarrow \bar{\mathcal{M}}^k : \chi \text{ is a gate function}\}$ which satisfies*

- (Definition 12) somewhere extractability for $(\mathcal{I}, \bar{\mathcal{M}})$ and
- (Definition 17) functional extractability for $(\mathcal{F}, \mathcal{I})$.

Then, the construction BARG in Figure 6 is a somewhere extractable batch argument (Definition 1) for NP whose proof size is $|\pi| = \ell_{\mathcal{C}} \cdot |\text{com}| + \ell_{\mathbf{w}} \cdot |\pi^b| + (\ell_{\mathcal{C}} - \ell_{\mathbf{x}, \mathbf{w}}) \cdot |\pi^\chi|$ where π^b is a PC subspace proof, and π^χ is a PCFC functional proof for $f \in \mathcal{F}$. Moreover, BARG admits efficient verification with preprocessing (Definition 2).

Proof. The completeness, succinctness and setup indistinguishability of BARG follow by inspection, given the correctness, succinctness and setup indistinguishability of PCFC. The somewhere extractability of BARG follows from Lemma 18. Its efficient verification property follows from Lemma 21. $\square$

In the remainder of the section, we prove somewhere extractability in Appendix A.2 and efficient verification in Appendix A.3.

A.2 Proof of somewhere extractability

Lemma 18. *Let PCFC and BARG be as defined in Theorem 4. It holds that BARG has somewhere extractability.*

Proof. We proceed by a series of hybrid games $\text{Hyb}^0, \text{Hyb}^{1,0}, \dots, \text{Hyb}^{1, \ell_{\mathbf{w}}}, \text{Hyb}^{2,0}, \dots, \text{Hyb}^{2, \ell_{\text{out}}}$ which progressively include additional winning conditions. The formal descriptions of the games are given in Figure 7 with explanations given below.

$\text{Setup}(1^\lambda, 1^k, 1^{\ell_C})$ $\text{ck} \leftarrow \text{PCFC.Setup}(1^\lambda, 1^k)$ return $\text{crs} := \text{ck}$ $\text{SetupTd}(1^\lambda, 1^k, 1^{\ell_C}, i^*)$ $(\text{ck}, \text{td}) \leftarrow \text{PCFC.ProjSetup}(1^\lambda, 1^k, i^*)$ return $(\text{crs}, \text{td}) := (\text{ck}, \text{td})$ $\text{Ext}(\text{td}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi)$ $\mathbf{w}_{i^*,g} \leftarrow \text{PCFC.ProjExt}(\text{td}, \text{com}_g) \quad \forall g \in \mathcal{G}_{\text{wit}}$ return $\mathbf{w}_{i^*} := (\mathbf{w}_{i^*,g})_{g \in \mathcal{G}_{\text{wit}}}$ $\text{Prove}(\text{crs}, \mathcal{C}, \{\mathbf{x}_i, \mathbf{w}_i\}_{i \in [k]})$ $(\chi_g)_{g \in \mathcal{G}_{\text{out}}} \leftarrow \mathcal{C}$ $x_{i,g} := \chi_g(x_{i,j} : j \in T_g) \quad \forall i \in [k], g \in \mathcal{G}_{\text{out}}$ $\mathbf{x}_g := (x_{1,g}, \dots, x_{k,g}) \quad \forall g \in \mathcal{G}_{\text{out}}$ $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{wit}} \cup \mathcal{G}_{\text{out}}$ $\pi_g^b \leftarrow \text{PCFC.ProjProve}(\text{ck}, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{wit}}$ $\pi_g^x \leftarrow \text{PCFC.FuncProve}(\text{ck}, (\mathbf{x}_t)_{t \in T_g}, f_{\chi_g}) \quad \forall g \in \mathcal{G}_{\text{out}}$ return $\pi := ((\text{com}_g)_{g \in \mathcal{G}_{\text{wit}} \cup \mathcal{G}_{\text{out}}}, (\pi_g^b)_{g \in \mathcal{G}_{\text{wit}}}, (\pi_g^x)_{g \in \mathcal{G}_{\text{out}}})$	$\text{Ver}(\text{crs}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi)$ $(\chi_g)_{g \in \mathcal{G}_{\text{out}}} \leftarrow \mathcal{C}$ $\mathbf{x}_g := (x_{1,g}, \dots, x_{k,g}) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ assert $\text{PCFC.ProjVer}(\text{ck}, \text{com}_g, \pi_g^b) = 1 \quad \forall g \in \mathcal{G}_{\text{wit}}$ assert $\text{PCFC.FuncVer}(\text{ck}, (\text{com}_i)_{i \in T_g}, \text{com}_g, f_{\chi_g}, \pi_g^x) = 1 \quad \forall g \in \mathcal{G}_{\text{out}}$ return 1 $\text{PreVer}(\text{crs}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]})$ $(\chi_g)_{g \in \mathcal{G}_{\text{out}}} \leftarrow \mathcal{C}$ $\mathbf{x}_g := (x_{1,g}, \dots, x_{k,g}) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, \mathbf{x}_g) \quad \forall g \in \mathcal{G}_{\text{stmt}}$ $\text{ck}_{[k]} \leftarrow \text{PCFC.PreProjVer}(\text{ck}, [k])$ $\text{ck}_g \leftarrow \text{PCFC.PreFuncVer}(\text{ck}, f_{\chi_g}) \quad \forall g \in \mathcal{G}_{\text{out}}$ $\text{vk} := ((\text{com}_g)_{g \in \mathcal{G}_{\text{stmt}}}, \text{ck}_{[k]}, (\text{ck}_g)_{g \in \mathcal{G}_{\text{out}}})$ return vk $\text{EffVer}(\text{vk}, \pi)$ assert $\text{PCFC.EffProjVer}(\text{ck}_{[k]}, \text{com}_g, \pi_g^b) \quad \forall g \in \mathcal{G}_{\text{wit}}$ assert $\text{PCFC.EffFuncVer}(\text{ck}_g, (\text{com}_i)_{i \in T_g}, \text{com}_g, \pi_g^x)$ $\quad \forall g \in \mathcal{G}_{\text{out}}$ return 1
--	---

Fig. 6: Basic Compiler from PCFC to BARG.

Hyb⁰: This is the standard somewhere extractability game (Definition 1) with the algorithms in $\overline{\text{BARG}}$ inlined. To recall, in this game the adversary outputs a proof π and the experiment runs $\mathbf{w}_{i^*} \leftarrow \text{BARG.Ext}(\text{td}, \mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi)$. The adversary wins if the proof verifies yet $\mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}_{i^*}) \neq 1$.

Hyb^{1,j}: $\text{Hyb}^{1,0}$ is identical to Hyb^0 . In $\text{Hyb}^{1,j}$, the game additionally checks that the first j entries of the extracted witness, i.e. those indexed by g belonging to the first j values in $\mathcal{G}_{\text{wit}}$, resides in the message subspace $\bar{\mathcal{M}}$, i.e. $x_g^* \in \bar{\mathcal{M}}^{\ell_w}$, and that they are consistent with the projected commitments, i.e. $\text{PCFC.ProjCom}(\text{td}_{i^*}, x_g^*) = \text{PCFC.Proj}(\text{td}_{i^*}, \text{com}_g)$.

Hyb^{2,j}: $\text{Hyb}^{2,0}$ is identical to Hyb^{1,ℓ_w} . To define the rest of the games, we let $\mathbf{x}^* \in \bar{\mathcal{M}}^{\ell_C}$ be the (honestly computed) values of the circuit wires of instance i^* that result from the evaluation of $\mathcal{C}(\mathbf{x}_{i^*}, \mathbf{w}^*)$. Note that $\mathbf{x}^*$ is guaranteed to satisfy $\mathbf{w}^* \in \bar{\mathcal{M}}^{\ell_w}$ by the winning condition. In $\text{Hyb}^{2,j}$, the game additionally checks that $x_g^* = \text{PCFC.ProjExt}(\text{td}_{i^*}, \text{com}_g)$ for all g belonging to the first j values in $\mathcal{G}_{\text{out}}$.

We now proceed to bound the advantage of any PPT adversary $\mathcal{A}$ against somewhere extractability, i.e. Hyb^0 , by a series of lemmas. We remark that the proof also works for unbounded adversaries (proving statistical somewhere extractability of BARG) if PCFC satisfies (1) statistical functional somewhere extractability and (2) statistical projective somewhere extractability.

```

HybA0(λ):


---


(ck, tdi*) ← PCFC.ProjSetup(1λ, 1k, i*)
(C, {xi}i∈[k], π) ← A(ck)
comg ← PCFC.Com(ck, (x1,g, ..., xk,g))  ∀g ∈ Gstmt
xg* ← PCFC.ProjExt(td, comg)  ∀g ∈ Gwit
w* := (xg*)g∈Gwit
assert comℓc = PCFC.Com(ck, 1k)
assert PCFC.ProjVer(ck, comg, πgb) = 1  ∀g ∈ Gwit
assert PCFC.FuncVer(ck, (comi)i∈Tg, comg, fχg, πgx) = 1  ∀g ∈ Gout
assert C(xi*, w*) ≠ 1
return 1

HybA1,j(λ), 0 ≤ j ≤ ℓw:


---


// identical to HybA0(λ) until before “return 1”
assert xg* ∈ M̄  ∀g ∈ Gwit ∩ [ℓx + j]
assert PCFC.ProjCom(tdi*, xg*) = PCFC.Proj(tdi*, comg)  ∀g ∈ Gwit ∩ [ℓx + j]
return 1

HybA2,j(λ), 0 ≤ j ≤ ℓout:


---


// identical to HybA1,ℓw(λ) until before “return 1”
x* ← Wires(C, xi*, w*)
assert xg* = PCFC.ProjExt(tdi*, comg)  ∀g ∈ Gout ∩ [ℓx,w + j]
return 1

```

Fig. 7: Games Hyb^0 , $\text{Hyb}^{1,j}$, $\text{Hyb}^{2,j}$ for the proof of BARG somewhere extractability. We highlight changes between games. We define the function $\mathbf{x} \leftarrow \text{Wires}(\mathcal{C}, \mathbf{x}_i, \mathbf{w}_i)$ to output the value of all the internal values of a circuit computation $\mathcal{C}(\mathbf{x}_i, \mathbf{w}_i)$. Namely, $x_j \leftarrow \chi_j((x_t)_{t \in T_j})$ where the initial values are given by $\mathbf{x} = \mathbf{x}_i | \mathbf{w}_i | \mathbf{0}$ and $\mathbf{x}$ is recomputed iteratively $\forall j \in [N]$.

Lemma 19. *For any $j \in [\ell_w]$, there exists a PPT adversary $\mathcal{B}$ against the somewhere extractability for $(\mathcal{I}, \bar{\mathcal{M}})$ of PCFC such that*

$$\left| \Pr[\text{Hyb}_{\mathcal{A}}^{1,j-1}(\lambda) = 1] - \Pr[\text{Hyb}_{\mathcal{A}}^{1,j}(\lambda) = 1] \right| \leq \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{se}}(\lambda).$$

Proof. Fix $j \in [\ell_w]$. We build an adversary $\mathcal{B}$ as follows. $\mathcal{B}$ receives input ck from its challenger and runs $\mathcal{A}(\text{ck})$. Then, it parses the proof π of $\mathcal{A}$ and retrieves the tuple $(\text{com}_{\ell_x+j}, \pi_{\ell_x+j}^b)$. Finally, $\mathcal{B}$ simply forwards these to its challenger.

We argue that if $\mathcal{A}$ wins in $\text{Hyb}^{1,j-1}$ but not in $\text{Hyb}^{1,j}$, then $\mathcal{B}$ also wins the game. First, as $\mathcal{A}$ wins in $\text{Hyb}^{1,j}$, it holds that $\text{PCFC.ProjVer}(\text{ck}, \text{com}_{\ell_x+j}, \pi_{\ell_x+j}^b) = 1$. Also, note that $x_{\ell_x+j}^* \leftarrow \text{PCFC.ProjExt}(\text{td}_{i^*}, \text{com}_{\ell_x+j})$. Therefore, all the preconditions for $\mathcal{B}$ in the PC somewhere extractability game are met. Finally, as $\mathcal{A}$ does not win in $\text{Hyb}^{1,j}$, it must be that either $\text{PCFC.Proj}(\text{td}_{i^*}, \text{com}_{\ell_x+j}) \neq \text{PCFC.ProjCom}(\text{td}_{i^*}, x_{\ell_x+j}^*)$ or $x_{\ell_x+j}^* \notin \bar{\mathcal{M}}$. $\square$

Lemma 20. *For any $j \in [\ell_{\text{out}}]$, there exists a PPT adversary $\mathcal{B}$ against the functional extractability for $(\mathcal{F}, \mathcal{I})$ of PCFC such that*

$$\left| \Pr[\text{Hyb}_{\mathcal{A}}^{2,j-1}(\lambda) = 1] - \Pr[\text{Hyb}_{\mathcal{A}}^{2,j}(\lambda) = 1] \right| \leq \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{fe}}(\lambda).$$

Proof. Fix $j \in [\ell_{\text{out}}]$. We build $\mathcal{B}$ from $\mathcal{A}$ as follows:

- Receives input ck from its challenger and runs $(\mathcal{C}, \{\mathbf{x}_i\}_{i \in [k]}, \pi) \leftarrow \mathcal{A}(\text{ck})$.
- Parses $\mathcal{C}$ to obtain the gate function χ_j and the input wire set T_j .
- Parses the proof π and obtains $\{\text{com}_g\}_{g \in T_j \setminus [\ell_{\mathbf{x}}]}, \pi_j^{\chi}, \text{com}_j$.
- Outputs $(f_{\chi_j}, \{\text{com}_g\}_{g \in T_j}, \text{com}_j, \pi_j^{\chi})$ where the com_g for $g \in \mathcal{G}_{\text{stmt}} \cap T_j$ are computed honestly as $\text{com}_g \leftarrow \text{PCFC.Com}(\text{ck}, (x_{1,g}, \dots, x_{k,g}))$.

We argue that if $\mathcal{A}$ wins in $\text{Hyb}^{2,j-1}$ but not in $\text{Hyb}^{2,j}$, then $\mathcal{B}$ is a successful adversary against CFC functional extractability.

- By the winning condition (inherited from Hyb^0), it holds that the functional proof verifies, $\text{FuncVer}(\text{ck}, (\text{com}_g)_{g \in T_j}, f_{\chi_j}, \text{com}_j, \pi_j^{\chi}) = 1$.
- Let $\tilde{x}_g \leftarrow \text{PCFC.ProjExt}(\text{td}_{i^*}, \text{com}_g)$ be the value extracted from the commitments for any $g \in \mathcal{G}$.
- As $\mathcal{A}$ wins in $\text{Hyb}^{2,j-1}$, we have that $\tilde{x}_g = x_g^* = \chi_j((x_g^*)_{g \in T_j})$ for every $g \in [\ell_{\mathbf{x}, \mathbf{w}} + j - 1]$, and therefore for every $g \in T_j$. By the definition of $\mathbf{x}^*$, it also follows that $\tilde{x}_g \in \mathcal{M}$ for every $g \in T_j$.
- As $\mathcal{A}$ does not win in $\text{Hyb}^{2,j}$, then $\tilde{x}_j \neq x_j^*$.

Hence, $\chi_j((\tilde{x}_g)_{g \in T_j}) \neq \tilde{x}_j$ and the output of $\mathcal{B}$ is a successful break of CFC functional extractability. $\square$

Note that the advantage of $\mathcal{A}$ in $\text{Hyb}^{2, \ell_{\text{out}}}$ is zero, as the condition $\mathcal{C}(\mathbf{x}_{i^*}, w^*) \neq 1$ is incompatible with $x_{\ell_{\mathbf{C}}}^* = \text{PCFC.ProjExt}(\text{td}_{i^*}, \text{com}_{\ell_{\mathbf{C}}}) = 1$, which holds as $\text{com}_{\ell_{\mathbf{C}}}$ is computed honestly by the verifier. We collect the bounds of Lemma 19 and Lemma 20, and conclude that the advantage of $\mathcal{A}$ in the original Hyb^0 is bounded by

$$\Pr[\text{Hyb}_{\mathcal{A}}^0(\lambda) = 1] \leq \ell_{\mathbf{w}} \cdot \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{sc}}(\lambda) + \ell_{\text{out}} \cdot \text{Adv}_{\text{PCFC}, \mathcal{B}}^{\text{fe}}(\lambda). \quad \square$$

A.3 Proof of efficient verification

Lemma 21. *Let PCFC and BARG be as defined in Theorem 4. If further PCFC has efficient verification with preprocessing (Definition 15), then BARG has efficient verification with preprocessing (Definition 2).*

Proof. As per Definition 2, it suffices to show that $|\mathbf{vk}|$ and the runtime of $\text{BARG.EffVer}(\lambda, \mathbf{vk})$ are at most $\text{poly}(\lambda, \log k, \ell_{\mathbf{C}})$. We actually prove below a stronger claim that these quantities are at most $\text{poly}(\lambda, \log k, \log \ell_{\mathbf{C}}) \cdot \ell_{\mathbf{C}}$.

We first examine $|\mathbf{vk}|$. Recall that $\mathbf{vk} = ((\text{com}_g)_{g \in \mathcal{G}_{\text{stmt}}}, \text{ck}_{[k]}, (\text{ck}_g)_{g \in \mathcal{G}_{\text{out}}})$. Note that by Definition 5 fresh commitments are of size at most $\text{poly}(\lambda, \log k)$ and by Definition 6 aggregated commitments are identical to fresh ones. Therefore, $|\text{com}_g| \leq \text{poly}(\lambda, \log k)$. Next, by Definition 9 it holds that $|\text{ck}_g| \leq \text{poly}(\lambda, \log k, \log m, \log |\chi_g|)$ where $m = |T_g| \leq \ell_{\mathbf{C}}$ in the current context and $|\chi_g| \leq \ell_{\mathbf{C}}$. Thus, $|\text{ck}_g| \leq \text{poly}(\lambda, \log k, \log \ell_{\mathbf{C}})$. Finally, by Definition 13 it holds that $|\text{ck}_{[k]}| \leq \text{poly}(\lambda, \log k)$. Putting everything together, we conclude that $|\mathbf{vk}| \leq \text{poly}(\lambda, \log k, \log \ell_{\mathbf{C}})$.

For the runtime of $\text{BARG.EffVer}(\lambda, \mathbf{vk})$, by Definition 6, each of the $\ell_{\mathbf{w}}$ calls to PCFC.EffProjVer runs in time at most $\text{poly}(\lambda, \log k)$ according to Definition 13. Also, each of the $|\mathcal{G}_{\text{out}}| \leq \ell_{\mathcal{C}}$ calls to PCFC.EffFuncVer runs in time at most $\text{poly}(\lambda, \log k, \log m, \log |\chi_g|)$ which as analysed above is at most $\text{poly}(\lambda, \log k, \log \ell_{\mathcal{C}})$. We thus conclude that $\text{BARG.EffVer}(\lambda, \mathbf{vk})$ runs in time at most $\text{poly}(\lambda, \log k, \log \ell_{\mathcal{C}}) \cdot \ell_{\mathcal{C}}$. $\square$

Note: in precomputation we could fix all wires of the statements corresponding to gates (when precomputing the function) by modifying the gates directly. The circuit would simply become a single-input $\mathcal{C}(\mathbf{w})$.

B Algebraic Interpretation of the Circuit-Succinct Compiler

Let $\mathcal{C}$ be the boolean circuit of the NP relation proven in a BARG, let $\ell_{\mathcal{C}}$ be the circuit size, and let k be the number of instances. The algebraic BARG underlying Corollary 1 and the proof of security can be intuitively seen as follows.

1. At setup, generate a PC commitment key of size $\ell = \ell_{\mathcal{C}} \cdot k$. In trapdoor mode for instance i^* , set the extraction set $I = \{(i^* - 1) \cdot \ell_{\mathcal{C}} + 1, \dots, i^* \cdot \ell_{\mathcal{C}}\}$.
2. Arrange the multiple BARG computation traces $\mathbf{x}_i \in \{0, 1\}^{\ell_{\mathcal{C}}}$ in rows in a matrix $\mathbf{X} \in \{0, 1\}^{k \times \ell_{\mathcal{C}}}$. Recall that the first $\ell_{\mathbf{x}, \mathbf{w}}$ coordinates of the $\mathbf{x}_i$ correspond to $\mathbf{x}_i|_{\mathbf{w}_i}$ and that the last coordinate corresponds to y_i (see Table 1 for a visualization of $\mathbf{X}$). Note that I corresponds exactly to the i^* -th row of $\mathbf{X}$.
3. Commit to each position of the inputs $\mathbf{x}_i|_{\mathbf{w}_i}$ across all instances of the batch. Graphically, this is equivalent to committing to the columns $\mathbf{x}_j$ of $\mathbf{X}$ corresponding to witnesses (i.e., $j \in [\ell_{\mathbf{x}, \mathbf{w}}]$). We obtain $\ell_{\mathbf{x}, \mathbf{w}}$ commitments $[\hat{\mathbf{c}}_j]_2$. Note that the intersection between I and the j -th column of $\mathbf{X}$ corresponds exactly to the $x_{i^*, j}$ element.
4. Prove the binariness of the values committed in $[\hat{\mathbf{c}}_j]_2$ corresponding to witness wires, i.e., for $j \in \{\ell_{\mathbf{x}} + 1, \dots, \ell_{\mathbf{x}, \mathbf{w}}\}$. Due to the somewhere extractability of the projection proofs, we can extract $\mathbf{w}_{i^*, j}$ from each $[\hat{\mathbf{c}}_j]_2$. Moreover, we know that their projective subspace commitment $[\hat{\mathbf{d}}_j]_2$ commits exactly to $\mathbf{w}_{i^*, j}$. Proving binariness for statement wires is not needed as these commitments are recomputed by the verifier.
5. Aggregate every input commitment $[\hat{\mathbf{c}}_j]_2$ into a commitment $[\hat{\mathbf{c}}]_2 = \sum_{j \in [\ell_{\mathbf{x}, \mathbf{w}}]} [\hat{\mathbf{c}}_j]_2$. The resulting commitment corresponds to the first $\ell_{\mathbf{x}, \mathbf{w}}$ rows of $\mathbf{X}$. Moreover, due to the algebraic nature of the aggregation, its projective commitment $[\hat{\mathbf{d}}]_2$ commits exactly to $\mathbf{x}_{i^*}|_{\mathbf{w}_{i^*}}$. Due to the structure of the commitment keys, $[\hat{\mathbf{c}}]_2$ can be seen as a commitment to the following vector of length ℓ , if the whole commitment key $[\hat{\mathbf{T}}]_2$ is used:

$$[\hat{\mathbf{c}}]_2 = [\hat{\mathbf{T}}]_2 \cdot [\mathbf{x}_1|_{\mathbf{w}_1}|_{\mathbf{0}_{\ell_{\mathcal{C}} - \ell_{\mathbf{x}, \mathbf{w}}}} \cdots \mathbf{x}_k|_{\mathbf{w}_k}|_{\mathbf{0}_{\ell_{\mathcal{C}} - \ell_{\mathbf{x}, \mathbf{w}}}}]^T,$$

where the i^* -th block corresponds to the extraction set I .

6. Prove the trace of the computation of $\mathcal{C}$ using the CFC proof, where the CFC function is defined as k parallel copies of $\mathcal{C}$. The input commitment is $[\hat{\mathbf{c}}]_2$, and the output commitment $[\hat{\mathbf{c}}_y]_2$ commits to the vector of ones $\mathbf{1}_k$. Internally, the CFC proof ensures that after evaluating each of the copies of $\mathcal{C}$, we obtain a commitment to

$$[\mathbf{x}_1|_{\mathbf{w}_1}|\mathbf{x}_1|_1 \cdots \mathbf{x}_k|_{\mathbf{w}_k}|\mathbf{x}_k|_1]^T.$$

Then, we project the last coordinate of each block to obtain the commitment $[\hat{\mathbf{c}}_y]_2$ to $\mathbf{1}_k$. Security follows from the projective chain binding property of the PCFC as detailed in Lemma 4.

C Additional descriptions of the WW24 CFC

WW24 functional openings. Given an arithmetic circuit $\mathcal{C} : \mathbb{F}^\ell \rightarrow \mathbb{F}^{\ell_y}$, the CFC algorithms $\text{FuncProve}(\text{ck}, \mathbf{x} \in \mathbb{F}^\ell, \mathcal{C})$ and $\text{FuncVer}(\text{ck}, \text{com}, \text{com}_y, \mathcal{C}, \pi)$ work as follows (note that com, com_y are standard type-I commitments):

- Evaluate $\mathcal{C}(\mathbf{x})$ and commit to the entire computation trace $\mathbf{z} \in \mathbb{F}_\mathcal{C}^\ell$ on both keys, obtaining σ_1, σ_2 . Note that $\mathbf{z} = \mathbf{x} | \mathbf{z}_\mathcal{C} | \mathbf{y}$ where $\mathbf{z}_\mathcal{C} \in \mathbb{F}^{\ell_\mathcal{C} - \ell - \ell_y}$.
- Compute a subvector proof for $\mathbf{x}$ on $\mathbf{z}$ as $\pi_{\text{Pre}} \leftarrow \text{OpenPre}(\text{ck}, \mathbf{x}, \mathbf{z})$, where ck has been extended via $\text{ck} \leftarrow \text{SetupPre}(\text{ck}_{\text{base}}, [\ell])$. This proof is verified as $\text{VerPre}(\text{ck}, \text{com}, \sigma_1, \pi_{\text{Pre}})$.
- Compute a linear proof for consistency between σ_1, σ_2 as $\pi_{\text{Lin}} \leftarrow \text{OpenLin}(\text{ck}, \mathbf{z}, \text{id})$ where $\text{id} : \mathbb{F}_\mathcal{C}^\ell \rightarrow \mathbb{F}_\mathcal{C}^\ell$ is the identity function. This proof is verified as $\text{VerLin}(\text{ck}, \sigma_1, \sigma_2, \text{id}, \pi_{\text{Lin}})$.
- Compute a quadratic gate proof as $\pi_{\text{Quad}} \leftarrow \text{OpenQuad}(\text{ck}, \mathbf{z}, \chi_\mathcal{C})$ where $\chi_\mathcal{C} : \mathbb{F}_\mathcal{C}^\ell \rightarrow \mathbb{F}_\mathcal{C}^\ell$ is the so-called “next-wire” function. Following the notation of Remark 8, $\chi_\mathcal{C}$ is a $([j-1], [j])$ -separable function for every $j \in [\ell_\mathcal{C}]$ where the j -th output is the result of evaluating the j -th gate function of $\mathcal{C}$, i.e., $y_j \leftarrow \chi_j(x_1, \dots, x_{j-1})$. This proof is verified as $\text{VerQuad}(\text{ck}, \sigma_2, \sigma_1, \chi_\mathcal{C}, \pi_{\text{Quad}})$.
- Compute an output consistency proof, given by $\pi_{\text{out}} \leftarrow \text{OpenQuad}(\text{ck}, \mathbf{z}, p)$ where $p : \mathbb{F}_\mathcal{C}^\ell \rightarrow \mathbb{F}_\mathcal{C}^\ell$ is given by $p(\mathbf{z}) = (z_{\ell_y+1}, \dots, z_{\ell_\mathcal{C}}, \mathbf{0}_{\ell_\mathcal{C}-\ell_y})$. This function projects the output of the circuit $\mathbf{y} = \mathcal{C}(\mathbf{x})$ into the first ℓ coordinates, and is a $([j+\ell_\mathcal{C}-\ell_y], [j])$ -separable function for every $j \in [\ell_y]$.¹⁹ This proof is verified as $\text{VerQuad}(\text{ck}, \sigma_2, \text{com}_y, p, \pi_{\text{out}})$.

CFC evaluation binding (Definition 8) follows by a complex sliding-window hybrid argument, which presents similarities with the proof of Theorem 3.

¹⁹ In the original scheme, this consistency proof is simply a linear projection from σ_1 to a type-II commitment to $\mathbf{y}$ computed by the verifier. Here, we introduce the quadratic consistency proof for chainability, namely to ensure that the output commitment is a well-formed type-I commitment.